

Multi-Buffered Serial Peripheral Interface Module (MibSPI) with Parallel Pin Option (MibSPIP)

This chapter provides the specifications for a 16-bit configurable synchronous multi-buffered multi-pin serial peripheral interface (MibSPI). This chapter also provides the specifications for MibSPI with Parallel Pin Option (MibSPIP). The MibSPI is a programmable-length shift register used for high-speed communication between external peripherals or other microcontrollers.

Throughout this chapter, all references to SPI also apply to MibSPI/MibSPIP, unless otherwise noted.

NOTE: This chapter describes a superset implementation of the MibSPI/SPI modules that includes features and functionality that may not be available on some devices. Device-specific content that should be determined by referencing the datasheet includes DMA functionality, MibSPI RAM size, number of transfer groups, number of chip selects, parallel mode support, and availability of 5-pin operation (SPIENA).

Topic	Page
27.1 Overview.....	1402
27.2 Operating Modes	1403
27.3 Test Features	1424
27.4 General-Purpose I/O	1426
27.5 Low-Power Mode	1426
27.6 Interrupts	1427
27.7 DMA Interface.....	1429
27.8 Module Configuration	1430
27.9 Control Registers.....	1432
27.10 Multi-Buffer RAM	1498
27.11 Parity Memory	1505
27.12 MibSPI Pin Timing Parameters	1508

27.1 Overview

The MibSPI/MibSPIP is a high-speed synchronous serial input/output port that allows a serial bit stream of programmed length (2 to 16 bits) to be shifted in and out of the device at a programmed bit-transfer rate. Typical applications for the SPI include interfacing to external peripherals, such as I/Os, memories, display drivers, and analog-to-digital converters.

The SPI has the following attributes:

- 16-bit shift register
- Receive buffer register
- 11-bit baud clock generator
- Serial clock (SPICLK) pin
- 1 SPISOMI/SPISIMO pin for data transfer, with programmable pin direction
- SPI enable ($\overline{\text{SPIENA}}$) pin
- Up to 6 slave chip select ($\overline{\text{SPICS}}$) pins
- SPICLK can be internally-generated (and driven) or received from an external clock source
- Each word transferred can have a unique format
- SPI pins can be used as functional or digital Input/Output pins (GIOs)

NOTE: SIMO - Slave In Master Out Pin
SOMI - Slave Out Master In Pin
SPICS - SPI Chip Select Pin
SPIENA - SPI Enable Pin

27.1.1 Word Format Options

Each word transferred can have a unique format. Several format characteristics are programmable for each word transferred:

- SPICLK frequency
- Character length (2 to 16 bits)
- Phase (with and without delay)
- Polarity (high or low)
- Parity enabled/disabled
- Chip Select(CS) timers for setup and hold
- Shift direction (Most-Significant Bit (MSB) first or Least-Significant Bit (LSB) first)
- Multi-pin parallel modes

27.1.2 Multi-buffering (Mib) Support

The MibSPI has a programmable buffer memory that enables programmed transmission to be completed without CPU intervention. The buffers are combined in different Transfer Groups (TGs) that can be triggered by external events (timers, Input/Output activity, and so on) or by the internal tick counter. The internal tick counter supports periodic trigger events. Each buffer of the MibSPI can be associated with different DMA channels in different TGs, allowing the user to move data between internal memory and an external slave with minimal CPU interaction.

27.1.2.1 Multi-buffer Mode

Multi-buffer Mode is an extension to the SPI. In multi-buffer mode, many extended features are configurable:

- Number of buffers for each peripheral (or data source/destination, up to 128 buffers supported) or group (up to 8 groupings)
- Triggers for each groups, trigger types, trigger sources for individual groups (14 external trigger sources and 1 internal trigger source supported)
- Memory fault detection via an internal parity circuit
- Number of DMA-controlled buffers and number of DMA request channels (up to 8 for each of transmit and receive)
- Number of DMA transfers for each buffer (up to 65536 words for up to 8 buffers)
- Uninterrupted DMA buffer transfer (NOBREAK buffer)

27.1.2.2 Compatibility Mode

Compatibility Mode of the MibSPI makes it behave exactly like a standard platform SPI module and ensures full compatibility with other SPIs. All features in compatibility mode of the MibSPI are directly applicable to a SPI. Multi-buffer Mode features are not available in Compatibility Mode.

NOTE: The SPIDAT0 register is not accessible in the multi-buffer mode of MibSPI. It is only accessible in compatibility mode.

27.1.3 Transmission Lock (Multi-Buffer Mode Master Only)

Some slave devices require transmission of a command followed by data. In this case the SPI transaction should not be interrupted by another group transfer. The LOCK bit within each buffer allows a consecutive transfer to happen without being interrupted by another higher-priority group transfer.

27.2 Operating Modes

The SPI can be configured via software to operate as either a master or a slave. The MASTER bit (SPIGCR1[0]) selects the configuration of the SPISIMO and SPISOMI pins. CLKMOD bit (SPIGCR1[1]) determines whether an internal or external clock source will be used.

The slave chip select ($\overline{\text{SPIC}}\text{S}$) pins are used when communicating with multiple slave devices or, with a single slave, to delimit messages containing a leading register address. When a write occurs to SPIDAT1 in master mode, the $\overline{\text{SPIC}}\text{S}$ pins are automatically driven to select the specified slave.

Handshaking mechanism, provided by the $\overline{\text{SPIEN}}\text{A}$ pin, enables a slave SPI to delay the generation of the clock signal supplied by the master if it is not prepared for the next exchange of data.

NOTE: If in the slave mode of operation and configured in either 3-pin or 4-pin (without $\overline{\text{SPIEN}}\text{A}$) modes, there must be a minimum of 8 VCLK cycles of delay between the last SPICLK and the start of the SPICLK for the next buffer transmit. In general, this equates to a VCLK/SPICLK ratio of ≤ 16 requiring a minimum of 1 SPICLK delay between transmissions.

27.2.1 Pin Configurations

The SPI supports data connections as shown in [Table 27-1](#).

NOTE:

1. When the SPICS signals are disabled, the chip select field in the transmit data is not used.
2. When the $\overline{\text{SPIENA}}$ signal is disabled, the $\overline{\text{SPIENA}}$ pin is ignored in master mode, and not driven as part of the SPI transaction in slave mode.

Table 27-1. Pin Configurations

Pin	Master Mode		Slave Mode	
SPICLK	Drives the clock to external devices		Receives the clock from the external master	
SPISOMI	Receives data from the external slave		Sends data to the external master	
SPISIMO	Transmits data to the external slave		Receives data from the external master	
$\overline{\text{SPIENA}}$	$\overline{\text{SPIENA}}$ disabled: GIO	$\overline{\text{SPIENA}}$ enabled: Receives ENA signal from the external slave	$\overline{\text{SPIENA}}$ disabled: GIO	$\overline{\text{SPIENA}}$ enabled: Drives ENA signal from the external master
SPICS	SPICS disabled: GIO	SPICS enabled: Selects one or more slave devices	SPICS disabled: GIO	SPICS enabled: Receives the CS signal from the external master

27.2.2 Data Handling

[Figure 27-1](#) shows the SPI transaction hardware. TXBUF and RXBUF are internal buffers that are intended to improve the overall throughput of data transfer. TXBUF is a transmit buffer, while RXBUF is a receive buffer.

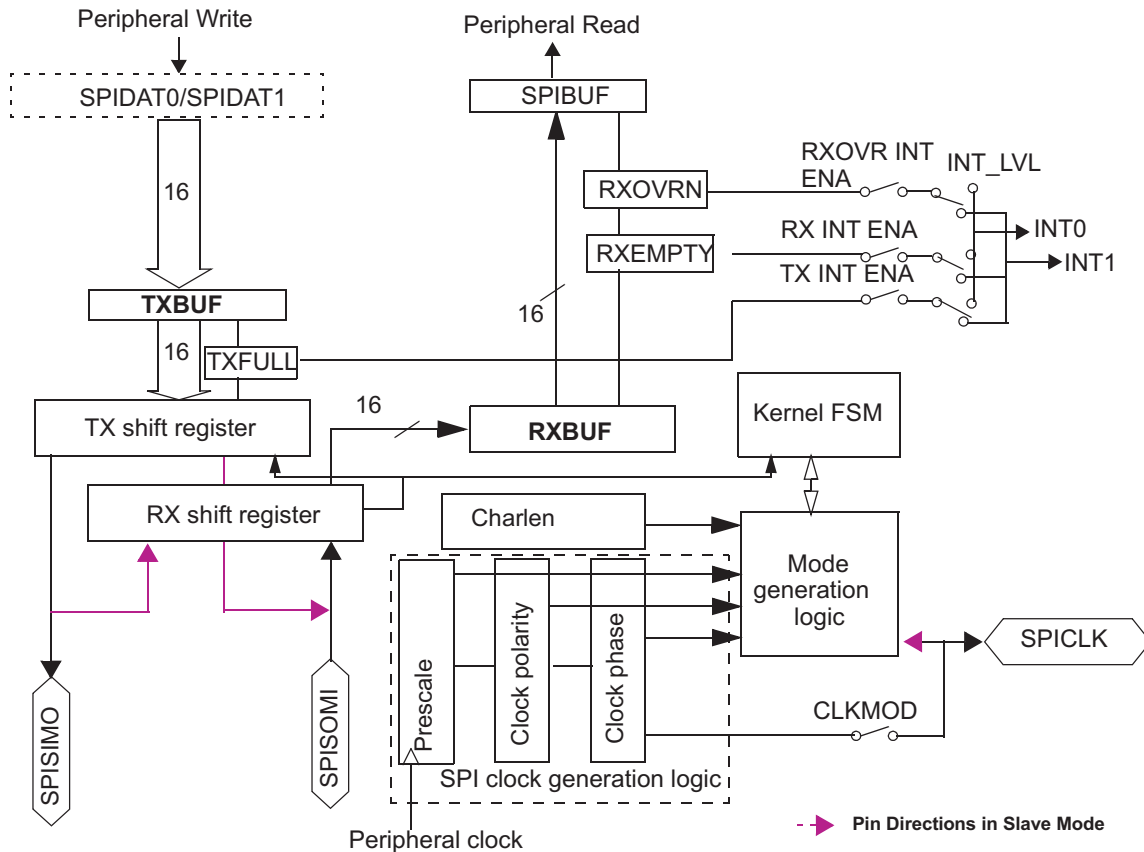
27.2.2.1 Data Sequencing when SPIDAT0 or SPIDAT1 is Written

- If both the TX shift register and TXBUF are empty, then the data is directly copied to the TX shift register. For devices with DMA, if DMA is enabled, a transmit DMA request (TX_DMA_REQ) is generated to cause the next word to be fetched. If transmit interrupts are enabled, a transmitter-empty interrupt is generated.
- If the TX shift register is already full or is in the process of shifting and if TXBUF is empty then the data written to SPIDAT0 / SPIDAT1 is copied to TXBUF and TXFULL flag is set to 1 at the same time.
- When a shift operation is complete, data from the TXBUF (if it is full) is copied into TX shift register and the TXFULL flag is cleared to 0 to indicate that next data can be fetched. A transmit DMA request (if enabled) or a transmitter-empty interrupt (if enabled) is generated at the same time.

27.2.2.2 Data Sequencing when All Bits Shifted into RXSHIFT Register

- If both SPIBUF and RXBUF are empty, the received data in RX shift register is directly copied into SPIBUF and the receive DMA request (if enabled) is generated and the receive-interrupt (if enabled) is generated. The RXEMPTY flag in SPIBUF is cleared at the same time.
- If SPIBUF is already full at the end of receive completion, the RX shift register contents is copied to RXBUF. A receive DMA request is generated, if enabled. The receive complete interrupt line remains high.
- If SPIBUF is read by the CPU or DMA and if RXBUF is full, then the contents of RXBUF are copied to SPIBUF as soon as SPIBUF is read. RXEMPTY flag remains cleared, indicating that SPIBUF is still full.
- If both SPIBUF and RXBUF are full, then RXBUF will be overwritten and the RXOVR interrupt flag is set and an interrupt is generated, if enabled.

Figure 27-1. SPI Functional Logic Diagram



- 1 This is a representative diagram, which shows three-pin mode hardware.
- 2 TXBUF, RXBUF, and SHIFT_REGISTER are user-invisible registers.
- 3 SPIDAT0 and SPIDAT1 are user-visible, and are physically mapped to the contents of TXBUF.
- 4 SPISIMO, SPISOMI, SPICLK pin directions depend on the Master or Slave Mode.

27.2.2.3 Three-Pin Mode

In master mode configuration (MASTER = 1 and CLKMOD = 1), the SPI provides the serial clock on the SPICLK pin. Data is transmitted on the SPISIMO pin and received on the SPISOMI pin (see [Figure 27-2](#)).

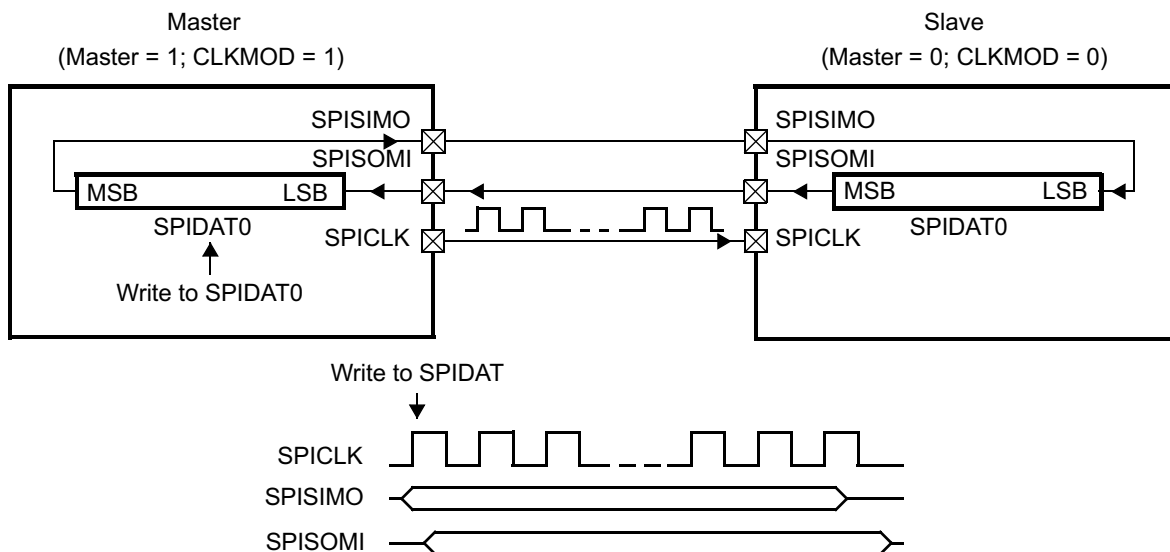
Data written to the shift register (SPIDAT0 / SPIDAT1) initiates data transmission on the SPISIMO pin, MSB first. Simultaneously, received data is shifted through the SPISOMI pin into the LSB of the SPIDAT0 register. When the selected number of bits have been transmitted, the received data in the shift register is transferred to the SPIBUF register for the CPU to read. Data is stored right-justified in SPIBUF.

See [Section 27.2.2.1](#) and [Section 27.2.2.2](#) for details about the data handling for transmit and receive operations.

In slave mode configuration (MASTER = 0 and CLKMOD = 0), data shifts out on the SPISOMI pin and in on the SPISIMO pin. The SPICLK pin is used as the input for the serial shift clock, which is supplied from the external network master. The transfer rate is defined by this clock.

Data written to the SPIDAT0 or SPIDAT1 register is transmitted to the network when the SPICLK signal is received from the network master. To receive data, the SPI waits for the network master to send the SPICLK signal and then shifts data on the SPISIMO pin into the RX shift register. If data is to be transmitted by the slave simultaneously, it must be written to the SPIDAT0 or SPIDAT1 register before the beginning of the SPICLK signal.

Figure 27-2. SPI Three-Pin Operation



27.2.3 Operation with $\overline{\text{SPICS}}$

In master mode, each chip select signal is used to select a specific slave. In slave mode, the chip select signal is used to enable and disable the transfer. Chip-select functionality is enabled by setting one of the $\overline{\text{SPICS}}$ pins as a chip select. It is disabled by setting all $\overline{\text{SPICS}}$ pins as GPIOs in SPIPC0.

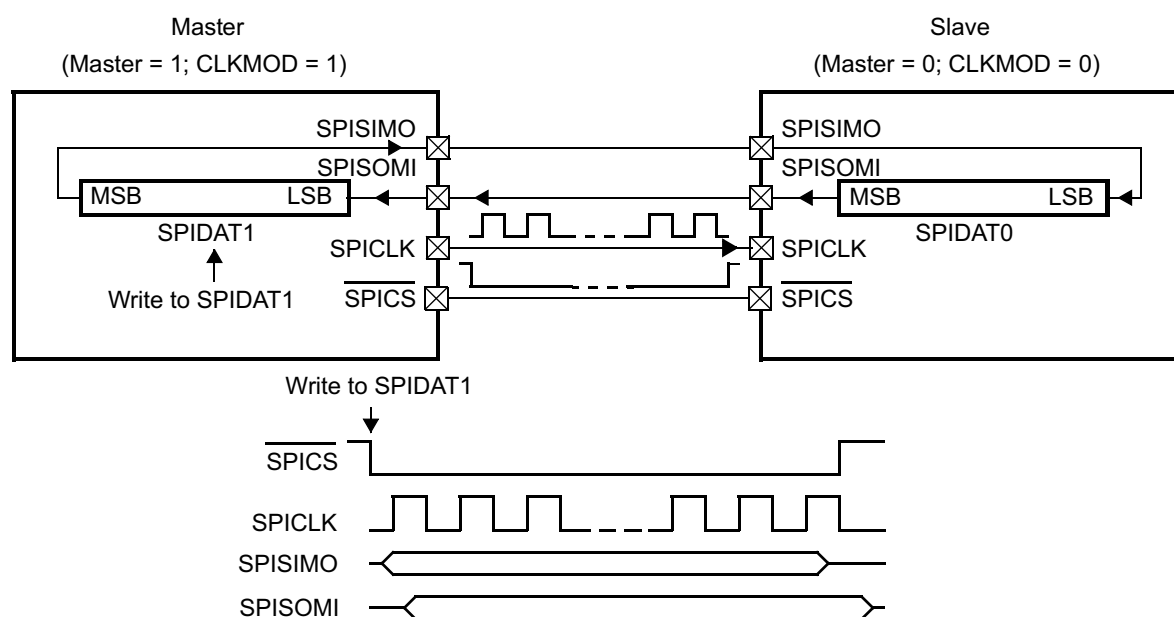
27.2.3.1 Multiple Chip Selects

The $\overline{\text{SPICS}}$ pins that are used must be configured as functional pins in the SPIPC0 register. The default pattern to be put on the $\overline{\text{SPICS}}$ when all the slaves are deactivated is set in the SPIDEF register. This pattern allows different slaves with different chip-select polarity to be activated by the SPI.

The master-mode SPI is capable of driving either 0 or 1 as the active value for any $\overline{\text{SPICS}}$ output pin. The drive state for the $\overline{\text{SPICS}}$ pins is controlled by the CSNR field of SPIDAT1. The pattern that is driven will select the slave to which the transmission is dedicated.

In slave mode, the SPI can only be selected by an active value of 0 on any of its selected $\overline{\text{SPICS}}$ input pins.

Figure 27-3. Operation with $\overline{\text{SPICS}}$



27.2.4 Operation with $\overline{\text{SPIENA}}$

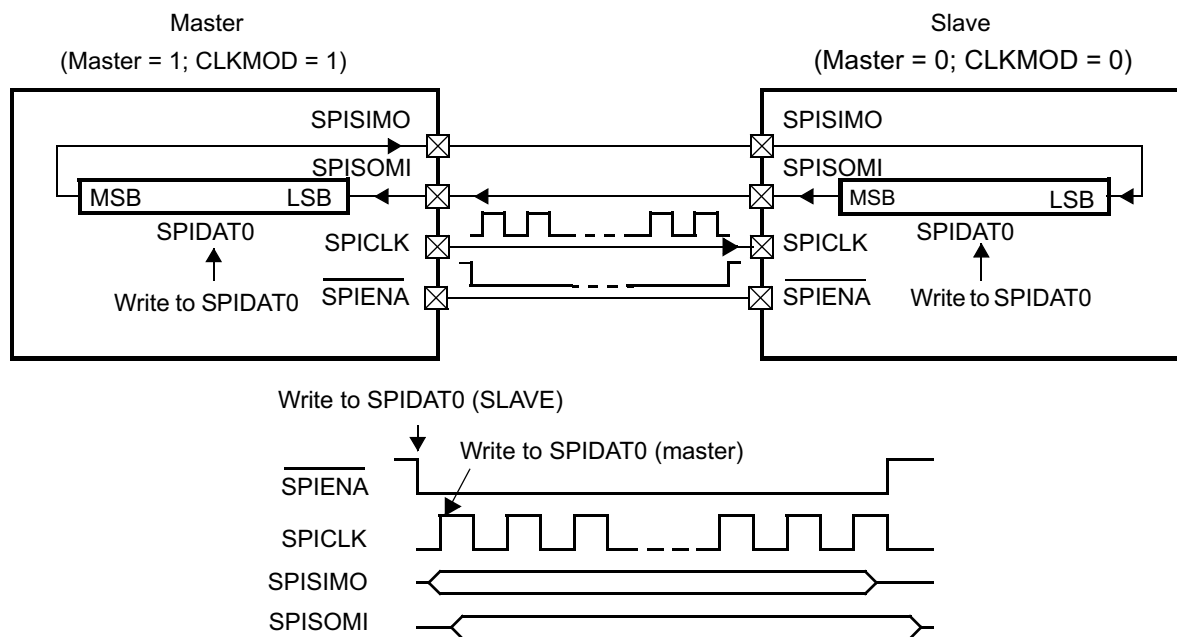
The $\overline{\text{SPIENA}}$ operates as a WAIT signal pin. For both the slave and the master, the $\overline{\text{SPIENA}}$ pin must be configured to be functional (SPIPC0[8] = 1). In this mode, an active-low signal from the slave on the $\overline{\text{SPIENA}}$ pin allows the master SPI to drive the clock pulse stream. A high signal tells the master to hold the clock signal (and delay SPI activity).

If the $\overline{\text{SPIENA}}$ pin is in high-impedance mode (ENABLE_HIGHZ = 1), the slave will put $\overline{\text{SPIENA}}$ into the high-impedance once it completes receiving a new character. If the $\overline{\text{SPIENA}}$ pin is in push-pull mode (ENABLE_HIGHZ = 0), the slave will drive $\overline{\text{SPIENA}}$ to 1 once it completes receiving a new character. The slave will drive $\overline{\text{SPIENA}}$ low again for the next word to transfer, after new data is written to the slave TX shift register.

In master mode (CLKMOD = 1), if the $\overline{\text{SPIENA}}$ pin is configured as functional, then the pin acts as an input pin. If configured as a slave SPI and as functional, the $\overline{\text{SPIENA}}$ pin acts as an output pin.

NOTE: During a transfer, if a slave-mode SPI detects a deassertion of its chip select before its internal character length counter overflows, then it places SPISOMI and $\overline{\text{SPIENA}}$ (if ENABLE_HIGHZ bit is set to 1) in high-impedance mode. Once this condition has occurred, if a SPICLK edge is detected while the chip select is deasserted, then the SPI stops that transfer and sets an DLENERR error flag and generates an interrupt (if enabled).

Figure 27-4. Operation with $\overline{\text{SPIENA}}$



27.2.5 Five-Pin Operation (Hardware Handshaking)

Five-pin operation combines the functionality of three-pin mode, plus the enable pin and one or more chip select pins. The result is full hardware handshaking. To use this mode, both the $\overline{\text{SPIENA}}$ pin and the required number of $\overline{\text{SPICS}}$ pins must be configured as functional pins.

If the $\overline{\text{SPIENA}}$ pin is in high-impedance mode ($\text{ENABLE_HIGHZ} = 1$), the slave SPI will put this signal into the high-impedance state by default. The slave will drive the signal $\overline{\text{SPIENA}}$ low when new data is written to the slave shift register and the slave has been selected by the master ($\overline{\text{SPICS}}$ is low).

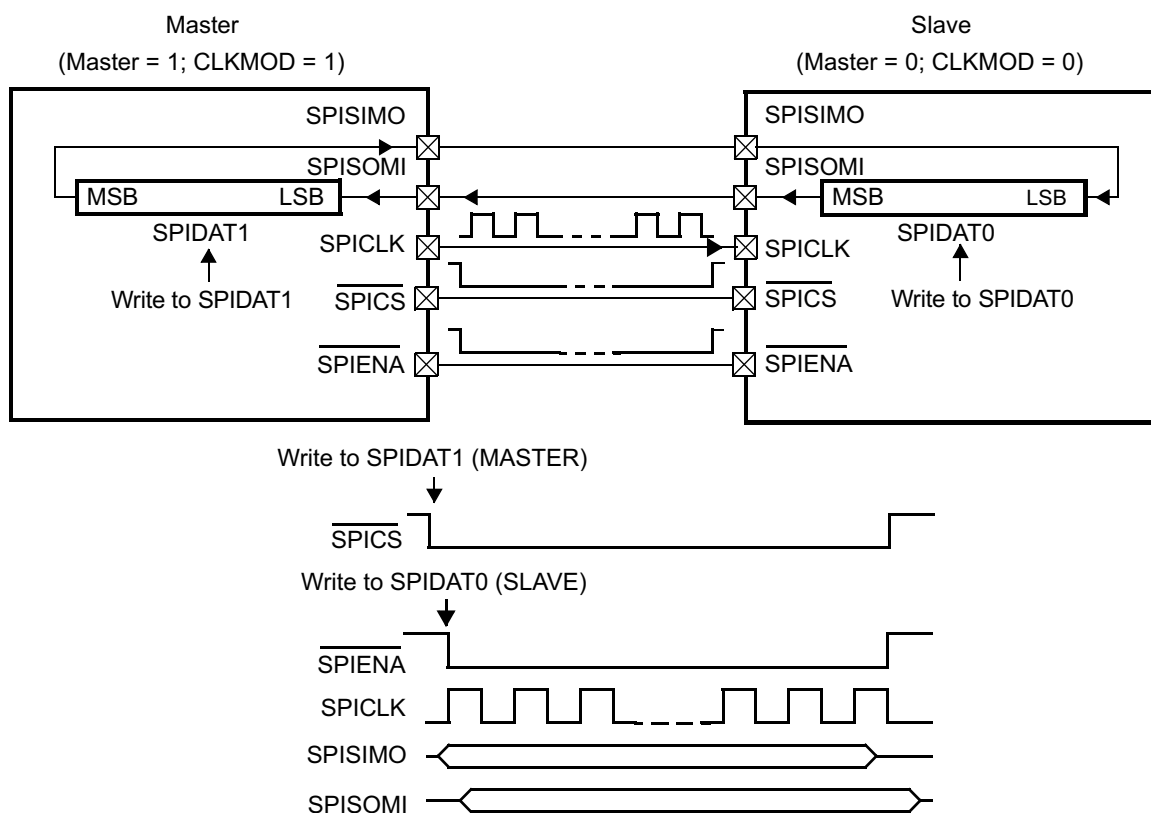
If the $\overline{\text{SPIENA}}$ pin is in push-pull mode ($\text{ENABLE_HIGHZ} = 0$), the slave SPI drives this pin high by default when it is in functional mode. The slave SPI will drive the $\overline{\text{SPIENA}}$ signal low when new data is written to the slave shift register (SPIDAT0/SPIDAT1) and the slave is selected by the master ($\overline{\text{SPICS}}$ is low). If the slave is deselected by the master ($\overline{\text{SPICS}}$ goes high), the slave $\overline{\text{SPIENA}}$ signal is driven high.

NOTE: Push-pull mode of the $\overline{\text{SPIENA}}$ pin can be used only when there is a single slave in the system. When multiple SPI slave devices are connected to the common $\overline{\text{SPIENA}}$ pin, all of the slaves should configure their $\overline{\text{SPIENA}}$ pins in high-impedance mode.

In master mode, if the $\overline{\text{SPICS}}$ pins are configured as functional pins, then the pins will be in output mode. A write to the master's SPIDAT1/SPIDAT0 register will automatically drive the $\overline{\text{SPICS}}$ signals low. The master will drive the $\overline{\text{SPICS}}$ signals high again after completing the transfer of the bits of the data.

In slave mode ($\text{CLKMOD} = 0$), the $\overline{\text{SPICS}}$ pins act as SPI functional inputs.

Figure 27-5. SPI Five-Pin Option with $\overline{\text{SPIENA}}$ and $\overline{\text{SPICS}}$



27.2.6 Data Formats

To support multiple different types of slaves in one SPI network, four independent data word formats are implemented that allow configuration of individual data word length, polarity, phase, and bit rate. Each word transmitted can select which data format to use via the bits DFSEL[1:0] in its control field from one of the four data word formats. Same data format can be supported on multiple chip selects.

Data formats 0, 1, 2, and 3 can be configured through SPIFMTx control registers.

Each SPI data format includes the standard SPI data format with enhanced features:

- Individually-configurable shift direction can be used to select MSB first or LSB first, whereas the position of the MSB depends on the configured data word length.
- Receive data is automatically right-aligned, independent of shift direction and data word length. Transmit data has to be written right-aligned into the SPI and the internal shift register will transmit according to the selected shift direction and data word length for correct transfer.
- To increase fault detection of data transmission and reception, an odd or even parity bit can be added at the end of a data word. The parity generator can be enabled or disabled individually for each data format. If a received parity bit does not match with the locally calculated parity bit, the parity error flag (PARITYERR) is set and an interrupt is asserted (if enabled).

Since the master-mode SPI can drive two consecutive accesses to the same slave, an 8-bit delay counter is available to satisfy the delay time for data to be refreshed in the accessed slave. The delay counter can be programmed as part of the data format.

CHARLEN[4:0] specifies the number of bits (2 to 16) in the data word. The CHARLEN[4:0] value directs the state control logic to count the number of bits received or transmitted to determine when a complete word is transferred.

Data word length **must** be programmed to the same length for both the **master** and the **slave**. However, when chip selects are used, there may be multiple targets with different lengths in the system.

NOTE: Data must be right-justified when it is written to the SPI for transmission irrespective of its character length or word length.

Figure 27-6 shows how a 12-bit word (0xEC9) needs to be written to the transmit buffer to be transmitted correctly.

Figure 27-6. Format for Transmitting an 12-Bit Word

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
x	x	x	x	1	1	1	0	1	1	0	0	1	0	0	1

NOTE: The received data is always stored right-justified regardless of the character length or direction of shifting and is padded with leading 0s when the character length is less than 16 bits.

Figure 27-7 shows how a 10-bit word (0x0A2) is stored in the buffer once it is received.

Figure 27-7. Format for Receiving an 10-Bit Word

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
0	0	0	0	0	0	0	0	1	0	1	0	0	0	1	0

27.2.7 Clocking Modes

SPICLK may operate in four different modes, depending on the choice of phase (delay/no delay) and the polarity (rising edge/falling edge) of the clock.

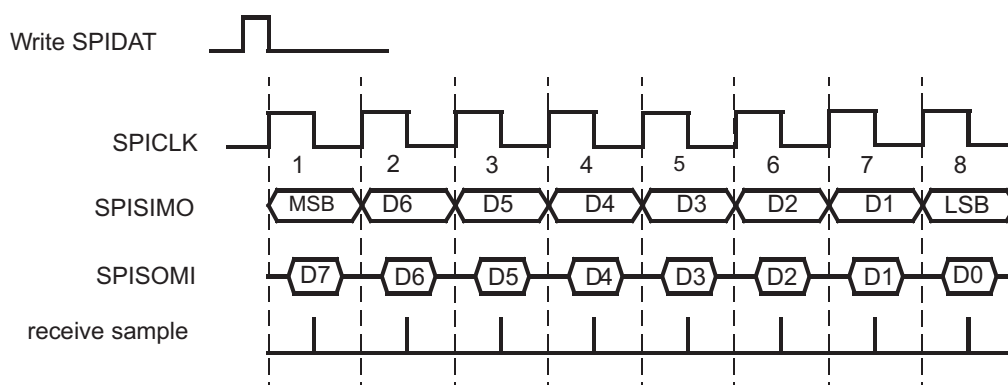
The data input and output edges depend on the values of both **POLARITY** and **PHASE** as shown in [Table 27-2](#).

Table 27-2. Clocking Modes

POLARITY	PHASE	Action
0	0	Data is output on the rising edge of SPICLK. Input data is latched on the falling edge.
0	1	Data is output one half-cycle before the first rising edge of SPICLK and on subsequent falling edges. Input data is latched on the rising edge of SPICLK.
1	0	Data is output on the falling edge of SPICLK. Input data is latched on the rising edge.
1	1	Data is output one half-cycle before the first falling edge of SPICLK and on subsequent rising edges. Input data is latched on the falling edge of SPICLK.

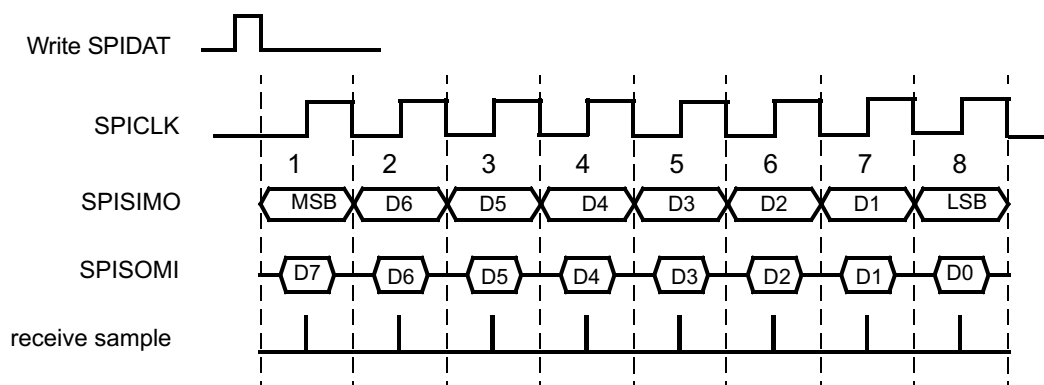
[Figure 27-8](#) to [Figure 27-11](#) illustrate the four possible configurations of **SPICLK** corresponding to each mode. Having four signal options allows the SPI to interface with many different types of serial devices.

Figure 27-8. Clock Mode with Polarity = 0 and Phase = 0

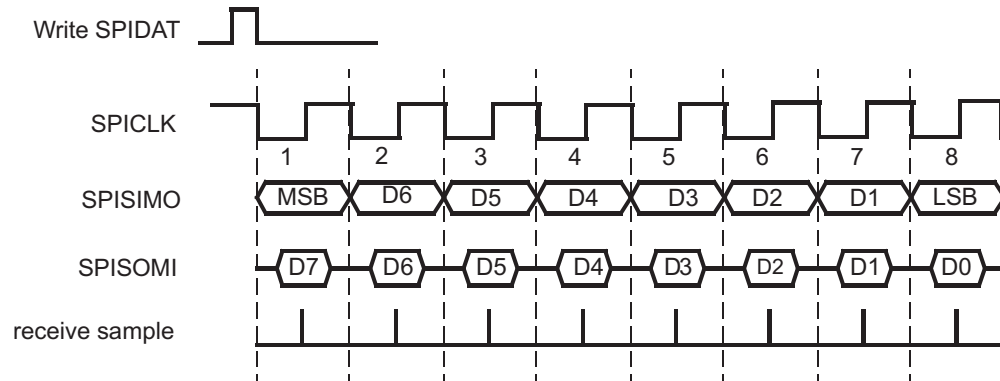


Data is output on the rising edge of SPICLK.
Input data is latched on the falling edge of SPICLK.

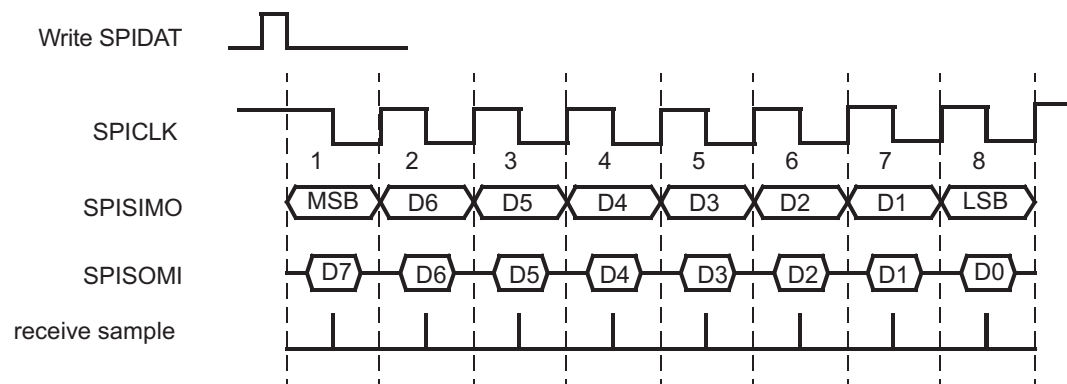
Figure 27-9. Clock Mode with Polarity = 0 and Phase = 1



Data is output one-half cycle before the first rising edge of SPICLK and on subsequent falling edges of SPICLK
Input data is latched on the rising edge of SPICLK

Figure 27-10. Clock Mode with Polarity = 1 and Phase = 0


Data is output on the falling edge of SPICLK.
Input data is latched on the rising edge of SPICLK.

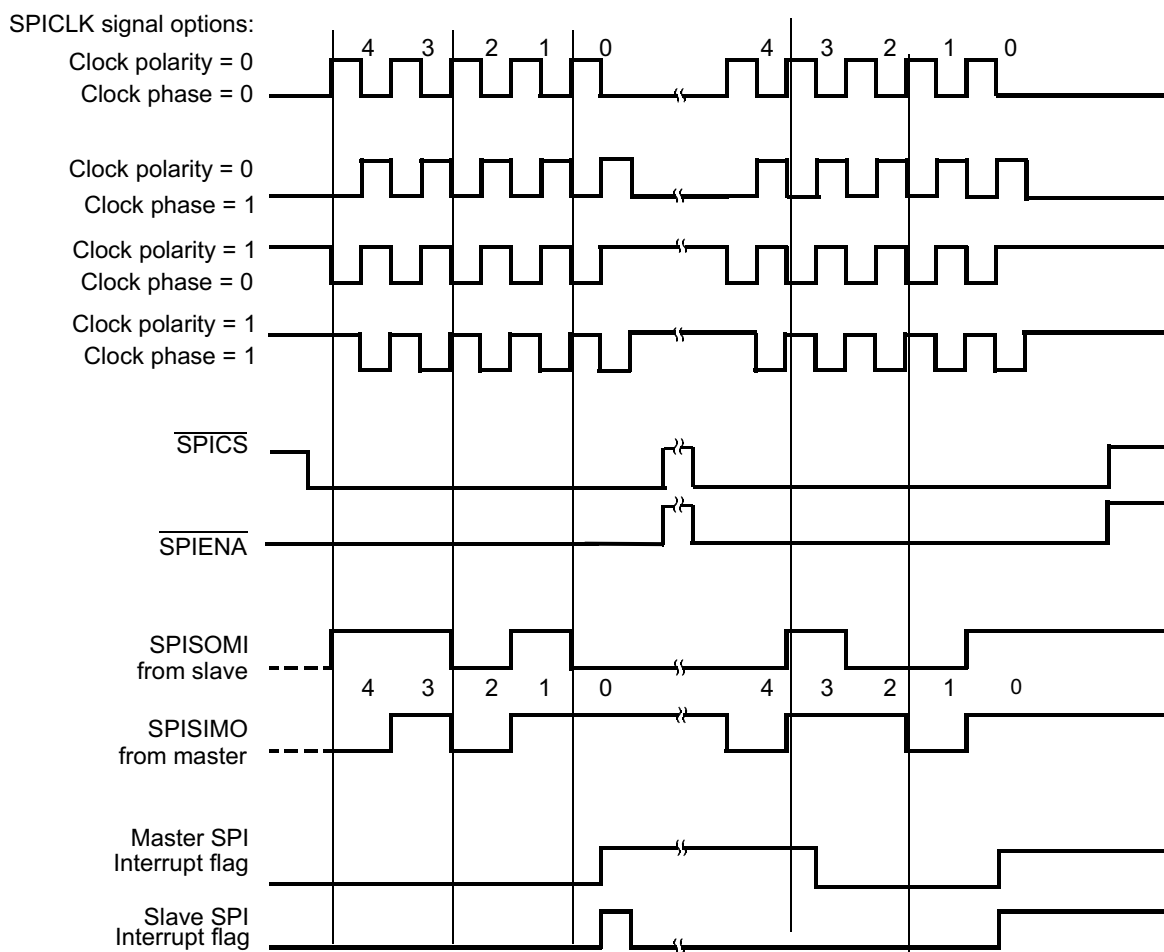
Figure 27-11. Clock Mode with Polarity = 1 and Phase = 1


Data is output one-half cycle before the first falling edge of SPICLK and on the subsequent rising edges of SPICLK.
Input data is latched on the falling edge of SPICLK.

27.2.8 Data Transfer Example

Figure 27-12 illustrates a SPI data transfer between two devices using a character length of five bits.

Figure 27-12. Five Bits per Character (5-Pin Option)



27.2.9 Decoded and Encoded Chip Select (Master Only)

In this device, the SPI can connect to up to 6 individual slave devices using chip-selects by routing one wire to each slave. The 6 chip selects in the control field are directly connected to the 6 pins. The default value of each chip select (not active) can be configured via the register CSDEF. During a transmission, the value of the chip select control field (CSNR) of the SPIDAT1 register (SPIDAT1) is driven on the SPICS pins. When the transmission finishes, the default chip-select value (defined by the CSDEF register) is put on the SPICS pins.

The SPI can support more than 6 slaves by using encoded chip selects. To connect the SPI with encoded slaves devices, the CSNR field allows multiple active SPICS pins at the same time, which enables encoded chip selects from 0 to 16. To use encoded chip selects, all 6 chip select lines have to be connected to each slave device and each slave needs to have a unique chip-select address. The CSDEF register is used to provide the address at which slaves devices are all de-selected.

Users can combine decoded and encoded chip selects. For example, n SPICS pins can be used for encoding an n -bit address and the remaining pins can be connected to decoded-mode slaves.

27.2.10 Variable Chip Select Setup and Hold Timing (Master Only)

In order to support slow slave devices, a delay counter can be configured to delay data transmission after the chip select is activated. A second delay counter can be configured to delay the chip select deactivation after the last data bit is transferred. Both delay counters are clocked with the peripheral clock (VCLK).

If a particular data format specifically does not require these additional set-up or hold times for the chip select pins, then they can be disabled in the corresponding SPIFMTx register.

27.2.11 Hold Chip-Select Active

Some slave devices require the chip select signal to be held continuously active during several consecutive data word transfers. Other slave devices require the chip select signal to be deactivated between consecutive data word transfers.

CSHOLD is programmable in both master and slave modes of the multi-buffer mode of SPI. However, the meaning of CSHOLD in master mode and slave mode are different.

NOTE: If the CSHOLD bit is set within the current data control field, the programmed hold time and the following programmed set-up time will not be applied between transactions.

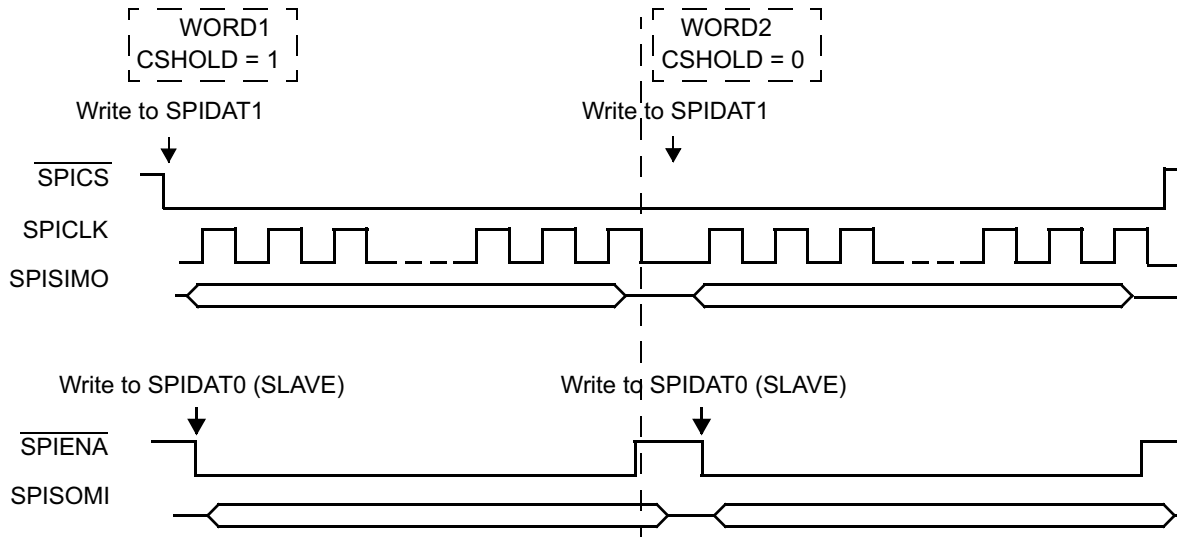
27.2.11.1 CSHOLD Bit in Master Mode

Each word in a master-mode SPI can be individually initialized for one of the two modes via the CSHOLD bit in its control field.

If the CSHOLD bit is set in the control field of a word, the chip select signal will not be deactivated until the next control field is loaded with new chip select information. Since the chip-select is maintained active between two transfers, the chip-select hold delay (T2CDELAY) is not applied at the end of the current transaction, and the chip-select set-up time delay (C2TDELAY) is not applied as well at the beginning of the following transaction. However, the wait delay (WDELAY) will be still applied between the two transactions, if the WDEL bit is set within the control field.

Figure 27-13 shows the SPI pins when a master-mode SPI transfers a word that has its CSHOLD bit set. The chip-select pins will not be deasserted after the completion of this word. If the next word to transmit has the same chip-select number (CSNR) value, the chip select pins will be maintained until the completion of the second word, regardless of whether the CSHOLD bit is set or not.

Figure 27-13. Typical Diagram when a Buffer in Master is in CSHOLD Mode (SPI-SPI)



27.2.11.2 CSHOLD Bit in Slave Mode (Multi-buffered Mode)

If the CSHOLD bit in a buffer is set to 1, then the MibSPI does not wait for the $\overline{\text{SPICS}}$ pins to be deasserted at the end of the shift operation to copy the received data to the receive RAM. With this feature, it is possible for a slave in multi-buffer mode to do multiple data transfers without requiring the $\overline{\text{SPICS}}$ pins to be deasserted between two buffer transfers.

If the CSHOLD bit in a buffer is cleared to 0 in a slave MibSPI, even after the shift operation is done, the MibSPI waits until the $\overline{\text{SPICS}}$ pin (if functional) is deasserted to copy the received data to the RXRAM.

If the CSHOLD bit is maintained as 0 across all the buffers, then the slave in multi-buffer mode requires its $\overline{\text{SPICS}}$ pins to be deasserted between any two buffer transfers; otherwise, the Slave SPI will be unable to respond to the next data transfer.

NOTE: In compatibility mode, the slave does not require the $\overline{\text{SPICS}}$ pin to be deasserted between two buffer transfers. The CSHOLD bit of the slave will be ignored in compatibility mode.

27.2.12 Detection of Slave Desynchronization (Master Only)

When a slave supports generation of an enable signal (ENA), desynchronization can be detected. With the enable signal a slave indicates to the master that it is ready to exchange data. A desynchronization can occur if one or more clock edges are missed by the slave. In this case, the slave may block the SOMI line until it detects clock edges corresponding to the next data word. This would corrupt the data word of the desynchronized slave and the consecutive data word. A configurable 8-bit time-out counter (T2EDELAY), which is clocked with SPICLK, is implemented to detect this slave malfunction. After the transmission has finished (end of last bit transferred: either last data bit or parity bit) the counter is started. If the ENA signal generated by the slave does not become inactive before the counter overflows, the DESYNC flag is set and an interrupt is asserted (if enabled).

NOTE: Inconsistency of Desynchronization Flag in Compatibility Mode MibSPI

Because of the nature of this error, under some circumstances it is possible for a desync error detected for the previous buffer to be visible in the current buffer. This is due to the fact that receive completion flag/interrupt will be generated when the buffer transfer is completed. But desync will be detected after the buffer transfer is completed. So, if VBUS master reads the received data quickly when an RXINT is detected, then the status flag may not reflect the correct desync condition. This inconsistency in the desync flag is valid only in compatibility mode of MibSPI. In multi-buffer mode, the desync flag is always assured to be for the current buffer.

27.2.13 ENA Signal Time-Out (Master Only)

The SPI in master mode waits for the hardware handshake signal (ENA) coming from the addressed slave before performing a data transfer. To avoid stalling the SPI by a non-responsive slave device, a time-out value can be configured using C2EDELAY. If the time-out counter overflows before an active ENA signal is sampled, the TIMEOUT flag in the status register SPIFLG is set and the TIMEOUT flag in the status field of the corresponding buffer is set.

NOTE: When the chip select signal becomes active, no breaks in transmission are allowed. The next arbitration is performed while waiting for the time-out to occur.

27.2.14 Data-Length Error

A SPI can generate an error flag by detecting any mismatch in length of received or transmitted data and the programmed character length under certain conditions.

Data-Length Error in Master Mode: During a data transfer, if the SPI detects a de-assertion of the $\overline{\text{SPIENA}}$ pin (by the slave) while the character counter is not overflowed, then an error flag is set to indicate a data-length error. This can be caused by a slave receiving extra clocks (for example, due to noise on the SPICLK line).

NOTE: In a master mode SPI, the data length error will be generated only if the $\overline{\text{SPIENA}}$ pin is enabled as a functional pin.

Data-Length Error in Slave Mode: During a transfer, if the SPI detects a de-assertion of the $\overline{\text{SPICS}}$ pin before its character length counter overflows, then an error flag is set to indicate a data-length error. This situation can arise if the slave SPI misses one or more SPICLK pulses from the master. This error in slave mode implies that both the transmitted and received data were not complete.

NOTE: In a slave mode SPI, the data-length error flag will be generated only if at least one of the $\overline{\text{SPICS}}$ pins are configured as functional, and are being used for selecting the slave.

27.2.15 Parallel Mode (Multiple SIMO/SOMI Support, not available on all devices)

In order to increase throughput, the parallel mode of the SPI enables the module to send data over more than one data line (parallel 2, 4, or 8). When parallel mode is used, the data length must be set as 16 bits. Only module MIBSPIP5 supports Parallel Mode.

This feature increases throughput by 2 for 2 pins, by 4 for 4 pins, or by 8 for 8 pins.

Parallel mode supports the following features:

- Scalable data lines (1, 2, 4, 8) per direction. (SOMI and SIMO lines)
- All clock schemes are supported (clock phase and polarity)
- Parity is supported. The parity bit will be transmitted on bit0 of the SIMO/SOMI lines. The receive parity is expected on bit0 of the SOMI/SIMO pins.

Parallel mode can be programmed using the PMODEx bits of SPIPMCTRL register. See [Section 27.9.25](#) for details about this register.

After reset the parallel mode selection bits are cleared (single SIMO/SOMI lines).

27.2.15.1 Parallel Mode Block Diagram

[Figure 27-14](#) and [Figure 27-15](#) show the parallel connections to the SPI shift register.

Figure 27-14. Block Diagram Shift Register, MSB First

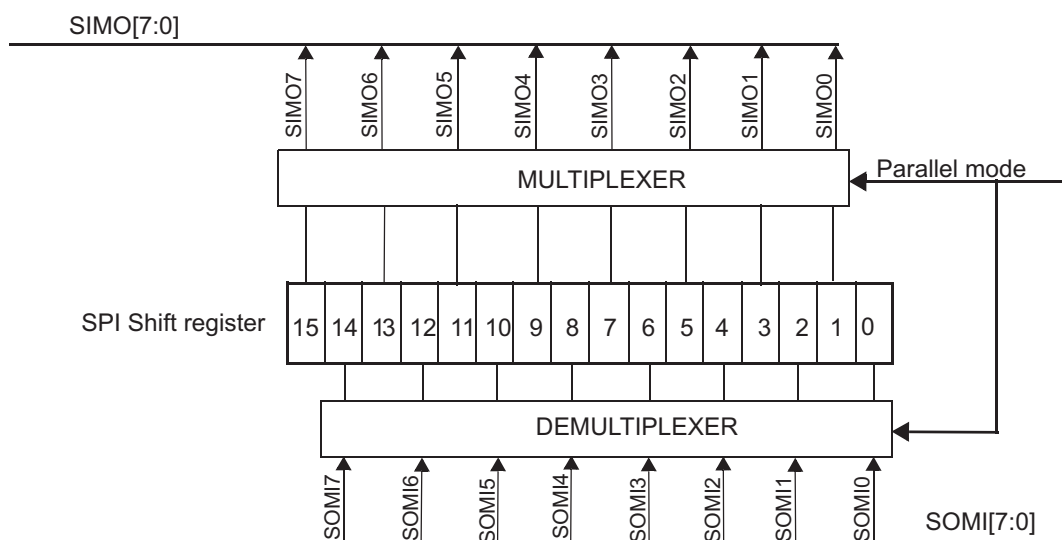
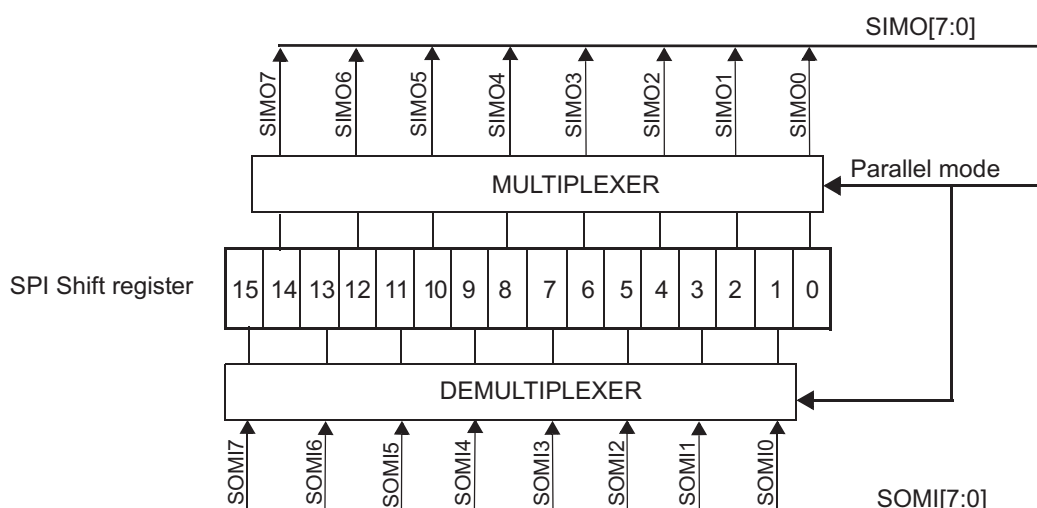


Figure 27-15. Block Diagram Shift Register, LSB First



27.2.15.2 Parallel Mode Pin Mapping, MSB First

Table 27-3 and Table 27-4 describe the SOMI and SIMO pin mapping when the SPI is used in parallel mode (1, 2, 4, 8) pin mode, MSB first.

NOTE: MSB-first or LSB-first can be configured using the SHIFTDIRx bit of the SPIFMTx registers.

Table 27-3. Pin Mapping for SIMO Pin with MSB First

Parallel Mode	Shift Register Bit	SIMO[7:0]
1	15	0
2	15	1
	7	0
4	15	3
	11	2
	7	1
	3	0
8	15	7
	13	6
	11	5
	9	4
	7	3
	5	2
	3	1
	1	0

Table 27-4. Pin Mapping for SOMI Pin with MSB First

Parallel Mode	Shift Register Bit	SOMI[7:0]
1	0	0
2	0	0
	8	1
4	0	0
	4	1
	8	2
	12	3
8	0	0
	2	1
	4	2
	6	3
	8	4
	10	5
	12	6
	14	7

27.2.15.3 Parallel Mode Pin Mapping, MSB-First, LSB-First

Table 27-5 and Table 27-6 describe the SIMO and SOMI pin mapping when SPI is used in parallel mode (1, 2, 4, 8) pin mode, LSB first.

Table 27-5. Pin Mapping for SIMO Pin with LSB First

Parallel Mode	Shift Register Bit	SIMO[7:0]
1	0	0
2	8	1
	0	0
4	12	3
	8	2
	4	1
	0	0
8	14	7
	12	6
	10	5
	8	4
	6	3
	4	2
	2	1
	0	0

Table 27-6. Pin Mapping for SOMI Pin with LSB First

Parallel Mode	Shift Register Bit	SOMI[7:0]
1	15	0
2	7	0
	15	1
4	3	0
	7	1
	11	2
	15	3
8	1	0
	3	1
	5	2
	7	3
	9	4
	11	5
	13	6
	15	7

27.2.15.4 2-Data Line Mode (MSB First, Phase 0, Polarity 0)

In 2-data line mode (master mode) the shift register bits 15 and 7 will be connected to the pins SIMO[1] and SIMO[0], and the shift register bits 8 and 0 will be connected to the pins SOMI[1] and SOMI[0] or vice versa in slave mode. After writing to the SPIDAT0/SPIDAT1 register, the bits 15 and 7 will be output on SIMO[1] and SIMO[0] on the rising edge of SPICLK. With the falling clock edge of the SPICLK, the received data on SOMI[1] and SOMI[0] will be latched to the shift register bits 8 and 0. The subsequent rising edge of SPICLK will shift the data in the shift register by 1 bit to the left. (SIMO[1] will shift the data out from bit 15 to 8, SIMO[0] will shift the data out from bit 7 to 0). After eight SPICLK cycles, when the full data word is transferred, the shift register (16 bits) is copied to the receive buffer, and the RXINT flag will be set. Figure 27-16 shows the clock /data diagram of the 2-data line mode. Figure 27-17 shows the timing of a two-pin parallel transfer.

Figure 27-16. 2-data Line Mode (Phase 0, Polarity 0)

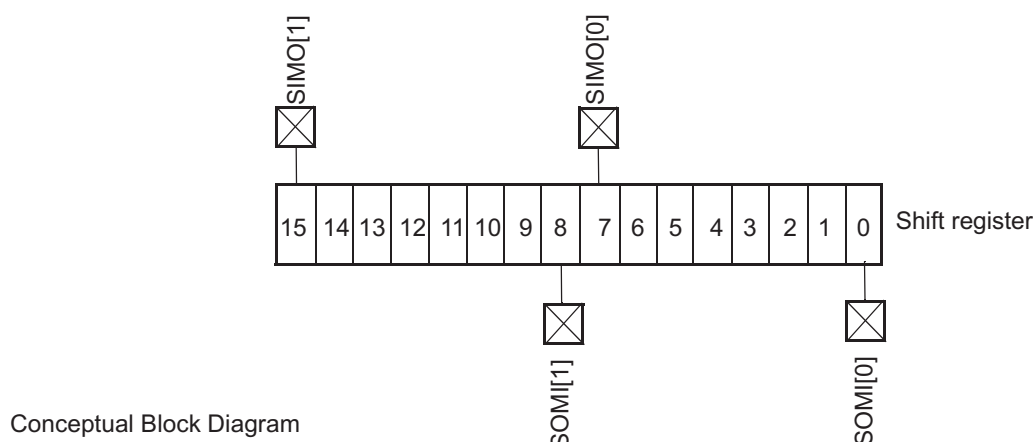
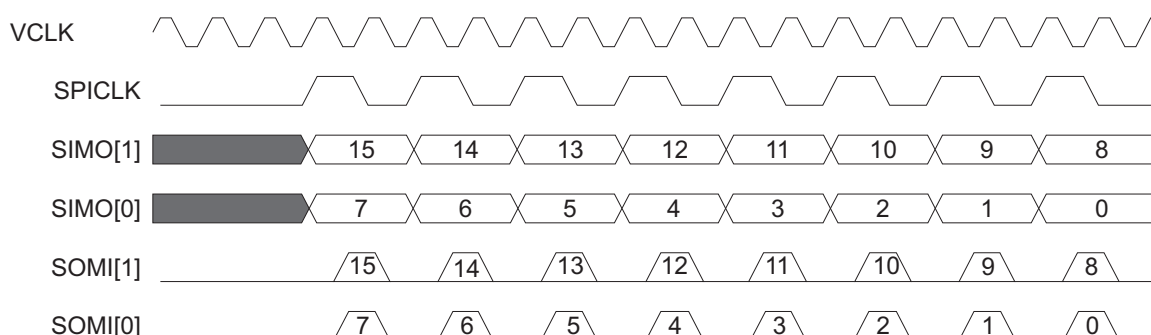


Figure 27-17. Two-Pin Parallel Mode Timing Diagram (Phase 0, Polarity 0)



27.2.15.5 4-Data Line Mode (MSB First, Phase 0, Polarity 0)

In 4-data line mode (master mode) the shift register bits 15, 11, 7, and 3 will be connected to the pins SIMO[3], SIMO[2], SIMO[1], and SIMO[0], and the shift register bits 12, 8, 4, and 0 will be connected to the pins SOMI[3], SOMI[2], SOMI[1], and SOMI[0] (or vice versa in slave mode). After writing to SPIDAT1/SPIDAT0, the bits 15, 11, 7, and 3 will be output on SIMO[3], SIMO[2], SIMO[1], and SIMO[0] on the rising edge of SPICLK. With the falling clock edge of the SPICLK, the received data on SOMI[3], SOMI[2], SOMI[1] and SOMI[0] will be latched to shift register bits 12, 8, 4, and 0. The subsequent rising edge of SPICLK will shift data in the shift register by 1 bit to the left (SIMO[3] will shift the data out from bit 15 to 12, SIMO[2] will shift the data out from bit 11 to 8, SIMO[1] will shift the data out from bit 7 to 4, SIMO[0] will shift the data out from bit 3 to 0). After four SPICLK cycles, when the full data word is transferred, the shift register (16 bits) is copied to the receive buffer, and the RXINT flag will be set.

Figure 27-18 shows the clock/data diagram of the four-data line mode. Figure 27-19, shows the timing diagram for four-data line mode.

Figure 27-18. 4-Data Line Mode (Phase 0, Polarity 0)

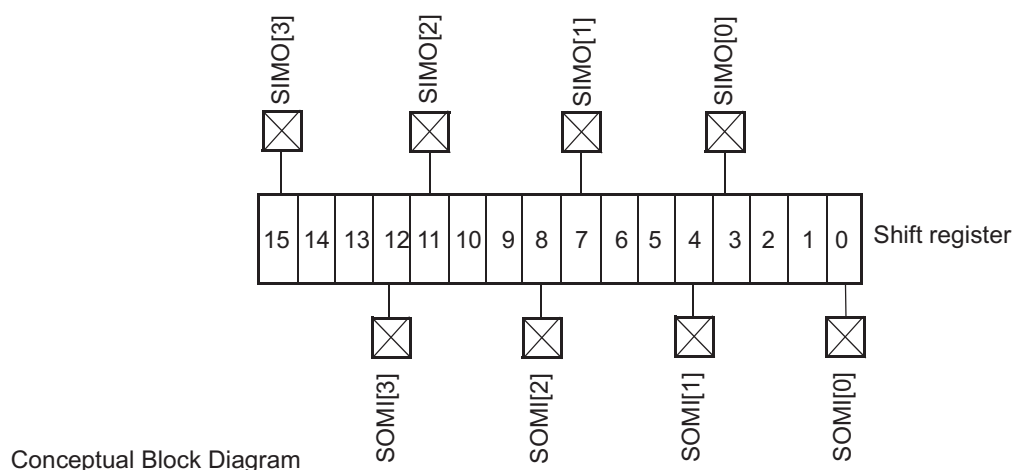
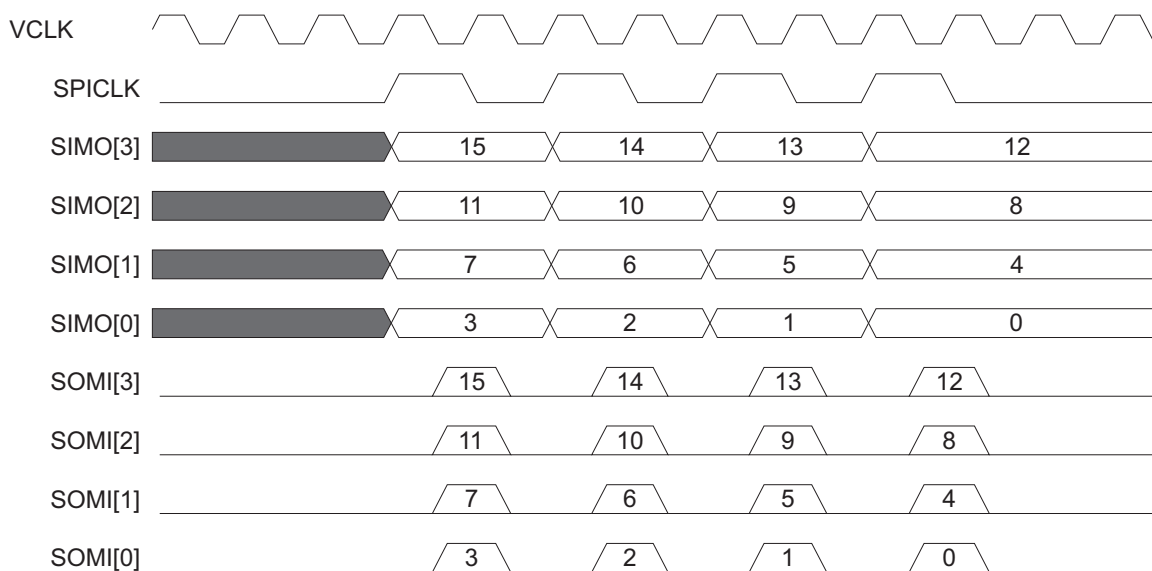


Figure 27-19. 4 Pins Parallel Mode Timing Diagram (Phase 0, Polarity 0)



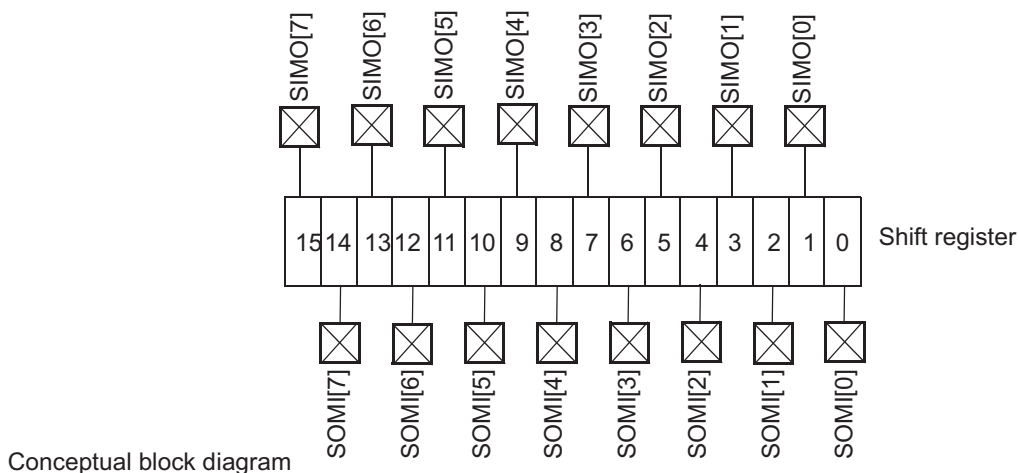
27.2.15.6 8-Data Line Mode (MSB First, Phase 0, Polarity 0)

In 8-data line mode (master mode) the shift register bits 15, 13, 11, 9, 7, 5 and 3 will be connected to the pins SIMO[7], SIMO[6], SIMO[5], SIMO[4], SIMO[3], SIMO[2], SIMO[1], and SIMO[0], and the shift-register bits 14, 12, 10, 8, 6, 4, and 0 will be connected to the pins SOMI[7], SOMI[6], SOMI[5], SOMI[4], SOMI[3], SOMI[2], SOMI[1], and SOMI[0] (or vice versa in slave mode).

After writing to SPIDAT0/SPIDAT1, the bits 15, 13, 11, 9, 7, 5, 3, and 1 will be output on SIMO[7], SIMO[6], SIMO[5], SIMO[4], SIMO[3], SIMO[2], SIMO[1], and SIMO[0], on the rising edge of SPICLK. On the falling clock edge of the SPICLK, the received data on SOMI[8], SOMI[7], SOMI[6], SOMI[5], SOMI[4], SOMI[3], SOMI[2], SOMI[1], and SOMI[0] will be latched to the shift register bits 14, 12, 10, 8, 6, 4, 2, and 0.

The subsequent rising edge of SPICLK will shift the data in the shift register by 1 bit to the left. After two SPICLK cycles, when the full data word is transferred the shift register (16 bits) is copied to the receive buffer, and the RXINT flag will be set. [Figure 27-20](#) shows the clock/data diagram of the 8-data line mode. [Figure 27-21](#) shows the pin timings for 8-data line mode.

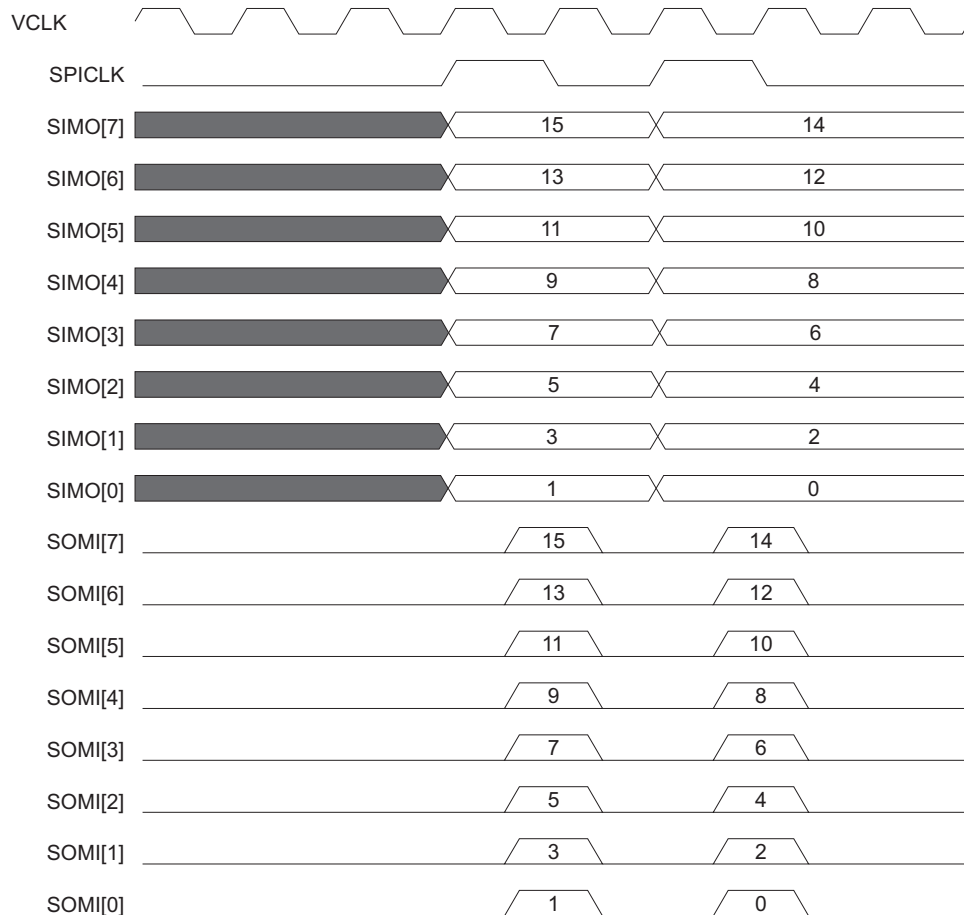
Figure 27-20. 8-data Line Mode (Phase 0, Polarity 0)



NOTE: Parity Support

Using the parity support in parallel mode may seriously affect throughput. For an eight-line mode to transfer 16 bits of data, only two SPICLK pulses are enough. If parity is enabled, one extra SPICLK pulse will be used to transfer and receive the parity bit. Parity will be transmitted and received on the 0th line regardless of 1/2/4/8-line modes. During the parity bit transfer, other data bits are not valid.

Figure 27-21. 8 Pins Parallel Mode Timing Diagram (Phase 0, Polarity 0)



NOTE: Modulo Count Parallel Mode is not supported in this device.

27.2.16 Continuous Self-Test (Master/Slave)

During data transfer, the SPI compares its own internal transmit data with its transmit data on the bus. The sample point for the compare is at one-half SPI clock after transmit point. If the data on the bus does not match the expected value, the bit-error (BITERR) flag is set and an interrupt is asserted if enabled.

NOTE: The compare is made from the output pin using its input buffer.

27.2.17 Half Duplex Mode

SPI by protocol is Full Duplex in nature, which means simultaneous TX and RX operations happen on two separate data pins, SIMO and SOMI. However, it is possible to use SPI/MibSPI to do the TX-only operation (ignoring the RX data) and the RX-only operation (using dummy TX data and ignoring the TX pin). But this requires that both SOMI and SIMO lines are bonded out in a chip to be able to support both TX-only or RX-only features.

27.2.17.1 Half Duplex Mode in Master

The Half Duplex Mode gives an additional flexibility to use the SIMO pin, which is normally used as a TX pin in Master mode, to work like an RX pin while the HDUPLEX_ENAx bit in SPIFMTx register is set to 1. In Half Duplex Master mode, the SIMO pin acts as an RX pin. Switching between Full Duplex and Half Duplex can be achieved using the SPIFMTx register being selected using the DFSEL bit of SPIDAT1 register or TXRAM locations.

27.2.17.2 Half Duplex Mode in Slave

In Half Duplex Slave mode, the SIMO pin, which is normally an RX pin, acts as a TX pin while the HDUPLEX_ENAx bit in SPIFMTx register is set to 1. In Half Duplex Slave mode, the SIMO pin acts as a TX pin. Switching between Full Duplex and Half Duplex can be achieved using the SPIFMTx register being selected using the DFSEL bit of SPIDAT1 register or TXRAM locations.

27.3 Test Features

27.3.1 Internal Loop-Back Test Mode (Master Only)

The internal loop-back self-test mode can be utilized to test the SPI transmit and receive paths, including the shift registers, the SPI buffer registers, and the parity generator. In this mode the transmit signal is internally feedback to the receiver, whereas the SIMO, SOMI, and CLK pin are disconnected; that is, the transmitted data is internally transferred to the corresponding receive buffer while external signals remain unchanged.

This mode allows the CPU to write into the transmit buffer, and check that the receive buffer contains the correct transmit data. If an error occurs the corresponding error is set within the status field.

NOTE: This mode cannot be changed during transmission.

27.3.2 Input/Output Loopback Test Mode

Input/Output Loopback Test mode supports the testing of all Input/Output pins without the aid of an external interface. Loopback can be configured as either analog-loopback (loopback through the pin-level input/output buffers) or digital loopback (internal to the SPI module). With Input/Output Loopback, all functional features of the SPI can be tested. Transmit data is fed back through the receive-data line(s). See Figure 27-22 for a diagram of the types of feedback available. The IOLPBKTSTCR register defines all of the available control fields.

In loopback mode, it is also possible to induce various error conditions. See Section 27.9.43 for details of the register field controlling these features.

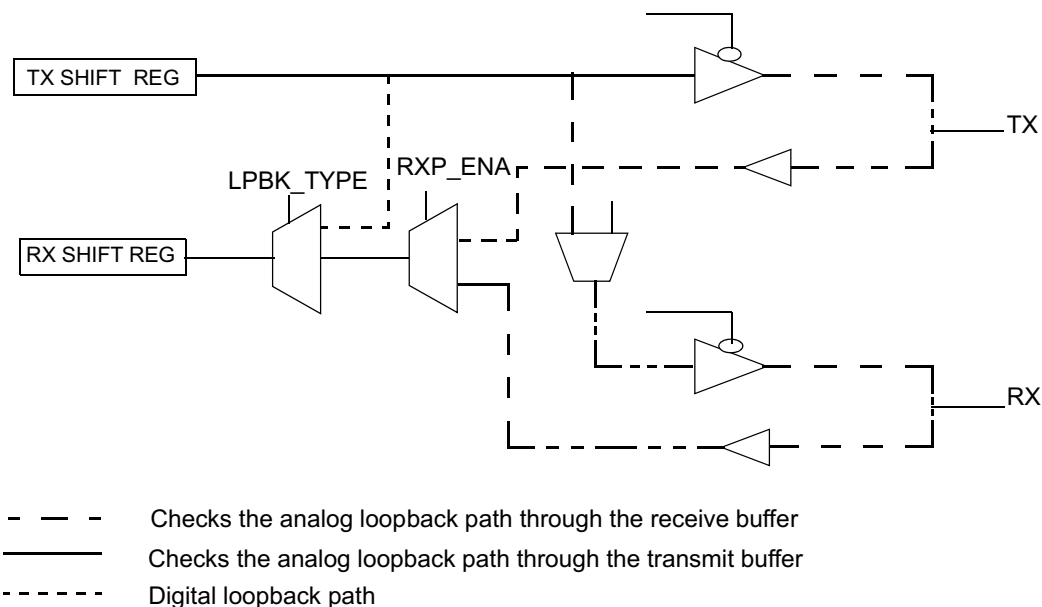
In Input/Output loopback test modes, even when the module is in slave mode, the SPICLK is generated internally. This SPICLK is used for all loopback-mode SPI transactions. Slave-mode features can be tested without the help of another master SPI, using the internally-generated SPICLK. Chip selects are also generated by the slave itself while it is in Input/Output loopback mode.

In Input/Output loopback test modes, if the module is in master mode, the $\overline{\text{ENA}}$ signal is also generated by internal logic so that an external interface is not required.

NOTE: Usage Guideline for Input/Output Loopback

Input/Output Loopback mode should be used with caution because, in some configurations, even the receive pins will be driven with transmit data. During testing, it should be ensured that none of the SPI pins are driven by any other device connected to them. Otherwise, if analog loopback is selected in I/O Loopback mode, then testing may damage the device.

Figure 27-22. I/O Paths during I/O Loopback Modes



This diagram is intended to illustrate loopback paths and therefore may omit some normal-mode paths.

27.3.2.1 Input/Output Loopback Mode Operation in Slave Mode

In multi-buffer slave mode, there are some additional requirements for using I/O loopback mode (IOLPBK). In multi-buffer slave mode, the chip-select pins are the triggers for various TGs. Enabling the IOLPBK mode by writing 0xA to the IOLPBTSTENA bits of the IOLPBKTSTCR register triggers TG0 by driving SPICS to 0. The actual number of chip selects can be programmed to have any or all of the SPICS pins as functional. All other configurations should be completed before enabling the IOLPBK mode in multi-buffer slave mode since it triggers TG0.

After the first buffer transfer is completed, the CSNR field of the current buffer is used to trigger the next buffer. So, if multiple TGs are desired to be tested, then the CSNR field of the final buffer in each TG should hold the number of the next TG to be triggered. As long as TG boundaries are well defined and are enabled, the completion of one TG will trigger the next TG.

To stop the transfer in multi-buffer slave mode in I/O Loopback configuration, either IOLPBK mode can be disabled by writing 0x5 to the IOLPBTSTENA bits or all of the TGs can be disabled.

27.4 General-Purpose I/O

All of the SPI pins may be programmed via the SPIPCx control registers to be either functional or general-purpose I/O pins.

If the SPI function is to be used, application software must ensure that at least the SPICLK pin and the SOMI and/or SIMO pins are configured as SPI functional pins, and not as GIO pins, or else the SPI state machine will be held in reset, preventing SPI transactions.

SPI pins support:

- internal pull-up resistors
- internal pull-down resistors
- open-drain or push-pull mode

27.5 Low-Power Mode

The SPI can be put into either local or global low-power mode. Global low-power mode is asserted by the system and is not controlled by the SPI. During global low-power mode, all clocks to the SPI are turned off, making the module completely inactive.

Local low-power mode is asserted by setting the POWERDOWN (SPIGCR1[8]) bit; setting this bit stops the clocks to the SPI internal logic and registers. Setting the POWERDOWN bit causes the SPI to enter local low-power mode and clearing the POWERDOWN bit causes SPI to exit from local low-power mode. All registers remain accessible during local power-down mode, since the clock to the SPI registers is temporarily re-enabled for each access. RAM buffers are also accessible during low power mode.

NOTE: Since entering a low-power mode has the effect of suspending all state-machine activities, care must be taken when entering such modes to ensure that a valid state is entered when low-power mode is active. Application software must ensure that a low power mode is not entered during a data transfer.

27.6 Interrupts

There are two levels of vectorized interrupts supported by the SPI. These interrupts can be caused under the following circumstances:

- Transmission error
- Receive overrun
- Receive complete (receive buffer full)
- Transmit buffer empty

These interrupts may be enabled or disabled via the SPIINT0 register.

During transmission, if one of the following errors occurs: BITERR, DESYNC, DLENERR, PARITYERR, or TIMEOUT, the corresponding bit in the SPIFLG register is set. If the corresponding enable bit is set, then an interrupt is generated. The level of all the above interrupts is set by the bit fields in the SPILVL register.

The error interrupts are enabled and prioritized independently from each other, but the interrupt generated will be the same if multiple errors are enabled on the same level. The SPIFLG register should be used to determine the actual cause of an error.

NOTE: Since there are two interrupt lines, one each for Level 0 and Level 1, it is possible for a programmer to separate out the interrupts for receive buffer full and transmit buffer empty. By programming one to Level 0 and the other to Level 1, it is possible to avoid a check on whether an interrupt occurred for transmit or for receive. A programmer can also choose to group all of the error interrupts into one interrupt line and both TX-empty and RX-full interrupts into another interrupt line using the LVL control register. In this way, it is possible to separate error-checking from normal data handling.

27.6.1 Interrupts in Multi-Buffer Mode

In multi-buffer mode, the SPI can generate interrupts on two levels.

In normal multi-buffer operation, the receive and transmit are not used and therefore the enable bits of SPIINT0 are not used.

The interrupts available in multi-buffer mode are:

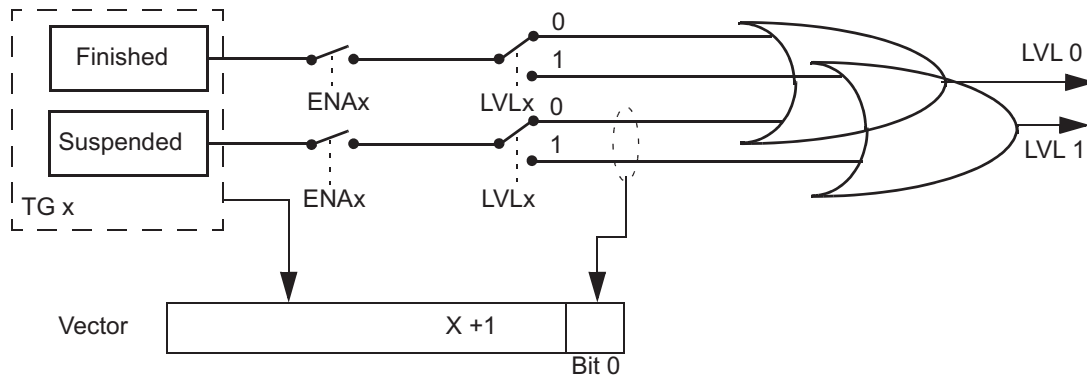
- Transmission error interrupt
- Receive overrun interrupt
- TG suspended interrupt
- TG completed interrupt

When a TG has finished and the corresponding enable bit in the TGINTENA register is set, a transfer-finished interrupt is generated. The level of priority of the interrupt is determined by the corresponding bit in the TGINTLVL register.

When a TG is suspended by a buffer that has been set as suspend to wait until TXFULL flag or/and RXEMPTY flag are set, and if the corresponding bit in the TGINTENA register is set, an transfer-suspended interrupt is generated. The level of priority of the interrupt is determined by the corresponding bit in the TGINTLVL register.

[Figure 27-23](#) illustrates the TG interrupts.

Figure 27-23. TG Interrupt Structure

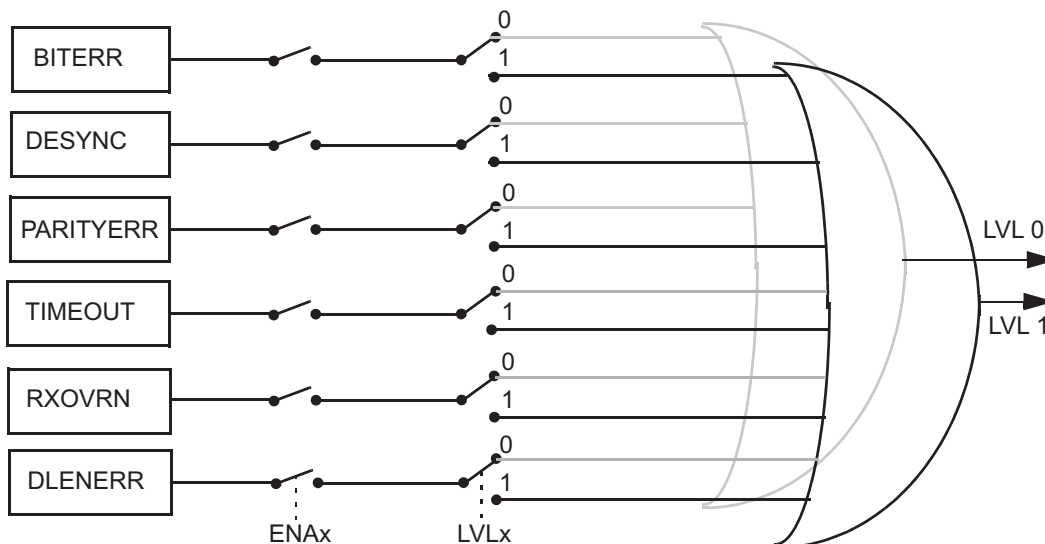


During transmission, if one of the following errors occurs, BITERR, DESYNC, PARITYERR, TIMEOUT, DLENERR, the corresponding flag in the SPIFLG register is set. If the enable bit is set, then an interrupt is generated. The level of the interrupts could be generated according to the bit field in SPILVL register.

The RXOVRN interrupt is generated when a buffer in the RXRAM is overwritten by a new received word. While writing newly received data to a RXRAM location, if the RXEMPTY bit of the corresponding location is 0, then the RXOVR bit will be set to 1 during the write operation, so that the buffer starts to indicate an overrun. This RXOVR flag is also reflected in SPIFLG register as RXOVRNINTFLG and the corresponding vector number is updated in TGINTVECT0/TGINTVECT1 register. If an overrun interrupt is enabled, then an interrupt will be generated indicating an overrun condition.

The error interrupts are enabled and prioritized independently from each other, but the vector generated by the SPI will be the same if multiple errors are enabled on the same level.

Figure 27-24. SPIFLG Interrupt Structure



Since the priority of an error interrupt is lower than a completion/suspend interrupt for a TG, the interrupts can be split into two levels. By programming all the error interrupts into Level 0 and TG-complete / TG-suspend interrupts into Level 1, it is possible to get a clear indication of the source of error interrupts. However, when a vector register shows an error interrupt, the actual buffer for which the error has occurred is not readily identifiable. Since each buffer in the multi-buffer RAM is stored along with its individual status flags, each buffer should be read until a buffer with any error flag set is found.

A separate interrupt line is provided to indicate the uncorrectable error condition in the MibSPI. This line is available (and valid) only in the multi-buffer mode of the MibSPI module and if the parity error detection feature for multi-buffer RAM is enabled.

27.7 DMA Interface

In order to reduce CPU overhead in handling SPI message traffic on a character-by-character basis, SPI can use the DMA controller to transfer the data. The DMA request enable bit (DMA REQ EN) controls the assertion of requests to the DMA controller module. When a character is being transmitted or received, the SPI will signal the DMA via the DMA request signals, TX_DMA_REQ and RX_DMA_REQ. The DMA controller will then perform the required data transfer.

For efficient behavior during DMA operations, the transmitter empty and receive-buffer full interrupts can be disabled. For specific DMA features, see the DMA controller specification.

The SPI generates a request on the TX_DMA_REQ line each time the TX data is copied to the TX shift register either from the TXBUF or from peripheral data bus (when TXBUF is empty).

The first TX_DMA_REQ pulse is generated when either of the following is true:

- DMAREQEN (SPIINT0[16]) is set to 1 while SPIEN (SPIGCR1[24]) is already 1.
- SPIEN (SPIGCR1[24]) is set to 1 while DMAREQEN (SPIINT0[16]) is already 1.

The SPI generates a request on the RX_DMA_REQ line each time the received data is copied to the SPIBUF.

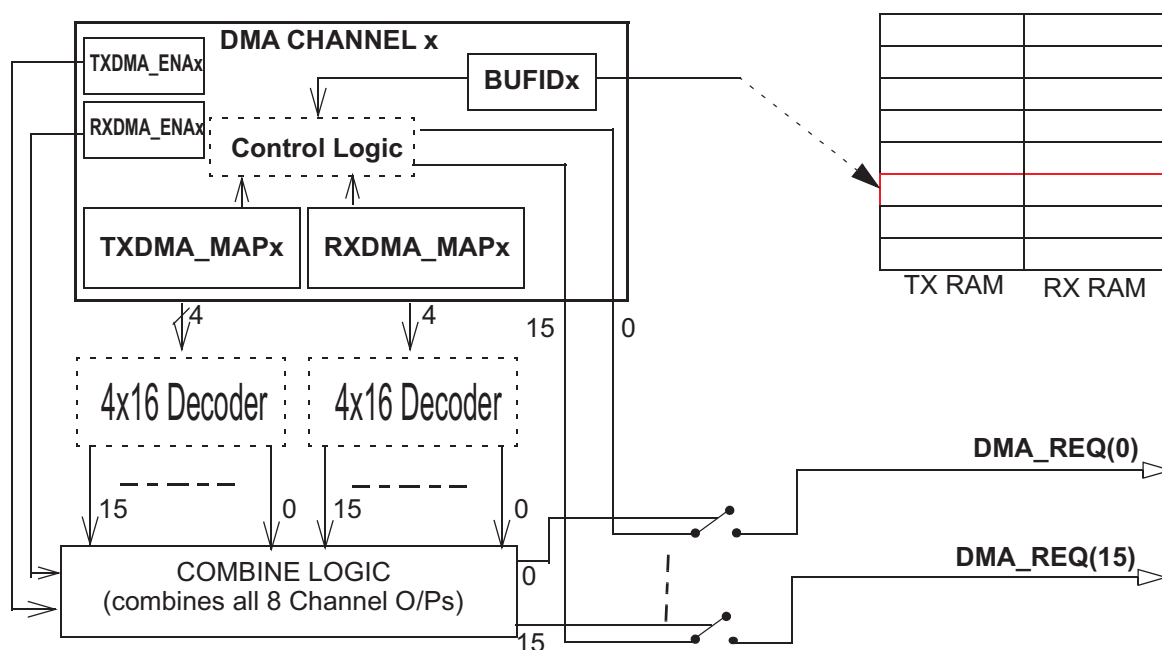
27.7.1 DMA in Multi-Buffer Mode

The MibSPI provides sophisticated programmable DMA control logic that completely eliminates the necessity of CPU intervention for data transfers, once programmed. When the multi-buffer mode is used, the DMA enable bit in the SPIINT0 register is ignored. DMA source or destination should be only the multi-buffer RAM and not SPIDAT0 / SPIDAT1 or SPIBUF register as in case of compatibility mode DMA.

The MibSPI offers up to eight DMA channels (for SEND and RECEIVE). All of the DMA channels are programmable individually and can be hooked to any buffer. The MibSPI provides up to 16 DMA request lines, and DMA requests from any channel can be programmed to be routed through any of these 16 lines. A DMA transfer can trigger both transmit and receive.

Each DMA channel has the capability to transfer a block of up to 32 data words without interruption using only one buffer of the array by configuring the DMAxCTRL register. Using the DMAxCOUNT and DMAxTNTLEN register, up to 65535 (64K) words of data can be transferred without any interruption using just one buffer of the array. This enables the transfer of memory blocks from or into an external SPI memory.

Figure 27-25. DMA Channel and Request Line (Logical) Structure in Multi-buffer Mode



27.8 Module Configuration

MibSPI/MibSPIP can be configured to function as Normal SPI and Multi-buffered SPI. Upon power-up or a system-level reset, each bit in the module registers is set to a default state. The registers are writable only after the RESET bit is set to 1.

27.8.1 Compatibility (SPI) Mode Configuration

The following list details the configuration steps that software should perform prior to the transmission or reception of data. As long as the SPIEN bit in the Global Control Register 1 (SPIGCR1) is cleared to 0 the entire time that the SPI is being configured, the order in which the registers are programmed is not important.

- Enable SPI by setting RESET bit.
- Configure the SIMO, SOMI, SPICLK, and optional $\overline{\text{SPICS}}$ and $\overline{\text{SPIENA}}$ pins for SPI functionality by setting the corresponding bit in SPIPC0 register.
- Configure the module to function as Master or Slave using CLKMOD and MASTER bits.
- Configure the required SPI data format using SPIFMTx register.
- If the module is selected to function as Master, the delay parameters can be configured using SPIDELAY register.
- Enable the Interrupts using SPIINT0 register if required.
- Select the chip select to be used by setting CSNR bits in SPIDAT1 register.
- Configure CSHOLD and WDEL bits in SPIDAT1 register if required.
- Select the Data word format by setting DFSEL bits. Select the Number of the configured SPIFMTx register (0 to 3) to used for the communication.
- Set LOOPBACK bit to connect the transmitter to the receiver internally. (This feature is used to perform a self-test. Do not configure for normal communication to external devices).
- Set SPIEN bit to 1 after the SPI is configured.
- Perform Transmit and receive data, using SPIDAT1 and SPIBUF register.
- You must wait for TXFULL to reset or TXINT before writing next data to SPIDAT1 register.
- You must wait for RXEMPTY to reset or RXINT before reading the data from SPIBUF register.

27.8.2 MibSPI Mode Configuration

The following list details the configuration steps that software should perform prior to the transmission or reception of data in MIBSPI mode. As long as the SPIEN bit in the Global Control Register 1 (SPIGCR1) is cleared to 0 the entire time that the SPI is being configured, the order in which the registers are programmed is not important.

- Enable SPI by setting RESET bit.
- Set MSPIENA bit to 1 to get access to multi-buffer mode registers.
- Configure the SIMO, SOMI, SPICLK, and optional $\overline{\text{SPICS}}$ and $\overline{\text{SPIENA}}$ pins for SPI functionality by setting the corresponding bit in SPIPC0 register.
- Configure the module to function as Master or Slave using CLKMOD and MASTER bits.
- Configure the required SPI data format using SPIFMTx register.
- If the module is selected to function as Master, the delay parameters can be configured using SPIDELAY register.
- Check for BUFINITACTIVE bit to be active before configuring MIBSPI RAM. (From Device Power On it take Number of Buffers $\times$ Peripheral clock period to initialize complete RAM.)
- Enable the Transfer Group interrupts using TGITENST register if required.
- Enable error interrupts using SPIINT0 register if required.
- Set SPIEN bit to 1 after the SPI is configured.
- The Trigger Source, Trigger Event, Transfer Group start address for the corresponding Transfer groups can be configured using the corresponding TGxCTRL register.
- Configure LPEND to specify the end address of the last TG.
- Similar to SPIDAT1 register, the 16 bit control fields in every TXRAM buffer in the TG have to be configured.
- Configure one of the eight BUFMODE available for each buffer.
- Fill the data to be transmitted in TXDATA field in TXRAM buffers.
- Configure TGENA bit to enable the required Transfer groups. (In case of Trigger event always setting TGENA will trigger the transfer group).
- At the occurrence of the correct trigger event, the Transfer group will be triggered and data gets transmitted and received one after the other with out any CPU intervention.
- You can poll Transfer group interrupt flag or wait for a transfer-completed interrupt to read and write new data to the buffers.

27.9 Control Registers

This section describes the SPI control, data, and pin registers. The registers support 8-bit, 16-bit and 32-bit writes. The offset is relative to the associated base address of this module in a system. The base address for the control registers is FFF7 F400h for MibSPI1, FFF7 F600h for SPI2, FFF7 F800h for MibSPI3, FFF7 FA00h for SPI4, and FFF7 FC00h for MibSPI5.

NOTE: TI highly recommends that write values corresponding to the reserved locations of registers be maintained as 0 consistently. This allows future enhancements to use these reserved bits as control bits without affecting the functionality of the module with any older versions of software.

Table 27-7. SPI Registers

Offset	Acronym	Register Description	Section
00h	SPIGCR0	SPI Global Control Register 0	Section 27.9.1
04h	SPIGCR1	SPI Global Control Register 1	Section 27.9.2
08h	SPIINT0	SPI Interrupt Register	Section 27.9.3
0Ch	SPIILVL	SPI Interrupt Level Register	Section 27.9.4
10h	SPIFLG	SPI Flag Register	Section 27.9.5
14h	SPIPC0	SPI Pin Control Register 0	Section 27.9.6
18h	SPIPC1	SPI Pin Control Register 1	Section 27.9.7
1Ch	SPIPC2	SPI Pin Control Register 2	Section 27.9.8
20h	SPIPC3	SPI Pin Control Register 3	Section 27.9.9
24h	SPIPC4	SPI Pin Control Register 4	Section 27.9.10
28h	SPIPC5	SPI Pin Control Register 5	Section 27.9.11
2Ch	SPIPC6	SPI Pin Control Register 6	Section 27.9.12
30h	SPIPC7	SPI Pin Control Register 7	Section 27.9.13
34h	SPIPC8	SPI Pin Control Register 8	Section 27.9.14
38h	SPIDAT0	SPI Transmit Data Register 0	Section 27.9.15
3Ch	SPIDAT1	SPI Transmit Data Register 1	Section 27.9.16
40h	SPIBUF	SPI Receive Buffer Register	Section 27.9.17
44h	SPIEMU	SPI Emulation Register	Section 27.9.18
48h	SPIDELAY	SPI Delay Register	Section 27.9.19
4Ch	SPIDEF	SPI Default Chip Select Register	Section 27.9.20
50h-5Ch	SPIFMT0-SPIFMT3	SPI Data Format Registers	Section 27.9.21
60h	INTVECT0	Interrupt Vector 0	Section 27.9.22
64h	INTVECT1	Interrupt Vector 1	Section 27.9.23
68h	SPIPC9 ⁽¹⁾	SPI Pin Control Register 9	Section 27.9.24
6Ch	SPIPMCTRL	Parallel/Modulo Mode Control Register	Section 27.9.25
70h	MIBSPIE	Multi-buffer Mode Enable Register	Section 27.9.26
74h	TGITENST	TG Interrupt Enable Set Register	Section 27.9.27
78h	TGITENCR	TG Interrupt Enable Clear Register	Section 27.9.28
7Ch	TGITLVST	Transfer Group Interrupt Level Set Register	Section 27.9.29
80h	TGITLVCR	Transfer Group Interrupt Level Clear Register	Section 27.9.30
84h	TGINTFLG	Transfer Group Interrupt Flag Register	Section 27.9.31
88h-8Ch	Reserved	Reserved	
90h	TICKCNT	Tick Count Register	Section 27.9.32
94h	LTGPEND	Last TG End Pointer	Section 27.9.33
98h-D4h	TGxCTRL	TGx Control Registers	Section 27.9.34
D8h-F4h	DMAxCTRL	DMA Channel Control Register	Section 27.9.35

⁽¹⁾ SPIPC9 only applies to SPI2.

Table 27-7. SPI Registers (continued)

Offset	Acronym	Register Description	Section
F8h-114h	ICOUNT	DMAxCOUNT Register	Section 27.9.36
118h	DMACNTLEN	DMA Large Count	Section 27.9.37
11Ch	Reserved	Reserved	
120h	UERRCTRL	Multi-buffer RAM Uncorrectable Parity Error Control Register	Section 27.9.38
124h	UERRSTAT	Multi-buffer RAM Uncorrectable Parity Error Status Register	Section 27.9.39
128h	UERRADDR1	RXRAM Uncorrectable Parity Error Address Register	Section 27.9.40
12Ch	UERRADDR0	TXRAM Uncorrectable Parity Error Address Register	Section 27.9.41
130h	RXOVRN_BUF_ADDR	RXRAM Overrun Buffer Address Register	Section 27.9.42
134h	IOLPBKTSTCR	I/O Loopback Test Control Register	Section 27.9.43
138h	EXTENDED_PRESCALE1	SPI Extended Prescale Register 1	Section 27.9.44
13Ch	EXTENDED_PRESCALE2	SPI Extended Prescale Register 2	Section 27.9.45

27.9.1 SPI Global Control Register 0 (SPIGCR0)

Figure 27-26. SPI Global Control Register 0 (SPIGCR0) [offset = 00]

31	16
Reserved	
R-0	
15	0
Reserved	nRESET
R-0	R/WP-0

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-8. SPI Global Control Register 0 (SPIGCR0) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	nRESET	0	This is the local reset control for the module. This bit needs to be set to 1 before any operation on SPI / MibSPI can be done. Only after setting this bit to 1, the Auto Initialization of Multi-buffer RAM starts. Clearing this bit to 0 will result in all of the control and status register values to return to their default values.. SPI is in the reset state.
		1	SPI is out of the reset state.

27.9.2 SPI Global Control Register 1 (SPIGCR1)

Figure 27-27. SPI Global Control Register 1 (SPIGCR1) [offset = 04h]

31	25	24	23	17	16
Reserved	SPIEN	Reserved	LOOPBACK		
R-0	R/W-0	R-0	R/WP-0		
15	9	8	7	2	1
Reserved	POWERDOWN	Reserved	CLKMOD	MASTER	
R-0	R/W-0	R-0	R/W-0	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-9. SPI Global Control Register 1 (SPIGCR1) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return 0. Writes have no effect.
24	SPIEN	0 1	SPI enable. This bit enables SPI transfers. This bit must be set to 1 after all other SPI configuration bits have been written. When the SPIEN bit is 0 or cleared to 0, the following SPI registers get forced to their default states: - Both TX and RX shift registers - The TXDATA fields of the SPI Transmit Data Register 0 (SPIDAT0) and the SPI Transmit Data Register 1 (SPIDAT1) - All the fields of the SPI Flag Register (SPIFLG) - Contents of SPIBUF and the internal RXBUF registers The SPI is not activated for transfers. Activates SPI.
23-17	Reserved	0	Reads return 0. Writes have no effect.
16	LOOPBACK	0 1	Internal loop-back test mode. The internal self-test option can be enabled by setting this bit. If the SPISIMO and SPISOMI pins are configured with SPI functionality, then the SPISIMO[7:0] pins are internally connected to the SPISOMI[7:0] pins (transmit data is looped back as receive data). GIO mode for these pins is not supported in loopback mode. Externally, during loop-back operation, the SPICLK pin outputs an inactive value and SPISOMI[7:0] remains in the high-impedance state. If the SPI is initialized in slave mode or a data transfer is ongoing, errors may result. Note: This loopback mode can only be used in master mode. Master mode must be selected before setting LOOPBACK. When this mode is selected, the CLKMOD bit should be set to 1, meaning that SPICLK is internally generated. Internal loop-back test mode is disabled. Internal loop-back test mode is enabled.
15-9	Reserved	0	Reads return 0. Writes have no effect.
8	POWERDOWN	0 1	When active, the SPI state machine enters a power-down state. The SPI is in active mode. The SPI is in power-down mode.
7-2	Reserved	0	Reads return 0. Writes have no effect.
1	CLKMOD	0 1	Clock mode. This bit selects either an internal or external clock source. This bit also determines the I/O direction of the SPIEN$\bar{A}$ and SPIC$\bar{S}$ pins in functional mode. Clock is external. - SPIEN$\bar{A}$ is an output. - SPIC$\bar{S}$ are inputs. Clock is internally-generated. - SPIEN$\bar{A}$ is an input. - SPIC$\bar{S}$ are outputs.

Table 27-9. SPI Global Control Register 1 (SPIGCR1) Field Descriptions (continued)

Bit	Field	Value	Description
0	MASTER	0 1	SPISIMO/SPISOMI pin direction determination. Sets the direction of the SPISIMO and SPISOMI pins. Note: For master-mode operation of the SPI, MASTER bit should be set to 1 and CLKMOD bit can be set either 1 or 0. The master-mode SPI can run on an external clock on SPICLK. For slave mode operation, both the MASTER and CLKMOD bits should be cleared to 0. Any other combinations may result in unpredictable behavior of the SPI. In slave mode, SPICLK will not be generated internally in slave mode. 0 SPISIMO[7:0] pins are inputs, SPISOMI[7:0] pins are outputs. 1 SPISOMI[7:0] pins are inputs, SPISIMO[7:0] pins are outputs.

27.9.3 SPI Interrupt Register (SPIINT0)

Figure 27-28. SPI Interrupt Register (SPIINT0) [offset = 08h]

31				25				24							
Reserved								ENABLEHIGHZ							
R-0								R/W-0							
23				17				16							
Reserved								DMAREQEN							
R-0								R/W-0							
15				10				9		8					
Reserved						TXINT ENA		RXINT ENA							
R-0								R/W-0		R/W-0					
7		6		5		4		3		2		1		0	
Reserved		RXOVRNINT ENA		Reserved		BITERR ENA		DESYNC ENA		PARERR ENA		TIMEOUT ENA		DLENERR ENA	
R-0		R/W-0		R-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-10. SPI Interrupt Register (SPIINT0) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return 0. Writes have no effect.
24	ENABLEHIGHZ	0 1	$\overline{\text{SPIEN}}\overline{\text{A}}$ pin high-impedance enable. When active, the $\overline{\text{SPIEN}}\overline{\text{A}}$ pin (when it is configured as a WAIT functional output signal in a slave SPI) is forced to high-impedance when not driving a low signal. If inactive, then the pin will output both a high and a low signal. 0 $\overline{\text{SPIEN}}\overline{\text{A}}$ pin is pulled high when not active. 1 $\overline{\text{SPIEN}}\overline{\text{A}}$ pin remains high-impedance when not active.
23-17	Reserved	0	Reads return 0. Writes have no effect.
16	DMAREQEN	0 1	DMA request enable. Enables the DMA request signal to be generated for both receive and transmit channels. Enable DMA REQ only after setting the SPIEN bit to 1. 0 DMA is not used. 1 DMA requests will be generated. Note: A DMA request will be generated on the TX DMA REQ line each time a word is copied to the shift register either from TXBUF or directly from SPIDAT0/SPIDAT1 writes. Note: A DMA request will be generated on the RX DMA REQ line each time a word is copied to the SPIBUF register either from RXBUF or directly from the shift register.
15-10	Reserved	0	Reads return 0. Writes have no effect.

Table 27-10. SPI Interrupt Register (SPIINT0) Field Descriptions (continued)

Bit	Field	Value	Description
9	TXINTENA	0 1	Causes an interrupt to be generated every time data is written to the shift register, so that the next word can be written to TXBUF. Setting this bit will generate an interrupt if the TXINTFLG bit (SPI Flag Register (SPIFLG)[9]) is set to 1. No interrupt will be generated upon TXINTFLG being set to 1. An interrupt will be generated upon TXINTFLG being set to 1. The transmitter empty interrupt is valid in compatibility mode of SPI only. In multi-buffered mode, this interrupts will not be generated, even if it is enabled. Note: An interrupt request will be generated as soon as this bit is set to 1. By default it will be generated on the INT0 line. The SPILVL register can be programmed to change the interrupt line.
8	RXINTENA	0 1	Causes an interrupt to be generated when the RXINTFLAG bit (SPI Flag Register (SPIFLG)[8]) is set by hardware. Interrupt will not be generated. Interrupt will be generated. The receiver full interrupt is valid in compatibility mode of SPI only. In multi-buffered mode, this interrupts will not be generated, even if it is enabled.
7	Reserved	0	Reads return 0. Writes have no effect.
6	RXOVRNINTENA	0 1	Overrun interrupt enable. Overrun interrupt will not be generated. Overrun interrupt will be generated.
5	Reserved	0	Reads return 0. Writes have no effect.
4	BITERRENA	0 1	Enables interrupt on bit error. No interrupt asserted upon bit error. Enables interrupt on bit error.
3	DESYNCENA	0 1	Enables interrupt on desynchronized slave. DESYNCENA is used in master mode only. No interrupt asserted upon desynchronization error. An interrupt is asserted on desynchronization of the slave (DESYNC = 1).
2	PARERRENA	0 1	Enables interrupt-on-parity-error. No interrupt asserted on parity error. An interrupt is asserted on a parity error.
1	TIMEOUTENA	0 1	Enables interrupt on ENA signal time-out. No interrupt asserted upon ENA signal time-out. An interrupt is asserted on a time-out of the ENA signal.
0	DLENERRENA	0 1	Data length error interrupt enable. A data length error occurs under the following conditions. Master: When $\overline{\text{SPIENA}}$ is used, if the $\overline{\text{SPIENA}}$ pin from the slave is deasserted before the master has completed its transfer, the data length error is set. That is, if the character length counter has not overflowed while $\overline{\text{SPIENA}}$ deassertion is detected, then it means that the slave has neither received full data from the master nor has it transmitted complete data. Slave: When $\overline{\text{SPICS}}$ pins are used, if the incoming valid $\overline{\text{SPICS}}$ pin is deactivated before the character length counter overflows, then the data length error is set. No interrupt is generated upon data length error. An interrupt is asserted when a data-length error occurs.

27.9.4 SPI Interrupt Level Register (SPILVL)

Figure 27-29. SPI Interrupt Level Register (SPILVL) [offset = 0Ch]

31																	16
Reserved																	
R-0																	
15											10	9	8				
Reserved										TXINT LVL		RXINT LVL					
R-0										R/W-0		R/W-0					
7	6	5	4	3	2	1	0										
Reserved	RXOVRNINT LVL	Reserved	BITERR LVL	DESYNC LVL	PARERR LVL	TIMEOUT LVL	DLENERR LVL										
R-0	R/W-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0										

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-11. SPI Interrupt Level Register (SPILVL) Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	Reads return 0. Writes have no effect.
9	TXINTLVL	0 1	Transmit interrupt level. Transmit interrupt is mapped to interrupt line INT0. Transmit interrupt is mapped to interrupt line INT1.
8	RXINTLVL	0 1	Receive interrupt level. Receive interrupt is mapped to interrupt line INT0. Receive interrupt is mapped to interrupt line INT1.
7	Reserved	0	Reads return 0. Writes have no effect.
6	RXOVRNINTLVL	0 1	Receive overrun interrupt level. Receive overrun interrupt is mapped to interrupt line INT0. Receive overrun interrupt is mapped to interrupt line INT1.
5	Reserved	0	Reads return 0. Writes have no effect.
4	BITERRLVL	0 1	Bit error interrupt level. Bit error interrupt is mapped to interrupt line INT0. Bit error interrupt is mapped to interrupt line INT1.
3	DESYNCLVL	0 1	Desynchronized slave interrupt level. (master mode only). An interrupt caused by desynchronization of the slave is mapped to interrupt line INT0. An interrupt caused by desynchronization of the slave is mapped to interrupt line INT1.
2	PARERRLVL	0 1	Parity error interrupt level. A parity error interrupt is mapped to interrupt line INT0. A parity error interrupt is mapped to interrupt line INT1.
1	TIMEOUTLVL	0 1	SPIENA pin time-out interrupt level. An interrupt on a time-out of the ENA signal (TIMEOUT = 1) is mapped to interrupt line INT0. An interrupt on a time-out of the ENA signal (TIMEOUT = 1) is mapped to interrupt line INT1.
0	DLENERRLVL	0 1	Data length error interrupt level (line) select. An interrupt on data length error is mapped to interrupt line INT0. An interrupt on data length error is mapped to interrupt line INT1.

Table 27-12. SPI Flag Register (SPIFLG) Field Descriptions (continued)

Bit	Field	Value	Description
8	RXINTFLG	0 1	Receiver-full interrupt flag. This flag is set when a word is received and copied into the buffer register (SPIBUF). If RXINTEN is enabled, an interrupt is also generated. This bit is cleared under the following methods: • Reading the SPIBUF register • Reading TGINTVECT0 or TGINTVECT1 register when there is a receive buffer full interrupt • Writing a 1 to this bit • Writing a 0 to SPIEN (SPIGCR1[24]) • System reset During emulation mode, however, a read to the emulation register (SPIEMU) does not clear this flag bit. No new received data pending. Receive buffer is empty. A newly received data is ready to be read. Receive buffer is full. Note: Clearing RXINTFLG bit by writing a 1 before reading the SPIBUF sets the RXEMPTY bit of the SPIBUF register too. In this way, one can ignore a received word. However, if the internal RXBUF is already full, the data from RXBUF will be copied to SPIBUF and the RXEMPTY bit will be cleared again. The SPIBUF contents should be read first if this situation needs to be avoided.
7	Reserved	0	Reads return 0. Writes have no effect.
6	RXOVRNINTFLG	0 1	Receiver overrun flag. The SPI hardware sets this bit when a receive operation completes before the previous character has been read from the receive buffer. The bit indicates that the last received character has been overwritten and therefore lost. The SPI will generate an interrupt request if this bit is set and the RXOVRN INTEN bit (SPIINT0.6) is set high. This bit is cleared under the following conditions in compatibility mode of MibSPI: • Reading TGINTVECT0 or TGINTVECT1 register when there is a receive-buffer-overrun interrupt • Writing a 1 to RXOVRNINTFLG in the SPI Flag Register (SPIFLG) itself • Writing a 0 to SPIEN • Reading the data field of the SPIBUF register Note: Reading the SPIBUF register does not clear this RXOVRNINTFLG bit. If an RXOVRN interrupt is detected, then the SPIBUF may need to be read twice to get to the overrun buffer. This is due to the fact that the overrun will always occur to the internal RXBUF. Each read to the SPIBUF will result in RXBUF contents (if it is full) getting copied to SPIBUF. Note: There is a special condition under which the RXOVRNINTFLG flag gets set. If both SPIBUF and RXBUF are already full and while another reception is underway, if any errors (TIMEOUT, BITERR, and DLEN_ERR) occur, then RXOVRN in RXBUF and RXOVRNINTFLG in SPIFLG registers will be set to indicate that the status flags are getting overwritten by the new transfer. This overrun should be treated like a receive overrun. In multi-buffer mode of MibSPI, this bit is cleared under the following conditions: • Reading the RXOVRN_BUF_ADDR register • Writing a 1 to RXOVRNINTFLG in the SPI Flag Register (SPIFLG) itself In multi-buffer mode, if RXOVRNINTFLG is set, then the address of the buffer which experienced the overrun is available in RXOVRN_BUF_ADDR. Overrun condition did not occur. Overrun condition has occurred.
5	Reserved	0	Reads return 0. Writes have no effect.
4	BITERRFLG	0 1	Mismatch of internal transmit data and transmitted data. This flag can be cleared by one of the following methods. • Write a 1 to this bit • Clear the SPIEN bit to 0 No bit error occurred. A bit error occurred. The SPI samples the signal of the transmit pin (master: SIMO, slave: SOMI) at the receive point (half clock cycle after transmit point). If the sampled value differs from the transmitted value a bit error is detected and the flag BITERRFLG is set. If BITERRENA is set an interrupt is asserted. Possible reasons for a bit error can be an excessively high bit rate, capacitive load, or another master/slave trying to transmit at the same time.

Table 27-12. SPI Flag Register (SPIFLG) Field Descriptions (continued)

Bit	Field	Value	Description
3	DESYNCFLG	0 1	Desynchronization of slave device. Desynchronization monitor is active in master mode only. This flag can be cleared by one of the following methods: - Write a 1 to this bit - Clear the SPIEN bit to 0 No slave desynchronization is detected. A slave device is desynchronized. The master monitors the ENAble signal coming from the slave device and sets the DESYNC flag after the last bit is transmitted plus $t_{T2DELAY}$. If DESYNCENA is set an interrupt is asserted. Desynchronization can occur if a slave device misses a clock edge coming from the master.
2	PARERRFLG	0 1	Calculated parity differs from received parity bit. If the parity generator is enabled (can be selected individually for each buffer) an even or odd parity bit is added at the end of a data word. During reception of the data word the parity generator calculates the reference parity and compares it to the received parity bit. In the event of a mismatch the PARITYERR flag is set and an interrupt is asserted if PARERRENA is set. This flag can be cleared by one of the following methods: - Write a 1 to this bit - Clear the SPIEN bit to 0 No parity error is detected. A parity error occurred.
1	TIMEOUTFLG	0 1	Time-out caused by nonactivation of ENA signal. This flag can be cleared by one of the following methods: - Write a 1 to this bit - Clear the SPIEN bit to 0 No ENA-signal time-out occurred. An ENA signal time-out occurred. The SPI generates a time-out because the slave hasn't responded in time by activating the ENA signal after the chip select signal has been activated. If a time-out condition is detected the corresponding chip select is deactivated immediately and the TIMEOUT flag is set. In addition the TIMEOUT flag in the status field of the corresponding buffer is set. The transmit request of the concerned buffer is cleared, that is, the SPI does not re-start a data transfer from this buffer.
0	DLENERRFLG	0 1	Data-length error flag. This flag can be cleared by one of the following methods: - Write a 1 to this bit - Clear the SPIEN bit to 0 Note: Whenever any transmission errors (TIMEOUT, BITERR, DLEN_ERR, PARITY_ERR, DESYNC) are detected and the error flags are cleared by writing to the error bit in the SPIFLG register, the corresponding error flag in SPIBUF does not get cleared. Software needs to read the SPIBUF until it becomes empty before proceeding. This ensures that all of the old status bits in SPIBUF are cleared before starting the next transfer. No data length error has occurred. A data length error has occurred.

27.9.6 SPI Pin Control Register 0 (SPIPC0)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of SPIPC0 to SPIPC9 reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-31. SPI Pin Control Register 0 (SPIPC0) [offset = 14h]

31		24	23		16
SOMIFUN			SIMOFUN		
R/W-0			R/W-0		
15		12	11	10	9
Reserved		SOMIFUN0	SIMOFUN0	CLKFUN	ENAFUN
R-0		R/W-0	R/W-0	R/W-0	R/W-0
7					0
SCSFUN					
R/W-0					

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-13. SPI Pin Control (SPIPC0) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIFUN	0 The SPISOMI[x] pin is a GIO pin. 1 The SPISOMI[x] pin is a SPI functional pin.	Slave out, master in function. Determines whether each SPISOMI[x] pin is to be used as a general-purpose I/O pin or as a SPI functional pin. Note: Duplicate Control Bits for SPISOMI[0]. Bit 24 is not physically implemented. It is a mirror of Bit 11. Any write to bit 24 will be reflected on bit 11. When bit 24 and bit 11 are simultaneously written, the value of bit 11 will control the SPISOMI[0] pin. The read value of bit 24 always reflects the value of bit 11.
23-16	SIMOFUN	0 The SPISIMO[x] pin is a GIO pin. 1 The SPISIMO[x] pin is a SPI functional pin.	Slave in, master out function. Determines whether each SPISIMO[x] pin is to be used as a general-purpose I/O pin or as a SPI functional pin. Note: Duplicate Control Bits for SPISIMO[0]. Bit 16 is not physically implemented. It is a mirror of Bit 10. Any write to bit 16 will be reflected on bit 10. When bit 16 and bit 10 are simultaneously written, the value of bit 10 will control the SPISIMO[0] pin. The read value of bit 16 always reflects the value of bit 10.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIFUN0	0 The SPISOMI[0] pin is a GIO pin. 1 The SPISOMI[0] pin is a SPI functional pin.	Slave out, master in function. This bit determines whether the SPISOMI[0] pin is to be used as a general-purpose I/O pin or as a SPI functional pin. Note: Regardless of the number of parallel pins used, the SPISOMI[0] pin will always have to be programmed as functional pins for any SPI transfers.
10	SIMOFUN0	0 The SPISIMO[0] pin is a GIO pin. 1 The SPISIMO[0] pin is a SPI functional pin.	Slave in, master out function. This bits determine whether each SPISIMO[0] pin is to be used as a general-purpose I/O pin or as a SPI functional pin. Note: Regardless of the number of parallel pins used, the SPISIMO[0] pin will always have to be programmed as functional pins for any SPI transfers.

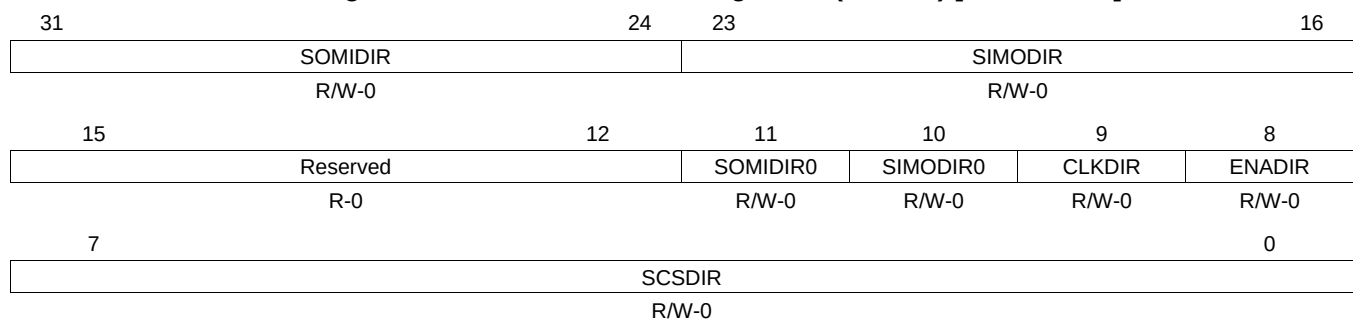
Table 27-13. SPI Pin Control (SPIPC0) Field Descriptions (continued)

Bit	Field	Value	Description
9	CLKFUN	0 1	SPI clock function. This bit determines whether the SPICLK pin is to be used as a general-purpose I/O pin, or as a SPI functional pin. The SPICLK pin is a GIO pin. The SPICLK pin is a SPI functional pin.
8	ENAFUN	0 1	$\overline{\text{SPIENA}}$ function. This bit determines whether the $\overline{\text{SPIENA}}$ pin is to be used as a general-purpose I/O pin or as a SPI functional pin. The $\overline{\text{SPIENA}}$ pin is a GIO pin. The $\overline{\text{SPIENA}}$ pin is a SPI functional pin.
7-0	SCSFUN	0 1	$\overline{\text{SPICS}}$ function. Determines whether each $\overline{\text{SPICS}}$ pin is to be used as a general-purpose I/O pin or as a SPI functional pin. If the slave $\overline{\text{SPICS}}$ pins are in functional mode and receive an inactive high signal, the slave SPI will place its output in a high-impedance state and disable shifting. The $\overline{\text{SPICS}}$ pin is a GIO pin. The $\overline{\text{SPICS}}$ pin is a SPI functional pin.

27.9.7 SPI Pin Control Register 1 (SPIPC1)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-32. SPI Pin Control Register 1 (SPIPC1) [offset = 18h]


LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-14. SPI Pin Control Register (SPIPC1) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIDIR	0 1	SPISOMI[x] direction. Controls the direction of each SPISOMI[x] pin when used for general-purpose I/O. If SPISOMI[x] pin is used as a SPI functional pin, the I/O direction is determined by the MASTER bit in the SPIGCR1 register. Note: Duplicate Control Bits for SPISOMI[0]. Bit 24 is not physically implemented. It is a mirror of Bit 11. Any write to bit 24 will be reflected on bit 11. When bit 24 and bit 11 are simultaneously written, the value of bit 11 will control the SPISOMI[0] pin. The read value of bit 24 always reflects the value of bit 11. The SPISOMI[x] pin is an input. The SPISOMI[x] pin is an output.

Table 27-14. SPI Pin Control Register (SPIPC1) Field Descriptions (continued)

Bit	Field	Value	Description
23-16	SIMODIR		SPISIMO[x] direction. Controls the direction of each SPISIMO[x] pin when used for general-purpose I/O. If SPISIMO[x] pin is used as a SPI functional pin, the I/O direction is determined by the MASTER bit in the SPIGCR1 register.
		0	The SPISIMO[x] pin is an input.
		1	The SPISIMO[x] pin is an output.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIDIR0		SPISOMI[0] direction. This bit controls the direction of the SPISOMI[0] pin when it is used as a general-purpose I/O pin. If the SPISOMI[0] pin is used as a SPI functional pin, the I/O direction is determined by the MASTER bit in the SPIGCR1 register.
		0	The SPISOMI[0] pin is an input.
		1	The SPISOMI[0] pin is an output.
10	SIMODIR0		SPISIMO[0] direction. This bit controls the direction of the SPISIMO[0] pin when it is used as a general-purpose I/O pin. If the SPISIMO[0] pin is used as a SPI functional pin, the I/O direction is determined by the MASTER bit in the SPIGCR1 register.
		0	The SPISIMO[0] pin is an input.
		1	The SPISIMO[0] pin is an output.
9	CLKDIR		SPICLK direction. This bit controls the direction of the SPICLK pin when it is used as a general-purpose I/O pin. In functional mode, the I/O direction is determined by the CLKMOD bit.
		0	The SPICLK pin is an input.
		1	The SPICLK pin is an output.
8	ENADIR		$\overline{\text{SPIENA}}$ direction. This bit controls the direction of the $\overline{\text{SPIENA}}$ pin when it is used as a general-purpose I/O. If the $\overline{\text{SPIENA}}$ pin is used as a functional pin, then the I/O direction is determined by the CLKMOD bit (SPIGCR1[1]).
		0	The $\overline{\text{SPIENA}}$ pin is an input.
		1	The $\overline{\text{SPIENA}}$ pin is an output.
7-0	SCSDIR		$\overline{\text{SPICS}}$ direction. These bits control the direction of each $\overline{\text{SPICS}}$ pin when it is used as a general-purpose I/O pin. Each pin could be configured independently from the others if the $\overline{\text{SPICS}}$ is used as a SPI functional pin. The I/O direction is determined by the CLKMOD bit (SPIGCR1[1]).
		0	The $\overline{\text{SPICS}}$ pin is an input.
		1	The $\overline{\text{SPICS}}$ pin is an output.

27.9.8 SPI Pin Control Register 2 (SPIPC2)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-33. SPI Pin Control Register 2 (SPIPC2) [offset = 1Ch]


LEGEND: R/W = Read/Write; R = Read only; U = Undefined; -n = value after reset

Table 27-15. SPI Pin Control Register 2 (SPIPC2) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIDIN		SPISOMI[x] data in. The value of each SPISOMI[x] pin.
		0	The SPISOMI[x] pin is logic 0.
		1	The SPISOMI[x] pin is logic 1.
23-16	SIMODIN		SPISIMO[x] data in. The value of each SPISIMO[x] pin.
		0	The SPISIMO[x] pin is logic 0.
		1	The SPISIMO[x] pin is logic 1.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIDIN0		SPISOMI[0] data in. The value of the SPISOMI[0] pin.
		0	The SPISOMI[0] pin is logic 0.
		1	The SPISOMI[0] pin is logic 1.
10	SIMODIN0		SPISIMO[0] data in. The value of the SPISIMO[0] pin.
		0	The SPISIMO[0] pin is logic 0.
		1	The SPISIMO[0] pin is logic 1.
9	CLKDIN		Clock data in. The value of the SPICLK pin.
		0	The SPICLK pin is logic 0.
		1	The SPICLK pin is logic 1.
8	ENADIN		$\overline{\text{SPIENA}}$ data in. The the value of the $\overline{\text{SPIENA}}$ pin.
		0	The $\overline{\text{SPIENA}}$ pin is logic 0.
		1	The $\overline{\text{SPIENA}}$ pin is logic 1.
7-0	SCSDIN		$\overline{\text{SPICS}}$ data in. The value of each $\overline{\text{SPICS}}$ pin.
		0	The $\overline{\text{SPICS}}$ pin is logic 0.
		1	The $\overline{\text{SPICS}}$ pin is logic 1.

27.9.9 SPI Pin Control Register 3 (SPIPC3)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-34. SPI Pin Control Register 3 (SPIPC3) [offset = 20h]

31		24	23		16
SOMIDOUT			SIMODOUT		
R/W-U			R/W-U		
15		12	11	10	9
Reserved			SOMIDOUT0	SIMODOUT0	CLKDOUT
R-0			R/W-U	R/W-U	R/W-U
7					0
SCSDOUT					
R/W-U					

LEGEND: R/W = Read/Write; R = Read only; U = Undefined; -n = value after reset

Table 27-16. SPI Pin Control Register 3 (SPIPC3) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIDOUT		SPISOMI[x] data out write. This bit is only active when the SPISOMI[x] pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin. Bit 11 or bit 24 can be used to set the direction for pin SPISOMI[0]. If a 32-bit write is performed, bit 11 will have priority over bit 24.
		0	Current value on SPISOMI[x] pin is logic 0.
		1	Current value on SPISOMI[x] pin is logic 1
23-16	SIMODOUT		SPISIMO[x] data out write. This bit is only active when the SPISIMO[x] pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin. Bit 10 or bit 16 can be used to set the direction for pin SPISIMO[0]. If a 32-bit write is performed, bit 10 will have priority over bit 16.
		0	Current value on SPISIMO[x] pin is logic 0.
		1	Current value on SPISIMO[x] pin is logic 1.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIDOUT0		SPISOMI[0] data out write. This bit is only active when the SPISOMI[0] pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin.
		0	Current value on SPISOMI[0] pin is logic 0.
		1	Current value on SPISOMI[0] pin is logic 1.
10	SIMODOUT0		SPISIMO[0] data out write. This bit is only active when the SPISIMO[0] pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin.
		0	Current value on SPISIMO[0] pin is logic 0.
		1	Current value on SPISIMO[0] pin is logic 1.
9	CLKDOUT		SPICLK data out write. This bit is only active when the SPICLK pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin.
		0	The SPICLK pin is logic 0.
		1	The SPICLK pin is logic 1.

Table 27-16. SPI Pin Control Register 3 (SPIPC3) Field Descriptions (continued)

Bit	Field	Value	Description
8	ENADOUT	0 1	$\overline{\text{SPIENA}}$ data out write. Only active when the $\overline{\text{SPIENA}}$ pin is configured as a general-purpose I/O pin and configured as an output pin. The value of this bit indicates the value sent to the pin. The $\overline{\text{SPIENA}}$ pin is logic 0. The $\overline{\text{SPIENA}}$ pin is logic 1.
7-0	SCSDOUT	0 1	$\overline{\text{SPICS}}$ data out write. Only active when the $\overline{\text{SPICS}}$ pins are configured as a general-purpose I/O pins and configured as output pins. The value of these bits indicates the value sent to the pins. The $\overline{\text{SPICS}}$ pin is logic 0. The $\overline{\text{SPICS}}$ pin is logic 1.

27.9.10 SPI Pin Control Register 4 (SPIPC4)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-35. SPI Pin Control Register 4 (SPIPC4) [offset = 24h]

31	24	23				16	
SOMISET				SIMOSET			
R/W-U				R/W-U			
15	12	11	10	9	8		
Reserved				SOMISET0	SIMOSET0	CLKSET	ENASET
R-0				R/W-U	R/W-U	R/W-U	R/W-U
7						0	
SCSSET							
R/W-U							

LEGEND: R/W = Read/Write; R = Read only; U = Undefined; -n = value after reset

Table 27-17. SPI Pin Control Register 4 (SPIPC4) Field Descriptions

Bit	Field	Value	Description
31-24	SOMISET	0 1	SPISOMI[x] data out set. This pin is only active when the SPISOMI[x] pin is configured as a general-purpose output pin. Bit 11 or bit 24 can be used to set the SPISOMI[0] pin. If a 32-bit write is performed, bit 11 will have priority over bit 24. Read: SPISOMI[x] is logic 0. Write: Writing a 0 to this bit has no effect. Read: SPISOMI[x] is logic 1. Write: Logic 1 is placed on SPISOMI[x] pin, if it is in general-purpose output mode.
23-16	SIMOSET	0 1	SPISIMO[x] data out set. This bit is only active when the SPISIMO[x] pin is configured as a general-purpose output pin. Bit 10 or bit 16 can be used to set the SPISIMO[0] pin. If a 32-bit write is performed, bit 10 will have priority over bit 16. Read: SPISIMO[x] is logic 0. Write: Writing a 0 to this bit has no effect. Read: SPISIMO[x] is logic 1. Write: Logic 1 is placed on SPISIMO[x] pin, if it is in general-purpose output mode.

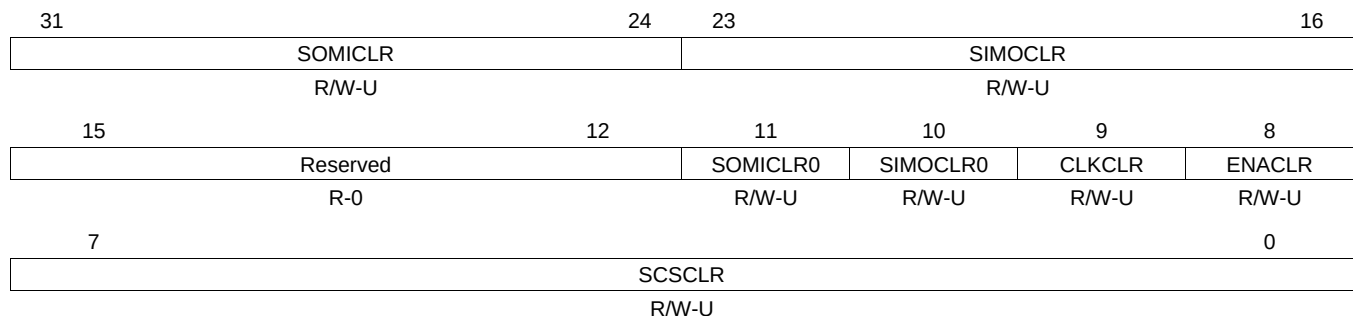
Table 27-17. SPI Pin Control Register 4 (SPIPC4) Field Descriptions (continued)

Bit	Field	Value	Description
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMISET0	0	SPISOMI[0] data out set. This pin is only active when the SPISOMI[0] pin is configured as a general-purpose output pin. Read: SPISOMI[0] is logic 0. Write: Writing a 0 to this bit has no effect.
		1	Read: SPISOMI[0] is logic 1. Write: Logic 1 is placed on SPISOMI[0] pin, if it is in general-purpose output mode.
10	SIMOSET0	0	SPISIMO[0] data out set. This pin is only active when the SPISIMO[0] pin is configured as a general-purpose output pin. Read: SPISIMO[0] is logic 0. Write: Writing a 0 to this bit has no effect.
		1	Read: SPISIMO[0] is logic 1. Write: Logic 1 is placed on SPISIMO[0] pin, if it is in general-purpose output mode.
9	CLKSET	0	SPICLK data out set. This bit is only active when the SPICLK pin is configured as a general-purpose output pin. Read: SPICLK is logic 0. Write: Writing a 0 to this bit has no effect.
		1	Read: SPICLK is logic 1. Write: Logic 1 is placed on SPICLK pin, if it is in general-purpose output mode.
8	ENASET	0	$\overline{\text{SPIENA}}$ data out set. This bit is only active when the $\overline{\text{SPIENA}}$ pin is configured as a general-purpose output pin. Read: $\overline{\text{SPIENA}}$ is logic 0. Write: Writing a 0 to this bit has no effect.
		1	Read: $\overline{\text{SPIENA}}$ is logic 1. Write: Logic 1 is placed on $\overline{\text{SPIENA}}$ pin, if it is in general-purpose output mode.
7-0	SCSSET	0	$\overline{\text{SPICS}}$ data out set. This bit is only active when the $\overline{\text{SPICS}}$ pin is configured as a general-purpose output pin. A value of 1 written to this bit sets the corresponding SCSDOUT bit to 1. Read: $\overline{\text{SPICS}}$ is logic 0. Write: Writing a 0 to this bit has no effect.
		1	Read: $\overline{\text{SPICS}}$ is logic 1. Write: Logic 1 is placed on $\overline{\text{SPICS}}$ pin, if it is in general-purpose output mode.

27.9.11 SPI Pin Control Register 5 (SPIPC5)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-36. SPI Pin Control Register 5 (SPIPC5) [offset = 28h]


LEGEND: R/W = Read/Write; R = Read only; U = Undefined; -n = value after reset

Table 27-18. SPI Pin Control Register 5 (SPIPC5) Field Descriptions

Bit	Field	Value	Description
31-24	SOMICLR		SPISOMI[x] data out clear. This pin is only active when the SPISOMI[x] pin is configured as a general-purpose output pin.
		0	Bit 11 or bit 24 can be used to set the SPISOMI[0] pin. If a 32-bit write is performed, bit 11 will have priority over bit 24. Read: The current value on SPISOMI[x] is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SPISOMI[x] is 1. Write: Logic 0 is placed on SPISOMI[x] pin, if it is in general-purpose output mode.
23-16	SIMOCLR		SPISIMO[x] data out clear. This bit is only active when the SPISIMO[x] pin is configured as a general-purpose output pin.
		0	Bit 10 or bit 16 can be used to set the SPISIMO[0] pin. If a 32-bit write is performed, bit 10 will have priority over bit 16. Read: The current value on SPISIMO[x] is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SPISIMO[x] is 1. Write: Logic 0 is placed on SPISIMO[x] pin, if it is in general-purpose output mode.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMICLR0		SPISOMI[0] data out clear. This pin is only active when the SPISOMI[0] pin is configured as a general-purpose output pin.
		0	Read: The current value on SPISOMI[0] is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SPISOMI[0] is 1. Write: Logic 0 is placed on SPISOMI[0] pin, if it is in general-purpose output mode.
10	SIMOCLR0		SPISIMO[0] data out clear. This pin is only active when the SPISIMO[0] pin is configured as a general-purpose output pin.
		0	Read: The current value on SPISIMO[0] is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SPISIMO[0] is 1. Write: Logic 0 is placed on SPISIMO[0] pin, if it is in general-purpose output mode.

Table 27-18. SPI Pin Control Register 5 (SPIPC5) Field Descriptions (continued)

Bit	Field	Value	Description
9	CLKCLR	0	SPICLK data out clear. This bit is only active when the SPICLK pin is configured as a general-purpose output pin. Read: The current value on SPICLK is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SPICLK is 1. Write: Logic 0 is placed on SPICLK pin, if it is in general-purpose output mode.
8	ENACLR	0	$\overline{\text{SPIEN}}$ data out clear. This bit is only active when the $\overline{\text{SPIEN}}$ pin is configured as a general-purpose output pin. A value of 1 written to this bit clears the corresponding ENABLEDOUT bit to 0. Read: The current value on $\overline{\text{SPIEN}}$ is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on $\overline{\text{SPIEN}}$ is 1. Write: Logic 0 is placed on $\overline{\text{SPIEN}}$ pin, if it is in general-purpose output mode.
7-0	SCSCLR	0	$\overline{\text{SPICS}}$ data out clear. This bit is only active when the $\overline{\text{SPICS}}$ pin is configured as a general-purpose output pin. Read: The current value on SCSDOUT is 0. Write: Writing a 0 to this bit has no effect.
		1	Read: The current value on SCSDOUT is 1. Write: Logic 0 is placed on $\overline{\text{SPICS}}$ pin, if it is in general-purpose output mode.

27.9.12 SPI Pin Control Register 6 (SPIPC6)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of SPIPC0 to SPIPC9 reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

Figure 27-37. SPI Pin Control Register 6 (SPIPC6) [offset = 2Ch]

31		24	23			16
SOMIPDR				SIMOPDR		
R/W-0				R/W-0		
15		12	11	10	9	8
Reserved		SOMIPDR0	SIMOPDR0	CLKPDR	ENAPDR	
R-0		R/W-0	R/W-0	R/W-0	R/W-0	
7						0
SCSPDR						
R/W-0						

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-19. SPI Pin Control Register 6 (SPIPC6) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIPDR	0 1	SPISOMI[x] open drain enable. This bit enables open drain capability for each SPISOMI[x] pin, if the following conditions are met: - SOMIDIRx = 1 (SPISOMI[x] pin is configured in GIO mode as an output pin) - SOMIDOUTx = 1 Bit 11 or bit 24 can both be used to enable open-drain for SPISOMI[0]. If a 32-bit write is performed, bit 11 will have priority over bit 24. Output value on the SPISOMI[x] pin is logic 1. Output pin SPISOMI[x] is in a high-impedance state.
23-16	SIMOPDR	0 1	SPISIMO[x] open drain enable. This bit enables open drain capability for each SPISIMO[x] pin, if the following conditions are met: - SIMODIRx = 1 (SPISIMO[x] pin is configured in GIO mode as an output pin) - SIMODOUTx = 1 Bit 10 or bit 16 can both be used to enable open-drain for SPISIMO[0]. If a 32-bit write is performed, bit 10 will have priority over bit 16. Output value on the SPISIMO[x] pin is logic 1. Output pin SPISIMO[x] is in a high-impedance state.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIPDR0	0 1	SPISOMI[0] open-drain enable. This bit enables open-drain capability for the SPISOMI[0] pin, if the following conditions are met: - SPISOMI[0] pin is configured in GIO mode as output pin - Output value on SPISOMI[0] pin is logic 1 Output value on the SPISOMI[0] pin is logic 1. Output pin SPISOMI[0] is in a high-impedance state.
10	SIMOPDR0	0 1	SPISIMO[0] open-drain enable. This bit enables open drain capability for the SPISIMO[0] pin, if the following conditions are met: - SPISIMO[0] pin is configured in GIO mode as output pin - Output value on SPISIMO[0] pin is logic 1 Output value on the SPISIMO[0] pin is logic 1. Output pin SPISIMO[0] is in a high-impedance state.
9	CLKPDR	0 1	SPICLK open drain enable. This bit enables open drain capability for the SPICLK pin, if the following conditions are met: - SPICLK pin is configured in GIO mode as an output pin - SPICLKDOUT = 1 Output value on the SPICLK pin is logic 1. Output pin SPICLK is in a high-impedance state.
8	ENAPDR	0 1	$\overline{\text{SPIENA}}$ open drain enable. This bit enables open drain capability for the $\overline{\text{SPIENA}}$ pin, if the following conditions are met: $\overline{\text{SPIENA}}$ pin is configured in GIO mode as an output pin - SPIENADOUT = 1 Output value on the $\overline{\text{SPIENA}}$ pin is logic 1. Output pin $\overline{\text{SPIENA}}$ is in a high-impedance state.
7-0	SCSPDR	0 1	$\overline{\text{SPICS}}$ open drain enable. This bit enables open drain capability for each $\overline{\text{SPICS}}$ pin, if the following conditions are met: $\overline{\text{SPICS}}$ pin is configured in GIO mode as an output pin - SCSDOUT = 1 Output value on the $\overline{\text{SPICS}}$ pin is logic 1. Output pin $\overline{\text{SPICS}}$ is in a high-impedance state.

27.9.13 SPI Pin Control Register 7 (SPIPC7)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

NOTE: Default Register Value

The default values of these register bits vary by device. See your device datasheet for information about default pin states, which correspond to the register reset values (see the pin-list table).

Figure 27-38. SPI Pin Control Register 7 (SPIPC7) [offset = 30h]

31		24	23		16
SOMIDIS			SIMODIS		
R/W-x			R/W-x		
15		12	11	10	9
Reserved			SOMIPDIS0	SIMOPDIS0	CLKPDIS
R-0			R/W-x	R/W-x	R/W-x
7					8
SCSPDIS					
R/W-x					

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset; -x = value varies by device

Table 27-20. SPI Pin Control Register 7 (SPIPC7) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIDIS	0 1	SPISOMI[x] pull control disable. This bit disables pull control capability for each SPISOMI[x] pin if it is in input mode, regardless of whether it is in functional or GIO mode. Note: Bit 11 or bit 24 can be used to set pull-disable for SPISOMI[0]. If a 32-bit write is performed, bit 11 will have priority over bit 24. Pull control on the SPISOMI[x] pin is enabled. Pull control on the SPISOMI[x] pin is disabled.
23-16	SIMODIS	0 1	SPISIMO[x] pull control disable. This bit disables pull control capability for each SPISIMO[x] pin if it is in input mode, regardless of whether it is in functional or GIO mode. Note: Bit 10 or bit 16 can be used to set pull-disable for SPISIMO[0]. If a 32-bit write is performed, bit 10 will have priority over bit 16. Pull control on the SPISIMO[x] pin is enabled. Pull control on the SPISIMO[x] pin is disabled.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIPDIS0	0 1	SPISOMI[0] pull control disable. This bit disables pull control capability for the SPISOMI[0] pin if it is in input mode, regardless of whether it is in functional or GIO mode. Pull control on the SPISOMI[0] pin is enabled. Pull control on the SPISOMI[0] pin is disabled.
10	SIMOPDIS0	0 1	SPISIMO[0] pull control disable. This bit disables pull control capability for the SPISIMO[0] pin if it is in input mode, regardless of whether it is in functional or GIO mode. Pull control on the SPISIMO[0] pin is enabled. Pull control on the SPISIMO[0] pin is disabled.

Table 27-20. SPI Pin Control Register 7 (SPIPC7) Field Descriptions (continued)

Bit	Field	Value	Description
9	CLKPDIS	0 1	SPICLK pull control disable. This bit disables pull control capability for the SPICLK pin if it is in input mode, regardless of whether it is in functional or GIO mode. Pull control on the SPICLK pin is enabled. Pull control on the SPICLK pin is disabled.
8	ENAPDIS	0 1	$\overline{\text{SPIENA}}$ pull control disable. This bit disables pull control capability for the $\overline{\text{SPIENA}}$ pin if it is in input mode, regardless of whether it is in functional or GIO mode. Pull control on the $\overline{\text{SPIENA}}$ pin is enabled. Pull control on the $\overline{\text{SPIENA}}$ pin is disabled.
7-0	SCSPDIS	0 1	$\overline{\text{SPICS}}$ pull control disable. This bit disables pull control capability for each $\overline{\text{SPICS}}$ pin if it is in input mode, regardless of whether it is in functional or GIO mode. Pull control on the $\overline{\text{SPICS}}$ pin is enabled. Pull control on the $\overline{\text{SPICS}}$ pin is disabled.

27.9.14 SPI Pin Control Register 8 (SPIPC8)

NOTE: Register bits vary by device

Register bits 31:24 and 23:16 of this register reflect the number of SIMO/SOMI data lines per device. On devices with 8 data-line support, all of bits 31 to 16 are implemented. On devices with less than 8 data lines, only a subset of these bits are available. Unimplemented bits return 0 upon read and are not writable.

NOTE: Default Register Value

The default values of these register bits vary by device. See your device datasheet for information about default pin states, which correspond to the register reset values (see the pin-list table).

Figure 27-39. SPI Pin Control Register 8 (SPIPC8) [offset = 34h]

31	24	23				16	
SOMIPSEL				SIMOPSEL			
R/W-x				R/W-x			
15	12	11	10	9	8		
Reserved				SOMIPSEL0	SIMOPSEL0	CLKPSEL	ENAPSEL
R-0				R/W-x	R/W-x	R/W-x	R/W-x
7						0	
SCSPSEL							
R/W-x							

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset; -x = value varies by device

Table 27-21. SPI Pin Control Register 8 (SPIPC8) Field Descriptions

Bit	Field	Value	Description
31-24	SOMIPSEL	0 1	SPISOMI[x] pull select. This bit selects the type of pull logic for each SPISOMI[x] pin. Note: Bit 11 or bit 24 can be used to set pull-select for SPISOMI[0]. If a 32-bit write is performed, bit 11 will have priority over bit 24. Pull down on the SPISOMI[x] pin. Pull up on the SPISOMI[x] pin.

Table 27-21. SPI Pin Control Register 8 (SPIPC8) Field Descriptions (continued)

Bit	Field	Value	Description
23-16	SIMOPSEL	0 1	SPISIMO[x] pull select. This bit selects the type of pull logic for each SPISIMO[x] pin. Note: Bit 10 or bit 16 can be used to set pull-select for SPISIMO[0]. If a 32-bit write is performed, bit 10 will have priority over bit 16. Pull down on the SPISIMO[x] pin. Pull up on the SPISIMO[x] pin.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMIPSEL0	0 1	SPISOMI[0] pull select. This bit selects the type of pull logic at the SPISOMI[0] pin. Pull down on the SPISOMI[0] pin. Pull up on the SPISOMI[0] pin.
10	SIMOPSEL0	0 1	SPISIMO[0] pull select. This bit selects the type of pull logic at the SPISIMO[0] pin. Pull down on the SPISIMO[0] pin. Pull up on the SPISIMO[0] pin.
9	CLKPSEL	0 1	SPICLK pull select. This bit selects the type of pull logic at the SPICLK pin. Pull down on the SPICLK pin. Pull up on the SPICLK pin.
8	ENAPSEL	0 1	$\overline{\text{SPIEN}}\text{A}$ pull select. This bit selects the type of pull logic at the $\overline{\text{SPIEN}}\text{A}$ pin. Pull down on the $\overline{\text{SPIEN}}\text{A}$ pin. Pull up on the $\overline{\text{SPIEN}}\text{A}$ pin.
7-0	SCSPSEL	0 1	$\overline{\text{SPICS}}$ pull select. This bit selects the type of pull logic for each $\overline{\text{SPICS}}$ pin. Pull down on the $\overline{\text{SPICS}}$ pin. Pull up on the $\overline{\text{SPICS}}$ pin.

27.9.15 SPI Transmit Data Register 0 (SPIDAT0)

Figure 27-40. SPI Transmit Data Register 0 (SPIDAT0) [offset = 38h]

31	Reserved	16
	R-0	
15	TXDATA	0
	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-22. SPI Transmit Data Register 0 (SPIDAT0) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	TXDATA	0-FFFFh	SPI transmit data. When written, these bits will be copied to the shift register if it is empty. If the shift register is not empty, TXBUF holds the written data. SPIEN (SPICGR1[24]) must be set to 1 before this register can be written to. Writing a 0 to the SPIEN register forces the lower 16 bits of the SPIDAT0 to 0x00. Note: When this register is read, the contents TXBUF, which holds the latest written data, will be returned. Note: Regardless of character length, the transmit word should be right-justified before writing to the SPIDAT1 register. Note: The default data format control register for SPIDAT0 is SPIFMT0. However, it is possible to reprogram the DFSEL[1:0] fields of SPIDAT1 before using SPIDAT0, to select a different SPIFMTx register. Note: It is highly recommended to use SPIDAT1 register, SPIDAT0 is supported for compatibility reasons.

27.9.16 SPI Transmit Data Register 1 (SPIDAT1)

NOTE: Writing to only the control fields, bits 28 through 16, does not initiate any SPI transfer in master mode. This feature can be used to set up SPICLK phase or polarity before actually starting the transfer by only updating the DFSEL bit field to select the required phase and polarity combination.

Figure 27-41. SPI Transmit Data Register 1 (SPIDAT1) [offset = 3Ch]

31	29	28	27	26	25	24	23	16
Reserved	CSHOLD	Rsvd	WDEL	DFSEL			CSNR	
R-0	R/W-0	R-0	R/W-0	R/W-0			R/W-0	
15								0
TXDATA								
R/W-0								

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-23. SPI Transmit Data Register 1 (SPIDAT1) Field Descriptions

Bit	Field	Value	Description
31-29	Reserved	0	Reads return 0. Writes have no effect.
28	CSHOLD	0	Chip select hold mode. The CSHOLD bit is supported in master mode only in compatibility-mode of SPI, (it is ignored in slave mode). CSHOLD defines the behavior of the chip select line at the end of a data transfer. The chip select signal is deactivated at the end of a transfer after the T2CDELAY time has passed. If two consecutive transfers are dedicated to the same chip select this chip select signal will be deactivated for at least 2VCLK cycles before it is activated again.
		1	The chip select signal is held active at the end of a transfer until a control field with new data and control information is loaded into SPIDAT1. If the new chip select number equals the previous one, the active chip select signal is extended until the end of transfer with CSHOLD cleared, or until the chip-select number changes.
27	Reserved	0	Reads return 0. Writes have no effect.
26	WDEL	0	Enable the delay counter at the end of the current transaction. Note: The WDEL bit is supported in master mode only. In slave mode, this bit will be ignored. No delay will be inserted. However, the SPICS pins will still be de-activated for at least for 2VCLK cycles if CSHOLD = 0.
		1	Note: The duration for which the SPICS pin remains deactivated depends upon the time taken to supply a new word after completing the shift operation. If TXBUF is already full, then the SPICS pin will be deasserted for at least two VCLK cycles (if WDEL = 0). After a transaction, WDELAY of the corresponding data format will be loaded into the delay counter. No transaction will be performed until the WDELAY counter overflows. The SPICS pins will be de-activated for at least (WDELAY + 2) × VCLK_Period duration.
25-24	DFSEL	0	Data word format select. Data word format 0 is selected.
		1h	Data word format 1 is selected.
		2h	Data word format 2 is selected.
		3h	Data word format 3 is selected.
23-16	CSNR	0-FFh	Chip select (CS) number. CSNR defines the chip select pins that will be activated during the data transfer. CSNR is a bit-mask that controls all chip select pins. See Table 27-24 . Note: If your MibSPI has less than 8 chip select pins, all unused upper bits will be 0. For example, MiBSPI3 has 6 chip select pins, if you write FFh to CSNR, the actual number stored in CSNR is 3Fh.

Table 27-23. SPI Transmit Data Register 1 (SPIDAT1) Field Descriptions (continued)

Bit	Field	Value	Description
15-0	TXDATA	0-FFFFh	Transfer data. When written, these bits are copied to the shift register if it is empty. If the shift register is not empty, then they are held in TXBUF. SPIEN must be set to 1 before this register can be written to. Writing a 0 to SPIEN forces the lower 16 bits of SPIDAT1 to 0x0000. A write to this register (or to the TXDATA field only) drives the contents of the CSNR field on the $\overline{\text{SPICS}}$ pins, if the pins are configured as functional pins (automatic chip select, see Section 27.2). When this register is read, the contents of TXBUF, which holds the latest data written, will be returned. Note: Regardless of the character length, the transmit data should be right-justified before writing to the SPIDAT1 register.

Table 27-24. Chip Select Number Active

CSNR Value	Chip Select Active:						CSNR Value	Chip Select Active:					
	CS[5] ⁽¹⁾	CS[4] ⁽¹⁾	CS[3] ⁽¹⁾	CS[2] ⁽¹⁾	CS[1] ⁽¹⁾	CS[0]		CS[5] ⁽¹⁾	CS[4] ⁽¹⁾	CS[3] ⁽¹⁾	CS[2] ⁽¹⁾	CS[1] ⁽¹⁾	CS[0]
0h	No chip select pin is active.						20h	x					
1h						x	21h	x					x
2h					x		22h	x				x	
3h					x	x	23h	x				x	x
4h				x			24h	x			x		
5h				x		x	25h	x			x		x
6h				x	x		26h	x			x	x	
7h				x	x	x	27h	x			x	x	x
8h			x				28h	x		x			
9h			x			x	29h	x		x			x
Ah			x		x		2Ah	x		x		x	
Bh			x		x	x	2Bh	x		x		x	x
Ch			x	x			2Ch	x		x	x		
Dh			x	x		x	2Dh	x		x	x		x
Eh			x	x	x		2Eh	x		x	x	x	
Fh			x	x	x	x	2Fh	x		x	x	x	x
10h		x					30h	x	x				
11h		x				x	31h	x	x				x
12h		x			x		32h	x	x			x	
13h		x			x	x	33h	x	x			x	x
14h		x		x			34h	x	x		x		
15h		x		x		x	35h	x	x		x		x
16h		x		x	x		36h	x	x		x	x	
17h		x		x	x	x	37h	x	x		x	x	x
18h		x	x				38h	x	x	x			
19h		x	x			x	39h	x	x	x			x
1Ah		x	x		x		3Ah	x	x	x		x	
1Bh		x	x		x	x	3Bh	x	x	x		x	x
1Ch		x	x	x			3Ch	x	x	x	x		
1Dh		x	x	x		x	3Dh	x	x	x	x		x
1Eh		x	x	x	x		3Eh	x	x	x	x	x	
1Fh		x	x	x	x	x	3Fh	x	x	x	x	x	x

⁽¹⁾ If your MibSPI does not have this chip select pin, this bit is 0.

27.9.17 SPI Receive Buffer Register (SPIBUF)

Figure 27-42. SPI Receive Buffer Register (SPIBUF) [offset = 40h]

31	30	29	28	27	26	25	24
RXEMPTY	RXOVR	TXFULL	BITERR	DESYNC	PARITYERR	TIMEOUT	DLENERR
R-1	R-0	R-0	R-0	R-0	R-0	R-0	R-0
23							16
LCSNR							
R-0							
15							0
RXDATA							
R-0							

LEGEND: R = Read only; -n = value after reset

Table 27-25. SPI Receive Buffer Register (SPIBUF) Field Descriptions

Bit	Field	Value	Description
31	RXEMPTY	0 1	Receive data buffer empty. When the host reads the RXDATA field or the entire SPIBUF register, it automatically sets the RXEMPTY flag. When a data transfer is completed, the received data is copied into RXDATA and the RXEMPTY flag is cleared. New data has been received and copied into RXDATA. No data has been received since the last read of RXDATA. This flag gets set to 1 under the following conditions: - Reading the RXDATA field of the SPIBUF register - Writing a 1 to clear the RXINTFLG bit in the SPI Flag Register (SPIFLG) Write-clearing the RXINTFLG bit before reading the SPIBUF indicates the received data is being ignored. Conversely, RXINTFLG can be cleared by reading the RXDATA field of SPIBUF (or the entire register).
30	RXOVR	0 1	Receive data buffer overrun. When a data transfer is completed and the received data is copied into RXBUF while it is already full, RXOVR is set. Overruns always occur to RXBUF, not to SPIBUF; the contents of SPIBUF are overwritten only after it is read by the Peripheral (VBUSP) master (CPU, DMA, or other host processor). If enabled, the RXOVRN interrupt is generated when RXBUF is overwritten, and reading either SPI Flag Register (SPIFLG) or SPIVCTx shows the RXOVRN condition. Two read operations from the SPIBUF register are required to reach the overwritten buffer word (one to read SPIBUF, which then transfers RXDATA into SPIBUF for the second read). Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. Note: A special condition under which RXOVR flag gets set. If both SPIBUF and RXBUF are already full and while another buffer receive is underway, if any errors such as TIMEOUT, BITERR, and DLEN_ERR occur, then RXOVR in RXBUF and SPI Flag Register (SPIFLG) registers will be set to indicate that the status flags are getting overwritten by the new transfer. This overrun should be treated like a normal receive overrun. No receive data overrun condition occurred since last read of the data field. A receive data overrun condition occurred since last read of the data field.
29	TXFULL	0 1	Transmit data buffer full. This flag is a read-only flag. Writing into the SPIDAT0 or SPIDAT1 field while the TX shift register is full will automatically set the TxFULL flag. Once the word is copied to the shift register, the TxFULL flag will be cleared. Writing to SPIDAT0 or SPIDAT1 when both TXBUF and the TX shift register are empty does not set the TxFULL flag. The transmit buffer is empty; SPIDAT0/SPIDAT1 is ready to accept a new data. The transmit buffer is full; SPIDAT0/SPIDAT1 is not ready to accept new data.

Table 27-25. SPI Receive Buffer Register (SPIBUF) Field Descriptions (continued)

Bit	Field	Value	Description
28	BITERR	0 1	Bit error. There was a mismatch of internal transmit data and transmitted data. Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. No bit error occurred. A bit error occurred. The SPI samples the signal of the transmit pins (master: SIMOx, slave: SOMIx) at the receive point (one-half clock cycle after the transmit point). If the sampled value differs from the transmitted value, a bit error is detected and the BITERR flag is set. Possible reasons for a bit error include noise, an excessively high bit rate, capacitive load, or another master/slave trying to transmit at the same time.
27	DESYNC	0 1	Desynchronization of slave device. This bit is valid in master mode only. The master monitors the ENA signal coming from the slave device and sets the DESYNC flag if ENA is deactivated before the last reception point or after the last bit is transmitted plus $t_{2EDELAY}$. If DESYNCENA is set, an interrupt is asserted. Desynchronization can occur if a slave device misses a clock edge coming from the master. Note: In the Compatibility Mode MibSPI, under some circumstances it is possible for a desync error detected for the previous buffer to be visible in the current buffer. This is because the receive completion flag/interrupt is generated when the buffer transfer is completed. But desynchronization is detected after the buffer transfer is completed. So, if the VBUS master reads the received data quickly when an RXINT is detected, then the status flag may not reflect the correct desync condition. In multi-buffer mode, the desync flag is always assured to be for the current buffer. Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. No slave desynchronization is detected. A slave device is desynchronized.
26	PARITYERR	0 1	Parity error. The calculated parity differs from the received parity bit. If the parity generator is enabled (selected individually for each buffer) an even or odd parity bit is added at the end of a data word. During reception of the data word, the parity generator calculates the reference parity and compares it to the received parity bit. If a mismatch is detected, the PARITYERR flag is set. Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. No parity error is detected. A parity error occurred.
25	TIMEOUT	0 1	Time-out because of non-activation of $\overline{\text{SPIEN}}$ pin. The SPI generates a time-out when the slave does not respond in time by activating the ENA signal after the chip select signal has been activated. If a time-out condition is detected, the corresponding chip select is deactivated immediately and the TIMEOUT flag is set. In addition, the TIMEOUT flag in the status field of the corresponding buffer and in the SPI Flag Register (SPIFLG) is set. Note: This bit is valid only in master mode. Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. No $\overline{\text{SPIEN}}$ pin time-out occurred. An $\overline{\text{SPIEN}}$ signal time-out occurred.
24	DLENERR	0 1	Data length error flag. Note: This flag is cleared to 0 when the RXDATA field of the SPIBUF register is read. No data-length error occurred. A data length error occurred.
23-16	LCSNR	0-FFh	Last chip select number. LCSNR in the status field is a copy of CSNR in the corresponding control field. It contains the chip select number that was activated during the last word transfer.
15-0	RXDATA	0-FFFFh	SPI receive data. This is the received word, transferred from the receive shift-register at the end of a transfer. Regardless of the programmed character length and the direction of shifting, the received data is stored right-justified in the register.

27.9.18 SPI Emulation Register (SPIEMU)

Figure 27-43. SPI Emulation Register (SPIEMU) [offset = 44h]

31	16
Reserved	
R-8000h	
15	0
EMU_RXDATA	
R-0	

LEGEND: R = Read only; -n = value after reset

Table 27-26. SPI Emulation Register (SPIEMU) Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	8000h	Reads return 0. Writes have no effect.
15-0	EMU_RXDATA	0-FFFFh	SPI receive data. The SPI emulation register is a mirror of the SPIBUF register. The only difference between SPIEMU and SPIBUF is that a read from SPIEMU does not clear any of the status flags.

27.9.19 SPI Delay Register (SPIDELAY)

Figure 27-44. SPI Delay Register (SPIDELAY) [offset = 48h]

31	24	23	16
C2TDELAY		T2CDELAY	
R/W-0		R/W-0	
15	8	7	0
T2EDELAY		C2EDELAY	
R/W-0		R/W-0	

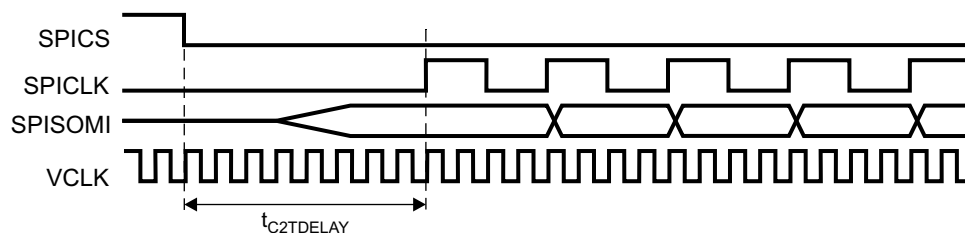
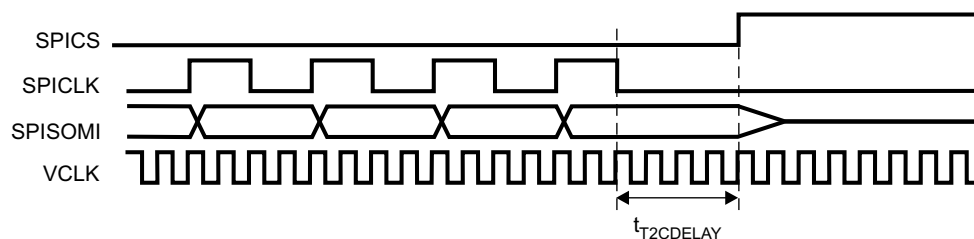
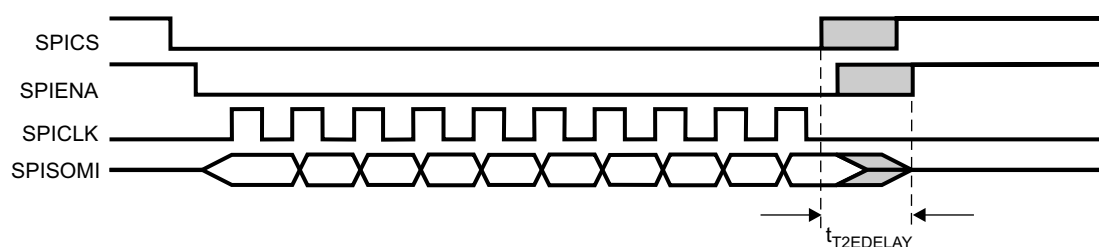
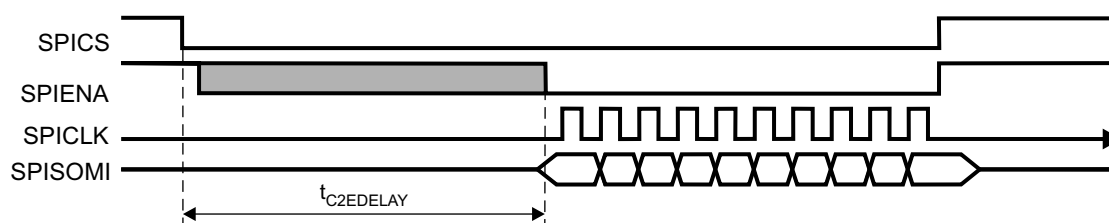
LEGEND: R/W = Read/Write; -n = value after reset

Table 27-27. SPI Delay Register (SPIDELAY) Field Descriptions

Bit	Field	Value	Description
31-24	C2TDELAY	0-FFh	Chip-select-active to transmit-start delay. See Figure 27-45 for an example. C2TDELAY is used only in master mode. It defines a setup time (for the slave device) that delays the data transmission from the chip select active edge by a multiple of VCLK cycles. The setup time value is calculated as follows. $t_{C2TDELAY} = (C2TDELAY + 2) \times VCLK \text{ Period}$ Example: VCLK = 25 MHz -> VCLK Period = 40ns; C2TDELAY = 07h; $t_{C2TDELAY} = 360 \text{ ns}$ When the chip select signal becomes active, the slave has to prepare data transfer within 360 ns. Note: If phase = 1, the delay between SPICS falling edge to the first edge of SPICLK will have an additional 0.5 SPICLK period delay. This delay is as per the SPI protocol.

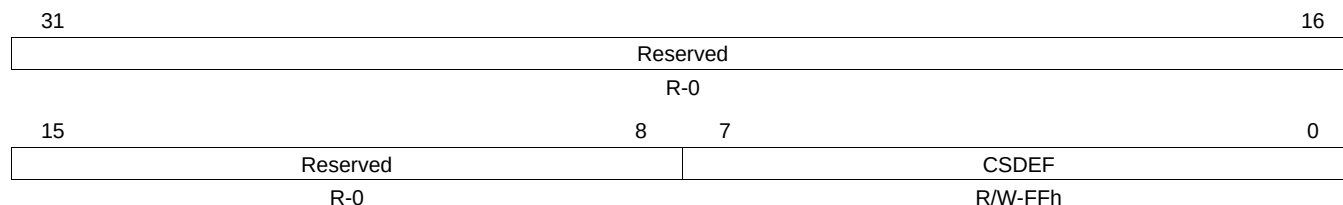
Table 27-27. SPI Delay Register (SPIDELAY) Field Descriptions (continued)

Bit	Field	Value	Description
23-16	T2CDELAY	0-FFh	Transmit-end-to-chip-select-inactive-delay. See Figure 27-46 for an example. T2CDELAY is used only in master mode. It defines a hold time for the slave device that delays the chip select deactivation by a multiple of VCLK cycles after the last bit is transferred. The hold time value is calculated as follows: $t_{T2CDELAY} = (T2CDELAY + 1) \times VCLK \text{ Period}$ Example: VCLK = 25 MHz → VCLK Period = 40ns; T2CDELAY = 03h; $> t_{T2CDELAY} = 160 \text{ ns}$ After the last data bit (or parity bit) is being transferred the chip select signal is held active for 160 ns. Note: If phase = 0, then between the last edge of SPICLK and rise-edge of SPICS there will be an additional delay of 0.5 SPICLK period. This is as per the SPI protocol. Both C2DELAY and T2CDELAY counters do not have any dependency on the $\overline{SPIENA}$ pin value. Even if the $\overline{SPIENA}$ pin is asserted by the slave, the master will continue to delay the start of SPICLK until the C2DELAY counter overflows. Similarly, even if the $\overline{SPIENA}$ pin is deasserted by the slave, the master will continue to hold the SPICS pins active until the T2CDELAY counter overflows. In this way, it is assured that the setup and hold times of the SPICS pins are determined by the delay timers alone. To achieve better throughput, it should be ensured that these two timers are kept at the minimum possible values.
15-8	T2EDELAY	0-FFh	Transmit-data-finished to ENA-pin-inactive time-out. T2EDELAY is used in master mode only. It defines a time-out value as a multiple of SPI clock before $\overline{SPIENA}$ signal has to become inactive and after SPICS becomes inactive. SPICLK depends on which data format is selected. If the slave device is missing one or more clock edges, it becomes de-synchronized. In this case, although the master has finished the data transfer, the slave is still waiting for the missed clock pulses and the ENA signal is not disabled. The T2EDELAY defines a time-out value that triggers the DESYNC flag, if the $\overline{SPIENA}$ signal is not deactivated in time. The DESYNC flag is set to indicate that the slave device did not de-assert its $\overline{SPIENA}$ pin in time to acknowledge that it received all bits of the sent word. See Figure 27-47 for an example of this condition. Note: DESYNC is also set if the SPI detects a de-assertion of $\overline{SPIENA}$ before the end of the transmission. The time-out value is calculated as follows: $t_{T2EDELAY} = T2EDELAY / SPI_{clock}$ Example: SPIclock = 8 Mbit/s; T2EDELAY = 10h; $> t_{T2EDELAY} = 2 \mu s$ The slave device has to disable the ENA signal within 2 μs, otherwise DESYNC is set and an interrupt is asserted (if enabled).
7-0	C2EDELAY	0-FFh	Chip-select-active to ENA-signal-active time-out. C2EDELAY is used only in master mode and it applies only if the addressed slave generates an ENA signal as a hardware handshake response. C2EDELAY defines the maximum time between when the SPI activates the chip-select signal and the addressed slave has to respond by activating the ENA signal. C2EDELAY defines a time-out value as a multiple of SPI clocks. The SPI clock depends on whether data format 0 or data format 1 is selected. See Figure 27-48 for an example of this condition. Note: If the slave device does not respond with the ENA signal before the time-out value is reached, the TIMEOUT flag in the SPIFLG register is set and a interrupt is asserted (if enabled). If a time-out occurs, the SPI clears the transmit request of the timed-out buffer, sets the TIMEOUT flag for the current buffer, and continues with the transfer of the next buffer in the sequence that is enabled. The timeout value is calculated as follows: $t_{C2EDELAY} = C2EDELAY / SPI_{clock}$ Example: SPIclock = 8 Mbit/s; C2EDELAY = 30h; $> t_{C2EDELAY} = 6 \text{ ms}$ The slave device has to activate the ENA signal within 6 ms after the SPI has activated the chip select signal (SPICS), otherwise the TIMEOUT flag is set and an interrupt is asserted (if enabled).

Figure 27-45. Example: $t_{C2TDELAY} = 8$ VCLK Cycles

Figure 27-46. Example: $t_{T2CDELAY} = 4$ VCLK Cycles

Figure 27-47. Transmit-Data-Finished-to-ENA-Inactive-Timeout

Figure 27-48. Chip-Select-Active-to-ENA-Signal-Active-Timeout


27.9.20 SPI Default Chip Select Register (SPIDEF)

Figure 27-49. SPI Default Chip Select Register (SPIDEF) [offset = 4Ch]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-28. SPI Default Chip Select Register (SPIDEF) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	CDEF	0-FFh	Chip select default pattern. Master-mode only.
		0	The CSDEF bits are output to the $\overline{\text{SPICS}}$ pins when no transmission is being performed. It allows the user to set a programmable chip-select pattern that deselects all of the SPI slaves.
		1	$\overline{\text{SPICS}}$ is cleared to 0 when no transfer is active.
			$\overline{\text{SPICS}}$ is set to 1 when no transfer is active.

27.9.21 SPI Data Format Registers (SPIFMT)

Figure 27-50. SPI Data Format Registers (SPIFMT[3:0]) [offset = 5Ch-50h]

31																24																															
WDELAY																																															
R/WP-0																																															
23				22				21				20				19				18				17				16																			
PARPOL				PARITYENA				WAITENA				SHIFTDIR				HDUPLEX_ENAx				DIS_CS_TIMERS				POLARITY				PHASE																			
R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0																			
15																8								7								5				4				0							
PRESCALE																Reserved								CHARLEN																							
R/WP-0																R-0								R/WP-0																							

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-29. SPI Data Format Registers (SPIFMT) Field Descriptions

Bit	Field	Value	Description
31-24	WDELAY	0-FFh	Delay in between transmissions for data format x (x= 0,1,2,3). Idle time that will be applied at the end of the current transmission if the bit WDEL is set in the current buffer. The delay to be applied is equal to: $WDELAY \times P_{VCLK} + 2 \times P_{VCLK}$ $P_{VCLK} \rightarrow \text{Period of VCLK.}$
23	PARPOL	0 1	Parity polarity: even or odd. PARPOLx can be modified in privilege mode only. It can be used for data format x (x= 0,1,2,3). An even parity flag is added at the end of the transmit data stream. An odd parity flag is added at the end of the transmit data stream.
22	PARITYENA	0 1	Parity enable for data format x. No parity generation/ verification is performed for this data format. A parity bit is transmitted at the end of each transmitted word. At the end of a transfer the parity generator compares the received parity bit with the locally-calculated parity flag. If the parity bits do not match, the RXERR flag is set in the corresponding control field. The parity type (even or odd) can be selected via the PARPOL bit. Note: If an uncorrectable error flag is set in a slave-mode SPI, then the wrong parity bit will be transmitted to indicate to the master that there has been some issue with the data parity. The SPISOMI pins will be forced to transmit all 0s, and the parity bit will be transmitted as 1 if even parity is selected and as 0 if odd parity is selected (using the PARPOLx bit of this register). This behavior occurs regardless of an uncorrectable parity error on either TXRAM or RXRAM.
21	WAITENA	0 1	The master waits for the ENA signal from slave for data format x. WAITENA is valid in master mode only. WAITENA enables a flexible SPI network where slaves with ENA signal and slaves without ENA signal can be mixed. WAITENA defines, for each transferred word, whether the addressed slave generates the ENA signal or not. The SPI does not wait for the ENA signal from the slave and directly starts the transfer. Before the SPI starts the data transfer it waits for the ENA signal to become low. If the ENA signal is not pulled down by the addressed slave before the internal time-out counter (C2EDELAY) overflows, then the master aborts the transfer and sets the TIMEOUT error flag.
20	SHIFTDIR	0 1	Shift direction for data format x. With bit SHIFTDIRx, the shift direction for data format x (x=0,1,2,3) can be selected. MSB is shifted out first. LSB is shifted out first.

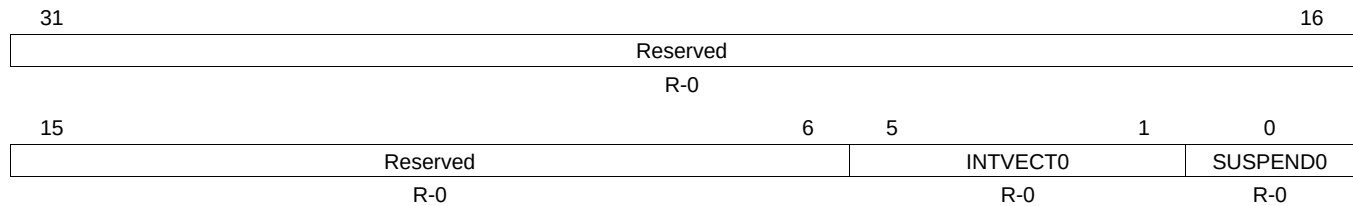
Table 27-29. SPI Data Format Registers (SPIFMT) Field Descriptions (continued)

Bit	Field	Value	Description
19	HDUPLEX_ENAx	0 1	Half Duplex transfer mode enable for Data Format x. This bit controls the I/O function of SOMI/SIMO lines for a specific requirement where in the case of Master mode, TX pin - SIMO will act as an RX pin, and in the case of Slave mode, RX pin - SIMO will act as a TX pin. Normal Full Duplex transfer. If MASTER = 1, SPISIMO pin will act as an RX pin (No TX possible) If MASTER = 0, SPISIMO pin will act as a TX pin (No RX possible). For all normal operations, HDUPLEX_ENAx bits should always remain 0. It is intended for the usage when the SPISIMO pin is used for both TX and RX operations at different times.
18	DIS CS TIMERS	0 1	Disable chip-select timers for this format. The C2TDELAY and T2CDELAY timers are by default enabled for all the data format registers. Using this bit, these timers can be disabled for a particular data format, if they are not required. When a master is handling multiple slaves, with varied set-up hold requirement, the application can selectively choose to include or not include the chip-select delay timers for any slaves. Both C2TDELAY and T2CDELAY counts are inserted for the chip selects. No C2TDELAY or T2CDELAY is inserted in the chip select timings.
17	POLARITY	0 1	SPI data format x clock polarity. POLARITYx defines the clock polarity of data format x. The following restrictions apply when switching clock phase and/or polarity: - In 3-pin/4-pin with $\overline{\text{SPIENA}}$ pin configuration of a slave SPI, the clock phase and polarity cannot be changed on-the-fly between two transfers. The slave should be reset and reconfigured if clock phase/polarity needs to be switched. In summary, SPI format switching is not fully supported in slave mode. - Even while using chip select pins, the polarity of SPICLK can be switched only while the slave is not selected by a valid chip select. The master SPI should ensure that while switching SPICLK polarity, it has deselected all of its slaves. Otherwise, the switching of SPICLK polarity may be incorrectly treated as a clock edge by some slaves. If POLARITYx is cleared to 0, the SPI clock signal is low-inactive, that is, before and after data transfer the clock signal is low. If POLARITYx is set to 1, the SPI clock signal is high-inactive, that is, before and after data transfer the clock signal is high.
16	PHASE	0 1	SPI data format x clock delay. PHASEx defines the clock delay of data format x. If PHASEx is cleared to 0, the SPI clock signal is not delayed versus the transmit/receive data stream. The first data bit is transmitted with the first clock edge and the first bit is received with the second (inverse) clock edge. If PHASEx is set to 1, the SPI clock signal is delayed by a half SPI clock cycle versus the transmit/receive data stream. The first transmit bit has to output prior to the first clock edge. The master and slave receive the first bit with the first edge.
15-8	PRESCALE		SPI data format x prescaler. PRESCALEx determines the bit transfer rate of data format x if the SPI is the network master. PRESCALEx is use to derive SPICLK from VCLK. If the SPI is configured as slave, PRESCALEx does not need to be configured. The clock rate for data format x can be calculated as: $\text{BR}_{\text{Formatx}} = \text{VCLK} / (\text{PRESCALEx} + 1)$ Note: When PRESCALEx is cleared to 0, the SPI clock rate defaults to VCLK/2.
7-5	Reserved	0	Reads return 0. Writes have no effect.
4-0	CHARLEN	0-1Fh	SPI data format x data-word length. CHARLENx defines the word length of data format x. Legal values are 0x02 (data word length = 2 bit) to 10h (data word length = 16). Illegal values, such as 00 or 1Fh are not allowed; their effect is indeterminate.

27.9.22 Interrupt Vector 0 (INTVECT0)

NOTE: The TG interrupt is not available in MibSPI in compatibility mode. Therefore, there is no possibility to access this register in compatibility mode.

Figure 27-51. Interrupt Vector 0 (INTVECT0) [offset = 60h]



LEGEND: R = Read only; -n = value after reset

Table 27-30. Transfer Group Interrupt Vector 0 (INTVECT0)

Bit	Field	Value	Description
31-6	Reserved	0	Reads return 0. Writes have no effect.
5-1	INTVECT0	0 1h + x 11h 13h 12h 14h All Other Combinations	INTVECT0. Interrupt vector for interrupt line INT0. Returns the vector of the pending interrupt at interrupt line INT0. If more than one interrupt is pending, INTVECT0 always references the highest prior interrupt source first. Note: This field reflects the status of the SPIFLG register in vector format. Any updates to the SPIFLG register will automatically cause updates to this field. There is no pending interrupt. Transfer group x (x = 0 to 15) has a pending interrupt. SUSPEND0 reflects the type of interrupt (<i>suspend or finished</i>). Error Interrupt pending. The lower half of SPIFLG contains more details about the type of error. The pending interrupt is a Receive Buffer Overrun interrupt. SPI mode: The pending interrupt is a Receive Buffer Full interrupt. Mib mode: Reserved. This bit combination should not occur. SPI mode: The pending interrupt is a Transmit Buffer Empty interrupt. Mib mode: Reserved. This bit combination should not occur. SPI mode: Reserved. These bit combinations should not occur.
0	SUSPEND0	0 1	Transfer suspended / Transfer finished interrupt flag. Every time INTVECT0 is read by the host, the corresponding interrupt flag of the referenced transfer group is cleared and INTVECT0 is updated with the vector coming next in the priority chain. The interrupt type is a transfer finished interrupt. In other words, the buffer array referenced by INTVECT0 has asserted an interrupt because all of data from the transfer group has been transferred. The interrupt type is a transfer suspended interrupt. In other words, the transfer group referenced by INTVECT0 has asserted an interrupt because the buffer to be transferred next is in suspend-to-wait mode.

NOTE: Reading from the INTVECT0 register when Transmit Empty is indicated does not clear the TXINTFLG flag in the SPI Flag Register (SPIFLG). Writing a new word to the SPIDATx register clears the Transmit Empty interrupt.

NOTE: In multi-buffer mode, INTVECT0 contains the interrupt for the highest priority transfer group. A read from INTVECT0 automatically causes the next-highest priority transfer group's interrupt status to get loaded into INTVECT0 and its corresponding SUSPEND flag to get loaded into SUSPEND0. The transfer group with the lowest number has the highest priority, and the transfer group with the highest number has the lowest priority.

Reading the INTVECT0 register when the RXOVRN interrupt is indicated in multi-buffer mode does not clear the RXOVRN flag and hence does not clear the vector. The RXOVRN interrupt vector may be cleared in multi-buffer mode either by write-clearing the RXOVRN flag in the SPI Flag Register (SPIFLG) or by reading the RXRAM Overrun Buffer Address Register (RXOVRN_BUF_ADDR).

27.9.23 Interrupt Vector 1 (INTVECT1)

NOTE: The TG interrupt is not available in SPI in compatibility mode compatibility mode. Therefore, there is no possibility to access this register in compatibility mode.

Figure 27-52. Interrupt Vector 1 (INTVECT1) [offset = 64h]

31													16
Reserved													
R-0													
15						6	5				1	0	
Reserved						INTVECT1				SUSPEND1			
R-0						R-0				R-0			

LEGEND: R = Read only; -n = value after reset

Table 27-31. Transfer Group Interrupt Vector 1 (INTVECT1)

Bit	Field	Value	Description
31-6	Reserved	0	Reads return 0. Writes have no effect.
5-1	INTVECT1	0 11h 13h 12h 14h All Other Combinations	INTVECT1. Interrupt vector for interrupt line INT1. Returns the vector of the pending interrupt at interrupt line INT1. If more than one interrupt is pending, INTVECT1 always references the highest prior interrupt source first. Note: This field reflects the status of the SPIFLG register in vector format. Any updates to the SPIFLG register will automatically cause updates to this field. There is no pending interrupt. SPI mode only. Error Interrupt pending. The lower half of SPIINT1 contains more details about the type of error. SPI mode only. The pending interrupt is a Receive Buffer Overrun interrupt. SPI mode only. The pending interrupt is a Receive Buffer Full interrupt. SPI mode only. The pending interrupt is a Transmit Buffer Empty interrupt. SPI mode only. Reserved. These bit combinations should not occur. SPI mode only.
0	SUSPEND1	0 1	Transfer suspended / Transfer finished interrupt flag. Every time INTVECT1 is read by the host, the corresponding interrupt flag of the referenced transfer group is cleared and INTVECT1 is updated with the vector coming next in the priority chain. The interrupt type is a transfer finished interrupt. In other words, the buffer array referenced by INTVECT1 has asserted an interrupt because all of data from the transfer group has been transferred. The interrupt type is a transfer suspended interrupt. In other words, the transfer group referenced by INTVECT1 has asserted an interrupt because the buffer to be transferred next is in suspend-to-wait mode.

NOTE: Reading from the INTVECT1 register when Transmit Empty is indicated does not clear the TXINTFLG flag in the SPI Flag Register (SPIFLG). Writing a new word to the SPIDATx register clears the Transmit Empty interrupt.

NOTE: In multi-buffer mode, INTVECT1 contains the interrupt for the highest priority transfer group. A read from INTVECT1 automatically causes the next-highest priority transfer group's interrupt status to get loaded into INTVECT1 and its corresponding SUSPEND flag to get loaded into SUSPEND1. The transfer group with the lowest number has the highest priority, and the transfer group with the highest number has the lowest priority.

Reading the INTVECT1 register when the RXOVRN interrupt is indicated in multi-buffer mode does not clear the RXOVRN flag and hence does not clear the vector. The RXOVRN interrupt vector may be cleared in multi-buffer mode either by write-clearing the RXOVRN flag in the SPI Flag Register (SPIFLG) or by reading the RXOVERN_BUF_ADDR register.

27.9.24 SPI Pin Control Register 9 (SPIPC9)

SPIPC9 only applies to SPI2.

Figure 27-53. SPI Pin Control Register 9 (SPIPC9) [offset = 68h]

31	25	24	23	17	16
Reserved		SOMISRS0	Reserved		SIMOSRS0
R-0		R/W-0	R-0		R/W-0
15	12	11	10	9	8
Reserved	SOMISRS0	SIMOSRS0	CLKSRS	Reserved	
R-0	R/W-0	R/W-0	R/W-0	R-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-32. SPI Pin Control Register 9 (SPIPC9) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return the value that was last written. Writes have no effect.
24	SOMISRS0	0 1	SPI2 SOMI[0] slew control. This bit controls between the fast or slow slew mode. Note: Duplicate Control Bits for SPI2 SOMI[0]. Bit 24 is not physically implemented. It is a mirror of bit 11. Any write to bit 24 will be reflected on bit 11. When bit 24 and bit 11 are simultaneously written, the value of bit 11 will control the SPI2 SOMI[0] pin. The read value of bit 24 always reflects the value of bit 11. Fast mode is enabled; the normal output buffer is used for this pin. Slow mode is enabled; slew rate control is used for this pin.
23-17	Reserved	0	Reads return the value that was last written. Writes have no effect.
16	SIMOSRS0	0 1	SPI2 SIMO[0] slew control. This bit controls between the fast or slow slew mode. Note: Duplicate Control Bits for SPI2 SIMO[0]. Bit 16 is not physically implemented. It is a mirror of bit 10. Any write to bit 16 will be reflected on bit 10. When bit 16 and bit 10 are simultaneously written, the value of bit 10 will control the SPI2 SIMO[0] pin. The read value of bit 16 always reflects the value of bit 10. Fast mode is enabled; the normal output buffer is used for this pin. Slow mode is enabled; slew rate control is used for this pin.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11	SOMISRS0	0 1	SPI2 SOMI[0] slew control. This bit controls between the fast or slow slew mode. Fast mode is enabled; the normal output buffer is used for this pin. Slow mode is enabled; slew rate control is used for this pin.
10	SIMOSRS0	0 1	SPI2 SIMO[0] slew control. This bit controls between the fast or slow slew mode. Fast mode is enabled; the normal output buffer is used for this pin. Slow mode is enabled; slew rate control is used for this pin.
9	CLKSRS	0 1	SPI2 CLK slew control. This bit controls between the fast or slow slew mode. Fast mode is enabled; the normal output buffer is used for this pin. Slow mode is enabled; slew rate control is used for this pin.
8-0	Reserved	0	Reads return the value that was last written. Writes have no effect.

27.9.25 Parallel/Modulo Mode Control Register (SPIPMCTRL)

NOTE: Do not configure MODCLKPOLx and MMODEx bits since this device does not support modulo mode.

NOTE: The bits of this register are used in conjunction with the SPIFMTx registers. Each byte of this register corresponds to one of the SPIFMTx registers.

1. Byte0 (Bits 7:0) are used when SPIFMT0 register is selected by DFSEL[1:0] = 00 in the control field of a buffer.
2. Byte1 (Bits 15:8) are used when SPIFMT1 register is selected by DFSEL[1:0] = 01 in the control field of a buffer.
3. Byte2 (Bits 23:16) are used when SPIFMT2 register is selected by DFSEL[1:0] = 10 in the control field of a buffer.
4. Byte3 (Bits 31:24) are used when SPIFMT3 register is selected by DFSEL[1:0] = 11 in the control field of a buffer.

Figure 27-54. Parallel/Modulo Mode Control Register (SPIPMCTRL) [offset = 6Ch]

31	30	29	28	26	25	24
Reserved	MODCLKPOL3		MMODE3		PMODE3	
R-0	R/WP-0		R/WP-0		R/WP-0	
23	22	21	20	18	17	16
Reserved	MODCLKPOL2		MMODE2		PMODE2	
R-0	R/WP-0		R/WP-0		R/WP-0	
15	14	13	12	10	9	8
Reserved	MODCLKPOL1		MMODE1		PMODE1	
R-0	R/WP-0		R/WP-0		R/WP-0	
7	6	5	4	2	1	0
Reserved	MODCLKPOL0		MMODE0		PMODE0	
R-0	R/WP-0		R/WP-0		R/WP-0	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-33. SPI Parallel/Modulo Mode Control Register (SPIPMCTRL) Field Descriptions

Bit	Field	Value	Description
31-30	Reserved	0	Reads return 0. Writes have no effect.
29	MODCLKPOL3	0 1	Modulo mode SPICLK polarity. This bit determines the polarity of the SPICLK in modulo mode only. If the MMODE3 bits are 000, this bit will be ignored. Normal SPICLK in all the modes. Polarity of the SPICLK will be inverted if Modulo mode is selected.
28-26	MMODE3	0 1h 2h 3h 4h 5h 6h-7h	These bits determine whether the SPI/MibSPI operates with 1, 2, 4, 5, or 6 data lines (if modulo option is supported by the module). Normal single data line mode - default (PMODE3 should be set to 00) 2-data line mode (PMODE3 should be set to 00) 3-data line mode (PMODE3 should be set to 00) 4-data line mode (PMODE3 should be set to 00) 5-data line mode (PMODE3 should be set to 00) 6-data line mode (PMODE3 should be set to 01) Reserved

Table 27-33. SPI Parallel/Modulo Mode Control Register (SPIPMCTRL) Field Descriptions (continued)

Bit	Field	Value	Description
25-24	PMODE3	0 1h 2h 3h	Parallel mode bits determine whether the SPI/MibSPI operates with 1, 2, 4, or 8 data lines. Normal operation/1-data line (MMODE3 should be set to 000) 2-data line mode (MMODE3 should be set to 000) 4-data line mode (MMODE3 should be set to 000) 8-data line mode (MMODE3 should be set to 000)
23-22	Reserved	0	Reads return 0. Writes have no effect.
21	MODCLKPOL2	0 1	Modulo mode SPICLK polarity. This bit determines the polarity of the SPICLK in modulo mode only. If the MMODE2 bits are 000, this bit will be ignored. Normal SPICLK in all the modes. Polarity of the SPICLK will be inverted if Modulo mode is selected.
20-18	MMODE2	0 1h 2h 3h 4h 5h 6h-7h	These bits determine whether the SPI/MibSPI operates with 1, 2, 4, 5, or 6 data lines (if modulo option is supported by the module). 1-data line Mode - default (PMODE2 should be set to 00) 2-data line Mode (PMODE2 should be set to 00) 3-data line mode (PMODE2 should be set to 00) 4-data line mode (PMODE2 should be set to 00) 5-data line mode (PMODE2 should be set to 00) 6-data line mode (PMODE2 should be set to 01) Reserved
17-16	PMODE2	0 1h 2h 3h	Parallel mode bits determine whether the SPI/MibSPI operates with 1, 2, 4, or 8 data lines. Normal operation/1-data line (MMODE2 should be set to 000) 2-data line mode (MMODE2 should be set to 000) 4-data line mode (MMODE2 should be set to 000) 8-data line mode (MMODE2 should be set to 000)
15-14	Reserved	0	Reads return 0. Writes have no effect.
13	MODCLKPOL1	0 1	Modulo mode SPICLK polarity. This bit determines the polarity of the SPICLK in modulo mode only. If the MMODE1 bits are 000, this bit will be ignored. Normal SPICLK in all the modes. Polarity of the SPICLK will be inverted if Modulo mode is selected.
12-10	MMODE1	0 1h 2h 3h 4h 5h 6h-7h	These bits determine whether the SPI/MibSPI operates with 1, 2, 4, 5, or 6 data lines (if modulo option is supported by the module). 1-data line mode - default (PMODE1 should be set to 00) 2-data line mode (PMODE1 should be set to 00) 3-data line mode (PMODE1 should be set to 00) 4-data line mode (PMODE1 should be set to 00) 5-data line mode (PMODE1 should be set to 00) 6-data line mode (PMODE1 should be set to 01) Reserved
9-8	PMODE1	0 1h 2h 3h	Parallel mode bits determine whether the SPI/MibSPI operates with 1, 2, 4, or 8 data lines. Normal operation/1-data line (MMODE1 should be set to 000) 2-data line mode (MMODE1 should be set to 000) 4-data line mode (MMODE1 should be set to 000) 8-data line mode (MMODE1 should be set to 000)
7-6	Reserved	0	Reads return 0. Writes have no effect.
5	MODCLKPOL0	0 1	Modulo mode SPICLK polarity. This bit determines the polarity of the SPICLK in modulo mode only. If the MMODE0 bits are 000, this bit will be ignored. Normal SPICLK in all the modes. Polarity of the SPICLK will be inverted if Modulo mode is selected.

Table 27-33. SPI Parallel/Modulo Mode Control Register (SPIPMCTRL) Field Descriptions (continued)

Bit	Field	Value	Description
4-2	MMODE0	0	These bits determine whether the SPI/MibSPI operates with 1, 2, 4, 5, or 6 data lines (if modulo option is supported by the module). 1-data line mode - default (PMODE0 should be set to 00)
		1h	2-data line mode (PMODE0 should be set to 00)
		2h	3-data line mode (PMODE0 should be set to 00)
		3h	4-data line mode (PMODE0 should be set to 00)
		4h	5-data line mode (PMODE0 should be set to 00)
		5h	6-data line mode (PMODE0 should be set to 01)
		6h-7h	Reserved
1-0	PMODE0	0	Parallel mode bits determine whether the SPI/MibSPI operates with 1, 2, 4, or 8 data lines. Normal operation/1-data line (MMODE0 should be set to 000)
		1h	2-data line mode (MMODE0 should be set to 000)
		2h	4-data line mode (MMODE0 should be set to 000)
		3h	8-data line mode (MMODE0 should be set to 000)

27.9.26 Multi-buffer Mode Enable Register (MIBSPIE)

NOTE: Accessibility of Multi-Buffer RAM

The multi-buffer RAM is not accessible unless the MSPIENA bit set to 1. The only exception to this is in test mode, where, by setting RXRAMACCESS to 1, the multi-buffer RAM can be fully accessed for both read and write.

Figure 27-55. Multi-buffer Mode Enable Register (MIBSPIE) [offset = 70h]

31	17	16
Reserved		RXRAM_ACCESS
R-0		R/WP-0
15	1	0
Reserved		MSPIENA
R-0		R/WP-0

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-34. Multi-buffer Mode Enable Register (MIBSPIE) Field Descriptions

Bit	Field	Value	Description
31-17	Reserved	0	Reads return 0. Writes have no effect.
16	RXRAM ACCESS	0 1	Receive-RAM access control. During normal operating mode of SPI, the receive data/status portion of multi-buffer RAM is read-only. To enable testing of receive RAM, direct read/write access is enabled by setting this bit. The RX portion of multi-buffer RAM is not writable by the CPU. The whole of multi-buffer RAM is fully accessible for read/write by the CPU. Note: The RX RAM ACCESS bit remains 0 after reset and it should remain cleared to 0 at all times, except when testing the RAM. SPI should be given a local reset by using the nRESET (SPIGCR0[0]) bit after RAM testing is performed so that the multi-buffer RAM gets re-initialized.
15-1	Reserved	0	Reads return 0. Writes have no effect.
0	MSPIENA	0 1	Multi-buffer mode enable. After power-up or reset, MSPIENA remains cleared, which means that the SPI runs in compatibility mode by default. If multi-buffer mode is desired, this register should be configured first after configuring the SPIGCR0 register. If MSPIENA is not set to 1, the multi-buffer mode registers are not writable. The SPI runs in compatibility mode, that is, in this mode the MibSPI is fully code-compliant to the standard device SPI. No multi-buffered-mode features are supported. The SPI is configured to run in multi-buffer mode.

NOTE: Accessibility of Registers

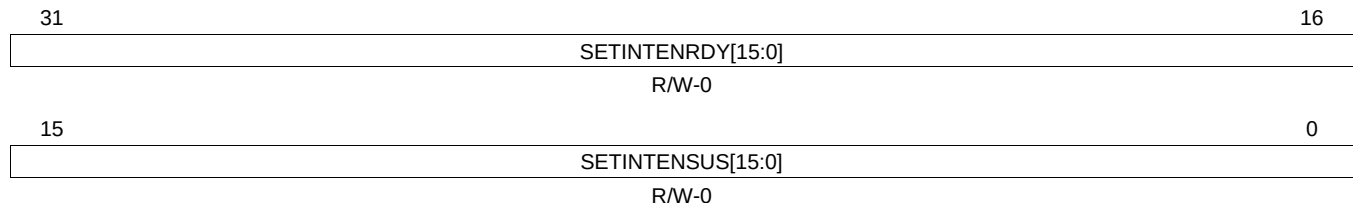
Registers from this offset address onwards are not accessible in SPI compatibility mode. They are accessible only in the multi-buffer mode.

27.9.27 TG Interrupt Enable Set Register (TGITENST)

The register TGITENST contains the TG interrupt enable flags for transfer-finished and for transfer-suspended events. Each of the enable bits in the higher half-word and the lower half-word of TGITENST belongs to one TG.

The register map shown in [Figure 27-56](#) and [Table 27-35](#) represents a super-set device with the maximum number of TGs (16) assumed. The actual number of bits available varies per device.

Figure 27-56. TG Interrupt Enable Set Register (TGITENST) [offset = 74h]



LEGEND: R/W = Read/Write; -n = value after reset

Table 27-35. TG Interrupt Enable Set Register (TGITENST) Field Descriptions

Bit	Field	Value	Description
31-16	SETINTENRDY[n]	0	TG interrupt set (enable) when transfer finished. Bit 16 corresponds to TG0, bit 17 corresponds to TG1, and so on. Read: The TGx-completed interrupt is disabled. This interrupt does not get generated when TGx completes. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is enabled. The interrupt gets generated when TGx completes. Write: Enable the TGx-completed interrupt. The interrupt gets generated when TGx completes.
15-0	SETINTENSUS[n]	0	TG interrupt set (enabled) when transfer suspended. Bit 0 corresponds to TG0, bit 1 corresponds to TG1, and so on. Read: The TGx-completed interrupt is disabled. This interrupt does not get generated when TGx is suspended. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is enabled. The interrupt gets generated when TGx is suspended. Write: Enable the TGx-completed interrupt. The interrupt gets generated when TGx is suspended.

27.9.28 TG Interrupt Enable Clear Register (TGITENCR)

The register TGITENCR is used to clear the interrupt enables for the TG-completed interrupt and the TG-suspended interrupts.

The register map shown in [Figure 27-57](#) and [Table 27-36](#) represents a super-set device with the maximum number of TGs (16) assumed. The actual number of bits available varies per device.

Figure 27-57. TG Interrupt Enable Clear Register (TGITENCR) [offset = 78h]

31	16
CLRINTENRDY[15:0]	
R/W-0	
15	0
CLRINTENSUS[15:0]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Table 27-36. TG Interrupt Enable Clear Register (TGITENCR) Field Descriptions

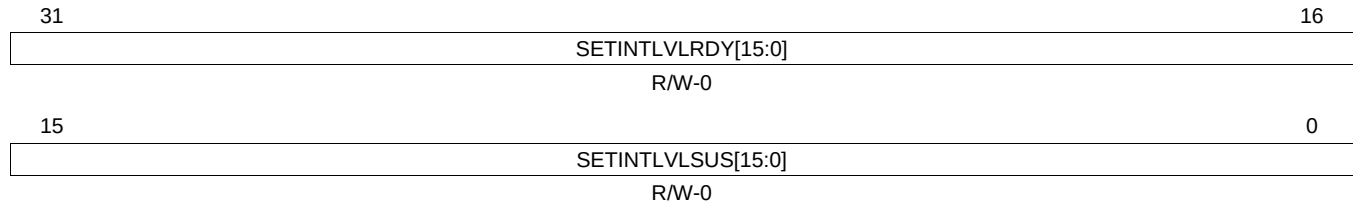
Bit	Field	Value	Description
31-16	CLRINTENRDY[n]	0	TG interrupt clear (disabled) when transfer finished. Bit 16 corresponds to TG0, bit 17 corresponds to TG1, and so on. Read: The TGx-completed interrupt is disabled. This interrupt does not get generated when TGx completes. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is enabled. The interrupt gets generated when TGx completes. Write: Disable the TGx-completed interrupt. The interrupt does not get generated when TGx completes.
15-0	CLRINTENSUS[n]	0	TG interrupt clear (disabled) when transfer suspended. Bit 0 corresponds to TG0, bit 1 corresponds to TG1, and so on. Read: The TGx-completed interrupt is disabled. This interrupt does not get generated when TGx is suspended. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is enabled. The interrupt gets generated when TGx is suspended. Write: Disable the TGx-completed interrupt. The interrupt does not get generated when TGx is suspended.

27.9.29 Transfer Group Interrupt Level Set Register (TGITLVST)

The register TGITLVST sets the level of interrupts for transfer completed interrupt and for transfer suspended interrupt to level 1.

The register map shown in [Figure 27-58](#) and [Table 27-37](#) represents a super-set device with the maximum number of TGs (16) assumed. The actual number of bits available varies per device.

Figure 27-58. Transfer Group Interrupt Level Set Register (TGITLVST) [offset = 7Ch]



LEGEND: R/W = Read/Write; -n = value after reset

Table 27-37. Transfer Group Interrupt Level Set Register (TGITLVST) Field Descriptions

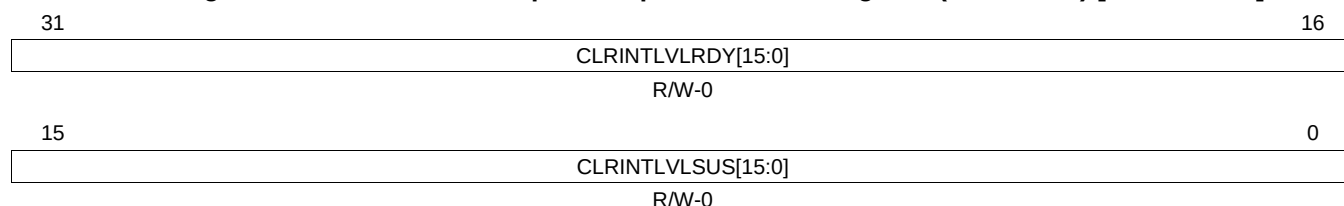
Bit	Field	Value	Description
31-16	SETINTLVLRDY[n]		Transfer-group completed interrupt level set. Bit 16 corresponds to TG0, bit 17 corresponds to TG1, and so on.
		0	Read: The TGx-completed interrupt is set to INT0. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is set to INT1. Write: Set the TGx-completed interrupt to INT1.
15-0	SETINTLVLSUS[n]		Transfer-group suspended interrupt level set. Bit 0 corresponds to TG0, bit 1 corresponds to TG1, and so on.
		0	Read: The TGx-suspended interrupt is set to INT0. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-suspended interrupt is set to INT1. Write: Set the TGx-suspended interrupt to INT1.

27.9.30 Transfer Group Interrupt Level Clear Register (TGITLVCR)

The register TGITLVCR clears the level of interrupts for transfer completed interrupt and for transfer suspended interrupt to level 0.

The register map shown in [Figure 27-59](#) and [Table 27-38](#) represents a super-set device with the maximum number of TGs (16) assumed. The actual number of bits available varies per device.

Figure 27-59. Transfer Group Interrupt Level Clear Register (TGITLVCR) [offset = 80h]



LEGEND: R/W = Read/Write; -n = value after reset

Table 27-38. Transfer Group Interrupt Level Clear Register (TGITLVCR) Field Descriptions

Bit	Field	Value	Description
31-16	CLRINTLVLRDY[n]		Transfer-group completed interrupt level clear. Bit 16 corresponds to TG0, bit 17 corresponds to TG1, and so on.
		0	Read: The TGx-completed interrupt is set to INT0. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-completed interrupt is set to INT1. Write: Clear the TGx-completed interrupt to INT0.
15-0	CLRINTLVLSUS[n]		Transfer group suspended interrupt level clear. Bit 0 corresponds to TG0, bit 1 corresponds to TG1, and so on.
		0	Read: The TGx-suspended interrupt is set to INT0. Write: A write of 0 to this bit has no effect.
		1	Read: The TGx-suspended interrupt is set to INT1. Write: Clear the TGx-suspended interrupt to INT0.

27.9.31 Transfer Group Interrupt Flag Register (TGINTFLG)

The TGINTFLG register comprises the transfer group interrupt flags for transfer-completed interrupts (INTFLGRDYx) and for transfer-suspended interrupts (INTFLGSUSx). Each of the interrupt flags in the higher half-word and the lower half-word of TGINTFLG belongs to one TG.

The register map shown in [Figure 27-60](#) and [Table 27-39](#) represents a super-set device with the maximum number of TGs (16) assumed. The actual number of bits available varies per device.

Figure 27-60. Transfer Group Interrupt Flag Register (TGINTFLG) [offset = 84h]

31	16
INTFLGRDY[15:0]	
R/W1C-0	
15	0
INTFLGSUS[15:0]	
R/W1C-0	

LEGEND: R/W = Read/Write; W1C = Write 1 to clear; -n = value after reset

Table 27-39. Transfer Group Interrupt Flag Register (TGINTFLG) Field Descriptions

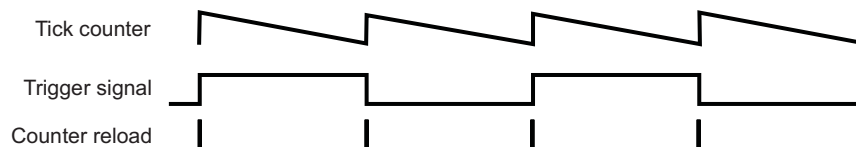
Bit	Field	Value	Description
31-16	INTFLGRDY[n]		Transfer-group interrupt flag for a transfer-completed interrupt. Bit 16 corresponds to TG0, bit 17 corresponds to TG1, and so on.
		0	Note: Read Clear Behavior. Reading the interrupt vector registers TGINTVECT0 or TGINTVECT1 automatically clears the interrupt flag bit INTFLGRDYx referenced by the vector number given by INTVECT0/INTVECT1 bits, if the SUSPEND[0:1] bit in the vector registers is 0. Read: No transfer-completed interrupt occurred since last clearing of the INTFLGRDYx flag. Write: A write of 0 to this bit has no effect.
		1	Read: A transfer finished interrupt from transfer group x occurred. No matter whether the interrupt is enabled or disabled (INTENRDYx = don't care) or whether the interrupt is mapped to INT0 or INT1, INTFLGRDYx is set right after the transfer from TGx is finished. Write: The corresponding bit flag is cleared.
15-0	INTFLGSUS[n]		Transfer-group interrupt flag for a transfer-suspend interrupt. Bit 0 corresponds to TG0, bit 1 corresponds to TG1, and so on.
		0	Note: Read Clear Behavior. Reading the interrupt vector registers TGINTVECT0 or TGINTVECT1 automatically clears the interrupt flag bit INTFLGSUSx referenced by the vector number given by INTVECT0/INTVECT1 bits, if the SUSPEND[0:1] bit in the corresponding vector registers is 1. Read: No transfer-suspended interrupt occurred since the last clearing of the INTFLGSUSx flag. Write: A write of 0 to this bit has no effect.
		1	Read: A transfer-suspended interrupt from TGx occurred. No matter whether the interrupt is enabled or disabled (INTENSUSx = don't care) or whether the interrupt is mapped to INT0 or INT1, INTFLGSUSx is set right after the transfer from transfer group x is suspended. Write: The corresponding bit flag is cleared.

27.9.32 Tick Count Register (TICKCNT)

One of the trigger sources for TGs is an internal periodic time trigger. This time trigger is called a tick counter and is basically a down-counter with a preload/reload value. Every time the tick counter detects an underflow it reloads the initial value and toggles the trigger signal provided to the TGs.

The trigger signal, shown in Figure 27-61 as a square wave, illustrates the different trigger event types for the TGs (for example, rising edge, falling edge, and both edges).

Figure 27-61. Tick Counter Operation



This register is shown in Figure 27-62 and described in Table 27-40.

Figure 27-62. Tick Count Register (TICKCNT) [offset = 90h]

31	30	29	28	27	16
TICKENA	RELOAD	CLKCTRL	Reserved		
R/W-0	R/S-0	R/W-0	R-0		
15					0
TICKVALUE					
R/W-0					

LEGEND: R/W = Read/Write; R = Read only; S = Set; -n = value after reset

Table 27-40. Tick Count Register (TICKCNT) Field Descriptions

Bit	Field	Value	Description
31	TICKENA	0 1	Tick counter enable. The internal tick counter is disabled. The counter value remains unchanged. Note: When the tick counter is disabled, the trigger signal is forced low. The internal tick counter is enabled and is clocked by the clock source selected by CLKCTRL. When TICKENA goes from 0 to 1, the tick counter is automatically loaded with the contents of TICKVALUE.
30	RELOAD		Pre-load the tick counter. RELOAD is a set-only bit; writing a 1 to it reloads the tick counter with the value stored in TICKVALUE. Reading RELOAD always returns a 0. Note: When the tick counter is reloaded by the RELOAD bit, the trigger signal is not toggled.
29-28	CLKCTRL	0 1h 2h 3h	Tick counter clock source control. CLKCTRL defines the clock source that is used to clock the internal tick counter. SPICLK of data word format 0 is selected as the clock source of the tick counter. SPICLK of data word format 1 is selected as the clock source of the tick counter. SPICLK of data word format 2 is selected as the clock source of the tick counter. SPICLK of data word format 3 is selected as the clock source of the tick counter.
27-16	Reserved	0	Reads return 0. Writes have no effect.
15-0	TICKVALUE	0-FFFFh	Initial value for the tick counter. TICKVALUE stores the initial value for the tick counter. The tick counter is loaded with the contents of TICKVALUE every time an underflow condition occurs and every time the RELOAD flag is set by the host.

27.9.33 Last TG End Pointer (LTGPEND)

Figure 27-63. Last TG End Pointer (LTGPEND) [offset = 94h]

31	29	28	24	23	16
Reserved		TG_IN_SERVICE		Reserved	
R-0		R-0		R-0	
15	14	8	7		0
Rsvd		LPEND		Reserved	
R-0		R/W-0		R-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-41. Last TG End Pointer (LTGPEND) Field Descriptions

Bit	Field	Value	Description
31-29	Reserved	0	Reads return 0. Writes have no effect.
28-24	TG_IN_SERVICE	0 1h : 10h 11h-1Fh	The TG number currently being serviced by the sequencer. These bits indicate the current TG that is being serviced. This field can generally be used for code debugging. No TG is being serviced by the sequencer. TG0 is being serviced by the sequencer. : TG15 is being serviced by the sequencer. Note: The number of transfer groups varies by device. Invalid values.
23-15	Reserved	0	Reads return 0. Writes have no effect.
14-8	LPEND	0-7Fh	Last TG end pointer. Usually the TG end address (PEND) is inherently defined by the start value of the starting pointer of the subsequent TG (PSTART). The TG ends one word before the next TG starts (PEND[x] = PSTART[x+1] - 1). For a full configuration of MibSPI, the last TG has no subsequent TG, that is, no end address is defined. Therefore, LPEND has to be programmed to specify explicitly the end address of the last TG. Note: When using all 8 transfer groups, program the LPEND bits to define the end of the last transfer group. When using less than 8 transfer groups, leave the LPEND bits programmed to point to the end of the buffer and create a dummy transfer group that defines the end of your last intentional transfer group and occupies all the remaining buffer space.
7-0	Reserved	0	Reads return 0. Writes have no effect.

27.9.34 TGx Control Registers (TGxCTRL)

Each TG can be configured via one dedicated control register. The register description shows one control register (x) that is identical for all TGs. For example, the control register for TG2 is named TG2CTRL and is located at *base address + 98h + 4 × 2*. The actual number of available control registers varies by device.

Figure 27-64. MibSPI TG Control Registers (TGxCTRL) [offsets = 98h-D4h]

31		30		29		28		27		24	
TGENA		ONESHOT		PRST		TGTD		Reserved			
R/W-0		R/W-0		R/W-0		R-0		R-0			
23						20		19		16	
TRIG EVT						TRIG SRC					
R/W-0						R/W-0					
15		14				8		7		6	
Rsvd		PSTART				Rsvd		PCURRENT			
R-0		R/W-0				R-0		R-0			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-42. TG Control Registers (TGxCTRL) Field Descriptions

Bit	Field	Value	Description
31	TGENA	0 TGx is disabled. 1 TGx is enabled.	TGx enable. If the correct event (TRIGEV_{Tx}) occurs at the selected source (TRIGSRC_{Tx}), a group transfer is initiated if no higher-priority TG is in active-transfer mode or if one or more higher-priority TGs are in transfer-suspend mode. Disabling a TG while a transfer is ongoing will finish the ongoing word transfer but not the whole group transfer.
30	ONESHOT _{Tx}	0 TGx initiates a transfer every time a trigger event occurs and TGENA is set. 1 A transfer from TGx will be performed only once (one shot) after a valid trigger event at the selected trigger source. After the transfer is finished the TGENA_{Tx} control bit will be cleared and therefore no additional transfer can be triggered before the host enables the TG again. This one shot mode ensures that after one group transfer the host has enough time to read the received data and to provide new transmit data.	Single transfer for TGx.
29	PRST _{Tx}	0 If a trigger event occurs during a transfer from TGx, the event is ignored and is not stored internally. The TGx transfer has priority over additional trigger events. 1 The TGx pointer (PCURRENT_{Tx}) will be reset to the start address (PSTART_{Tx}) when a valid trigger event occurs at the selected trigger source while a transfer from the same TG is ongoing. Every trigger event resets PCURRENT_{Tx} no matter whether the concerned TG is in transfer mode or not. The trigger events have priority over the ongoing transfer.	TGx pointer reset mode. Configures the way to resolve trigger events during an ongoing transfer. This bit is meaningful only for level-triggered TGs. Edge-triggered TGs cannot be restarted before their completion by another edge. The PRST bit will have no effect on this behavior. Note: When the PRST bit is set, if the buffer being transferred at the time of a new trigger event is a LOCK, CSHOLD or NOBRK buffer, then only after finishing those transfers, the TG will be restarted. This means that even if the TG is retriggered, the TG will only be restarted after finishing the transfer of the first non-LOCK or non-CSHOLD buffer. In the case of the NOBRK buffer, after completing the ICOUNT number of transfers, the TG will be restarted from its PSTART. This means that TX control fields such as LOCK and CSHOLD, and DMA control fields such as NOBRK have higher priority over anything else. They have the capability to delay the restart of the TG even if it is retriggered when PRST is 1.
28	TGTD _{Tx}	0 TGx has not been triggered or is no longer waiting for service. 1 TGx has been triggered and is either currently being serviced or waiting for servicing.	TG triggered.

Table 27-42. TG Control Registers (TGxCTRL) Field Descriptions (continued)

Bit	Field	Value	Description
27-24	Reserved	0	Reads return 0. Writes have no effect.
23-20	TRIGEVTx		Type of trigger event. A level-triggered TG can be stopped by de-activating the level trigger. However, the following restrictions apply. - Deactivating the level trigger for a TG during a NOBRK transfer does not stop the transfers until all of the ICOUNT number of buffers are transferred for the NOBRK buffer. Once a NOBRK buffer is prefetched, the trigger event loses control over the TG until the NOBRK buffer transfer is completed. - Once the transfer of a buffer with CSHOLD or LOCK bit set starts, deactivating the trigger level does not stop the transfer until the sequencer completes the transfer of the next non-CSHOLD or non-LOCK buffer in the same TG. - Once the last buffer in a TG is pre-fetched, deactivating the trigger level does not stop the transfer group until the last buffer transfer is completed. This means even if the trigger level is deactivated at the beginning of the penultimate (one-before-last) buffer transfer, the sequencer continues with the same TG until it is completed.
		0	never Never trigger TGx. This is the default value after reset.
		1h	rising edge A rising edge (0 to 1) at the selected trigger source (TRIGSRCx) initiates a transfer for TGx
		2h	falling edge A falling edge (1 to 0) at the selected trigger source (TRIGSRCx) initiates a transfer for TGx
		3h	both edges Rising and falling edges at the selected trigger source (TRIGSRCx) initiates a transfer for TGx
		4h	Rsvd Reserved
		5h	high-active While the selected trigger source (TRIGSRCx) is at a logic-high level (1), the group transfer is continued and at the end of one group, transfer is restarted at the beginning. If the logic level changes to low (0) during an ongoing group transfer, the whole group transfer will be stopped. Note: If ONESHOTx is set, the transfer is performed only once.
		6h	low-active While the selected trigger source (TRIGSRCx) is at a logic-low level (0), the group transfer is continued and at the end of one group, transfer is restarted at the beginning. If the logic level changes to high (1) during an ongoing group transfer, the whole group transfer will be stopped. Note: If ONESHOTx is set, the transfer is performed only once.
		7h	always A repetitive group transfer will be performed. Note: By setting the TRIGSRC to 0, the TRIGEVT to 7h (ALWAYS), and the ONESHOTx bit to 1, software can trigger this TG. Upon setting the TGENA bit, the TG is immediately triggered. Note: If ONESHOTx is set, the transfer is performed only once.
		8h-Fh	Rsvd Reserved

Table 27-42. TG Control Registers (TGxCTRL) Field Descriptions (continued)

Bit	Field	Value	Description
19-16	TRIGSRCx	0	Disabled
		1h	EXT0 External trigger source 0. The actual source varies per device (for example, HET I/O channel, event pin).
		2h	EXT1 External trigger source 1. The actual source varies per device (for example, HET I/O channel, event pin).
		3h	EXT2 External trigger source 2. The actual source varies per device (for example, HET I/O channel, event pin).
		4h	EXT3 External trigger source 3. The actual source varies per device (for example, HET I/O channel, event pin).
		5h	EXT4 External trigger source 4. The actual source varies per device (for example, HET I/O channel, event pin).
		6h	EXT5 External trigger source 5. The actual source varies per device (for example, HET I/O channel, event pin).
		7h	EXT6 External trigger source 6. The actual source varies per device (for example, HET I/O channel, event pin).
		8h	EXT7 External trigger source 7. The actual source varies per device (for example, HET I/O channel, event pin).
		9h	EXT8 External trigger source 8. The actual source varies per device (for example, HET I/O channel, event pin).
		Ah	EXT9 External trigger source 9. The actual source varies per device (for example, HET I/O channel, event pin).
		Bh	EXT10 External trigger source 10. The actual source varies per device (for example, HET I/O channel, event pin).
		Ch	EXT11 External trigger source 11. The actual source varies per device (for example, HET I/O channel, event pin).
		Dh	EXT12 External trigger source 12. The actual source varies per device (for example, HET I/O channel, event pin).
		Eh	EXT13 External trigger source 13. The actual source varies per device (for example, HET I/O channel, event pin).
		Fh	TICK Internal periodic event trigger. The tick counter can initiate periodic group transfers.
15	Reserved	0	Reads return 0. Writes have no effect.
14-8	PSTARTx	0-7Fh	TG start address. PSTARTx stores the start address of the corresponding TG. The corresponding end address is inherently defined by the subsequent TG start address minus 1 (PENDx[TGx] = PSTARTx[TGx+1]-1). PSTARTx is copied into PCURRENTx when: - The TG is enabled - The end of the TG is reached during a transfer - A trigger event occurs while PRST is set to 1
7	Reserved	0	Reads return 0. Writes have no effect.
6-0	PCURRENTx	0-7Fh	Pointer to current buffer. PCURRENT is read-only. PCURRENTx stores the address (0...127) of the buffer that corresponds to this TG. If the TG switches from active-transfer mode to suspend-to-wait mode, PCURRENTx contains the address of the currently suspended word. After the TG resumes from suspend-to-wait mode, the next buffer will be transferred; that is, no buffer data is transferred because of suspend-to-wait mode.

NOTE: Register bits vary by device

TG0 has the highest priority and TG15 has the lowest priority. Under the following conditions, a lower-priority TG cannot be interrupted by a higher-priority TG:

1. When there is a CSHOLD or LOCK buffer, until the completion of the next buffer transfer that is a non-CSHOLD or non-LOCK buffer.
2. An entire sequence of words transferred for a NOBRK DMA buffer.
3. Once the last word in a TG is pre-fetched.

27.9.35 DMA Channel Control Register (DMAxCTRL)

Each DMA channel can be configured via one dedicated control register. The register description below shows one exemplary control register that is identical for all DMA channels; for example, the control register for DMA channel 0 is named DMA0CTRL. The MibSPI supports up to 8 bidirectional DMA channels.

The number of bidirectional DMA channels varies by device. The number of DMA channels and hence the number of DMA channel control registers may vary.

Figure 27-65. DMA Channel Control Register (DMAxCTRL) [offset = D8h-F4h]

31	30	24	23	20	19	16
ONESHOT	BUFID			RXDMA_MAP		TXDMA_MAP
R/W-0		R/W-0		R/W-0		R/W-0
15	14	13	12	8		
RXDMAENA	TXDMAENA	NOBRK	ICOUNT			
R/W-0		R/W-0	R/W-0			
7	6	5	0			
Reserved	COUNT_BIT17	COUNT				
R-0	R-0	R-0				

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-43. DMA Channel Control Register (DMAxCTRL) Field Descriptions

Bit	Field	Value	Description
31	ONESHOT	0 1	Auto-disable of DMA channel after ICOUNT + 1 transfers. Note: This ONESHOT applies to the DMA channel identified by x and will autotisable based on ICOUNTx. The length of the block transfer is fully controlled by the DMA controller. The enable bits RXDMAENAx and TXDMAENAx are not modified by the MibSPI. ONESHOT allows a block transfer of defined length (ICOUNTx + 1), mainly controlled by the MibSPI and not by the DMA controller. After ICOUNTx + 1 transfers, the enable bits RXDMAENAx and TXDMAENAx are automatically cleared by the MibSPI, hence no more DMA requests are generated. In conjunction with NOBRKx, a burst transfer can be initiated without any other transfer through another buffer.
30-24	BUFIDx	0-7Fh	Buffer utilized for DMA transfer. BUFIDx defines the buffer that is utilized for the DMA transfer. In order to synchronize the transfer with the DMA controller with the NOBRK condition the "suspend to wait until..." modes must be used.
23-20	RXDMA_MAPx	0-Fh	Each MibSPI DMA channel can be linked to two physical DMA Request lines of the DMA controller. One request line for receive data and the other for request line for transmit data. RXDMA_MAPx defines the number of the physical DMA Request line that is connected to the receive path of the MibSPI DMA channel. If RXDMAENAx and TXDMAENAx are both set to 1, then RXDMA_MAPx shall differ from TXDMA_MAPx and shall differ from any other used physical DMA Request line. Otherwise, unexpected interference may occur.
19-16	TXDMA_MAPx	0-Fh	Each MibSPI DMA channel can be linked to two physical DMA Request lines of the DMA controller. One request line for receive data and the other for request line for transmit data. TXDMA_MAPx defines the number of the physical DMA Request line that is connected to the transmit path of the MibSPI DMA channel. If RXDMAENAx and TXDMAENAx are both set to 1, then TXDMA_MAPx shall differ from RXDMA_MAPx and shall differ from any other used physical DMA Request line. Otherwise, unexpected interference may occur.
15	RXDMAENAx	0 1	Receive data DMA channel enable. 0 No DMA request upon new receive data. 1 The physical DMA channel for the receive path is enabled. The first DMA request pulse is generated after the first transfer from the referenced buffer (BUFIDx) is finished. The buffer should be configured in as "skip until RXEMPTY is set" or "suspend to wait until RXEMPTY is set" in order to ensure synchronization between the DMA controller and the MibSPI sequencer.

Table 27-43. DMA Channel Control Register (DMAxCTRL) Field Descriptions (continued)

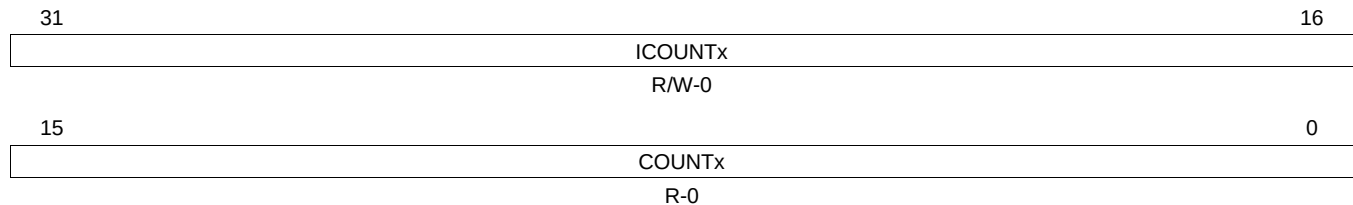
Bit	Field	Value	Description
14	TXDMAENAx	0 1	Transmit data DMA channel enable. 0 No DMA request upon new transmit data. 1 The physical DMA channel for the transmit path is enabled. The first DMA request pulse is generated right after setting TXDMAENAx to load the first transmit data. The buffer should be configured in the as "skip until TXFULL is set" or "suspend to wait until TXFULL is set" in order to ensure synchronization between the DMA controller and the MibSPI sequencer.
13	NOBRKx	0 1	Non-interleaved DMA block transfer. This bit is available in master mode only. Note: Special Conditions during a NOBRK Buffer Transfer. If a NOBRK DMA buffer is currently being serviced by the sequencer, then it is not allowed to be disabled prematurely. During a NOBRK transfer, the following operations are not allowed: • Clearing the NOBRKx bit to 0 • Clearing the RXDMAENAx to 0 (if it is already 1) • Clearing the TXDMAENAx to 0 (if it is already 1) • Clearing the BUFMODE[2:0] bits in TXRAM to 000 Note: Any attempts to perform these actions during a NOBRK transfer will produce unpredictable results. 0 DMA transfers through the buffer referenced by BUFIDx are interleaved by data transfers from other active buffers or TGs. Every time the sequencer checks the DMA buffer, it performs one transfer and then steps to the next buffer. 1 NOBRKx ensures that ICOUNTx + 1 data transfers are performed from the buffer referenced by BUFIDx without a data transfer from any other buffer. The sequencer remains at the DMA buffer until ICOUNTx + 1 transfers have been processed. For example, this can be used to generate a burst transfer to one device without disabling the chip select signal in-between (the concerned buffer has to be configured with CSHOLD = 1). Another example would be to have a defined block data transfer in slave mode, synchronous to the master SPI. Note: Triggering of higher priority TGs or enabling of higher priority DMA channels will not interrupt a NOBRK block transfer.
12-8	ICOUNTx	0-1Fh	Initial count of DMA transfers. ICOUNTx is used to preset the transfer counter COUNTx. Every time COUNTx hits 0, it is reloaded with ICOUNTx. The real number of transfers equals ICOUNTx plus 1. If ONESHOTx is set, ICOUNTx defines the number of DMA transfers that are performed before the MibSPI automatically disables the DMA channels. If NOBRKx is set, ICOUNTx defines the number of DMA transfers that are performed in one sequence without a transfer from any other buffer. If ONESHOTx and NOBRKx are not set, ICOUNTx should be 0. Note: See Section 27.9.36 (ICOUNT) and Section 27.9.37 (DMACNTLEN) about how to increase the ICOUNT to a 16-bit value. With this extended capability, MibSPI can transfer a block of up to 65535 (65K) words without interleaving (if NOBRK is used) or without deasserting the chip select between the buffers (if CSHOLD is used).
7	Reserved	0	Reads return 0. Writes have no effect.
6	COUNT_BIT17x		The 17th bit of the COUNT field of DMAxCOUNT register.
5-0	COUNTx	0-3Fh	Actual number of remaining DMA transfers. This field contains the actual number of DMA transfers that remain, until the DMA channel is disabled, if ONESHOTx is set. Note: If the TX and RX DMA requests are enabled, the COUNT register will be decremented when the RX has been serviced.

27.9.36 DMaxCOUNT Register (ICOUNT)

NOTE: These registers are used only if the LARGE COUNT bit in the DMACNTLEN register is set.

The number of bidirectional DMA channels varies by device. The number of DMA channels and hence the number of DMA registers varies by device.

Figure 27-66. DMaxCOUNT Register (ICOUNT) [offset = F8h-114h]



LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27-44. MibSPI DMaxCOUNT Register (ICOUNT) Field Descriptions

Bit	Field	Value	Description
31-16	ICOUNTx	0-FFFFh	Initial number of DMA transfers. ICOUNTx is used to preset the transfer counter COUNTx. Every time COUNTx hits 0, it is reloaded with ICOUNTx. The real number of transfer equals ICOUNTx plus 1. If ONESHOTx is set, ICOUNTx defines the number of DMA transfers that are performed before the MibSPI automatically disables the corresponding DMA channel. If NOBRKx is set, ICOUNTx defines the number of DMA transfers that are performed in one sequence without a transfer from any other buffer.
15-0	COUNTx	0-FFFFh	Actual number of remaining DMA transfers. COUNTx Contains the actual number of DMA transfers that remain, until the DMA channel is disabled, if ONESHOTx is set. Since the real counter value is always ICOUNTx +1, the 17th bit of COUNTx is available on DMACTRLx[6] bit. Note: Usage Tip for Block Transfer Using a Single DMA Request. It is possible to use the multi-buffer RAM to transfer chunks of data to/from an external SPI. A DMA Controller can be used to handle the data in bursts. Suppose a chunk of 64 bytes of data needs to be transferred and a single DMA request needs to be generated at the end of transferring the 64 bytes. This can be easily achieved by configuring a TG register for the 64 buffer locations and using the DMAxCTRL/DMAxCOUNT registers to configure the last buffer (64th) of the TG as the BUFID and enable RXDMA (NOBRK = 0). At the end of the transfer of the 64th buffer, a DMA request will be generated on the selected DMA request channel. The DMA controller can do a burst read of all 64 bytes from RXRAM and/or then do a burst write to all 64 bytes to the TXRAM for the next chunk.

27.9.37 DMA Large Count (DMACNTLEN)

Figure 27-67. DMA Large Count Register (DMACNTLEN) [offset = 118h]

31																16	
Reserved																	
R-0																	
15											1	0					
Reserved											LARGE_COUNT						
R-0											R/WP-0						

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-45. MibSPI DMA Large Count Register (DMACNTLEN) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	LARGE_COUNT	0	Select either the 16-bit DMAxCOUNT counters or the smaller counters in DMAxCTRL. Select the DMAxCTRL counters. Writes to the DMAxCTRL register will modify the ICOUNT value. Reading ICOUNT and COUNT can be done from the DMAxCTRL register. The DMAxCOUNT register should not be used since any write to this register will be overwritten by a subsequent write to the DMAxCTRL register to set the TXDMAENA or RXDMAENA bits.
		1	Select the DMAxCOUNT counters. Writes to the DMAxCTRL register will not modify the ICOUNT value. The ICOUNT value must be written to in the DMAxCOUNT register before the RXDMAENA or TXDMAENA bits are set in the DMAxCTRL register. The DMAxCOUNT register should be used for reading COUNT or ICOUNT.

27.9.38 Multi-buffer RAM Uncorrectable Parity Error Control Register (UERRCTRL)

Figure 27-68. Multi-buffer RAM Uncorrectable Parity Error Control Register (UERRCTRL) [offset = 120h]

31																16
Reserved																
R-0																
15								9	8	7				4	3	0
Reserved								PTESTEN		Reserved				EDEN		
R-0								R/WP-0		R-0				R/WP-5h		

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-46. Multi-buffer RAM Uncorrectable Parity Error Control Register (UERRCTRL) Field Descriptions

Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8	PTESTEN	0	Parity memory test enable. This bit maps the parity bits corresponding to multi-buffer RAM locations into the peripheral RAM frame to make them accessible by the CPU. See Section 27.11 for further details about parity memory testing. Parity bits are not memory-mapped.
		1	Parity bits are memory-mapped.
7-4	Reserved	0	Reads return 0. Writes have no effect.
3-0	EDEN	5h	Error detection enable. These bits enable parity error detection. Parity error detection logic (default) is disabled.
		All Other Values	Parity error detection logic is enabled. Note: It is recommended to write a 1010 to enable error detection, to guard against a soft error from disabling parity error detect

27.9.39 Multi-buffer RAM Uncorrectable Parity Error Status Register (UERRSTAT)

Figure 27-69. Multi-buffer RAM Uncorrectable Parity Error Status Register (UERRSTAT)
[offset = 124h]

31																	16
Reserved																	
R-0																	
15															2	1	0
Reserved														EDFLG1		EDFLG0	
R-0														R/W1C-0		R/W1C-0	

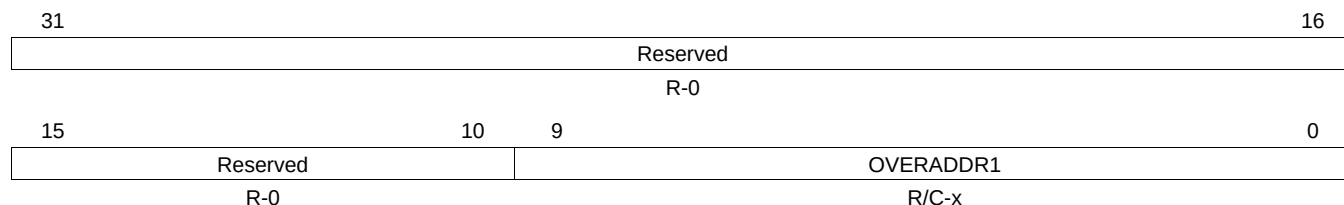
LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; -n = value after reset

Table 27-47. Multi-buffer RAM Uncorrectable Parity Error Status Register (UERRSTAT)
Field Descriptions

Bit	Field	Value	Description
31-2	Reserved	0	Reads return 0. Writes have no effect.
1	EDFLG1	0	Uncorrectable parity error detection flag. This flag indicates if a parity error occurred in the RXRAM. Note: Reading the UERRADDR1 register clears the EDFLG1 bit. Read: No error has occurred. Write: Writing a 0 to this bit has no effect.
		1	Read: An error was detected and the address is captured in the UERRADDR1 register. Write: The bit is cleared to 0.
0	EDFLG0	0	Uncorrectable parity error detection flag. This flag indicates if a parity error occurred in the TXRAM. Note: Reading the UERRADDR0 register clears the EDFLG0 bit. Read: No error has occurred. Write: Writing a 0 to this bit has no effect.
		1	Read: An error was detected and the address is captured in the UERRADDR0 register. Write: The bit is cleared to 0.

27.9.40 RXRAM Uncorrectable Parity Error Address Register (UERRADDR1)

Figure 27-70. RXRAM Uncorrectable Parity Error Address Register (UERRADDR1) [offset = 128h]



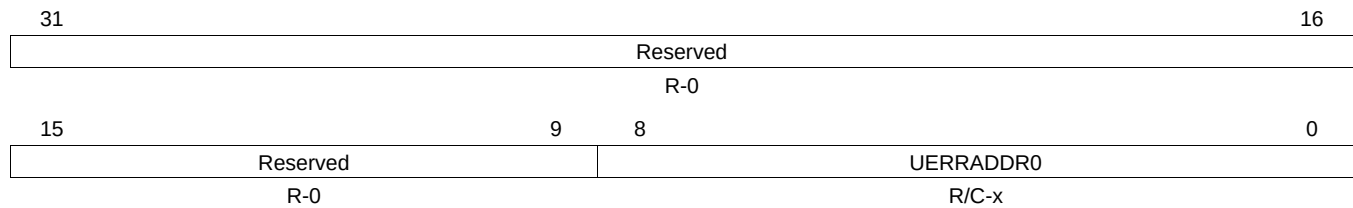
LEGEND: R = Read only; C = Clear; -n = value after reset

Table 27-48. RXRAM Uncorrectable Parity Error Address Register (UERRADDR1) Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	OVERADDR1	200h-3FFh	Uncorrectable parity error address for RXRAM. This register holds the address where a parity error is generated while reading RXRAM. Only the CPU or DMA can read from RXRAM locations. The address captured is byte-aligned. This error address is frozen from being updated until it is read by the CPU. The offset address of RXRAM varies from 200h-3FFh. The register does not clear its contents during or after module-level reset, system-level reset or even power-on reset. A read operation to this register clears its contents to the default value 200h. After a power-on reset the contents will be unpredictable. A read operation can be performed after power-up to keep the register at its default value, if required. However, the contents of this register are meaningful only when EDFLG1 is set to 1. Note: A read of the UERRADDR1 register will clear EDFLG1 in the UERRSTAT register. However, in emulation mode when the SUSPEND signal is high, a read from the UERRADDR1 register does not clear EDFLG1.

27.9.41 TXRAM Uncorrectable Parity Error Address Register (UERRADDR0)

Figure 27-71. TXRAM Uncorrectable Parity Error Address Register (UERRADDR0) [offset = 12Ch]



LEGEND: R = Read only; C = Clear; -n = value after reset

Table 27-49. TXRAM Uncorrectable Parity Error Address Register (UERRADDR0) Field Descriptions

Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8-0	UERRADDR0	0-1FFh	Uncorrectable parity error address for TXRAM. This register holds the address where a parity error is generated while reading from TXRAM. The TXRAM can be read either by CPU or by the MibSPI sequencer logic for transmission. The address captured is byte-aligned. This error address is frozen from being updated until it is read by the CPU. The offset address of TXRAM varies from 0-1FFh. The register does not clear its contents during or after module-level reset, system-level reset, or even power-on reset. A read operation to this register clears its contents to all 0s. After a power-on reset, the contents of this register will be unpredictable. A read operation can be performed after power-up to clear the this register's contents, if required. However, the contents of this register are meaningful only when EDFLG0 is set to 1. Note: A read from the UERRADDR0 register will clear EDFLG0 in the UERRSTAT register. However, in emulation mode when the SUSPEND signal is high, a read from the UERRADDR0 register does not clear EDFLG0.

27.9.42 RXRAM Overrun Buffer Address Register (RXOVRN BUF ADDR)

In multi-buffer mode, if a particular RXRAM location is written by the MibSPI sequencer logic after the completion of a new transfer when that location already contains valid data, the RX_OVR bit will be set to 1 while the data is being written. The RXOVRN_BUF_ADDR register captures the address of the RXRAM location for which a receiver overrun condition occurred.

Figure 27-72. RXRAM Overrun Buffer Address Register (RXOVRN BUF ADDR) [offset = 130h]

31			16
Reserved			
R-0			
15	10	9	0
Reserved		RXOVRN_BUF_ADDR	
R-0		R-200h	

LEGEND: R = Read only; -n = value after reset

Table 27-50. RXRAM Overrun Buffer Address Register (RXOVRN_BUF_ADDR) Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	RXOVRN_BUF_ADDR	200h-3FCh	Address in RXRAM at which an overwrite occurred. This address value will show only the offset address of the RAM location in the multi-buffer RAM address space. Refer to the device-specific data sheet for the actual absolute address of RXRAM. This word-aligned address can vary from 200h-3FCh. Contents of this register are valid only when any of the INTVECT0 or INTVECT1 and SPIFLG registers show an RXOVRN error vector while in multi-buffer mode. If there are multiple overrun errors, then this register holds the address of first overrun address until it is read. Note: Reading this register clears the RXOVRN interrupt flag in the SPIFLG register and the TGINTVECTx. Note: Receiver overrun errors in multi-buffer mode can be completely avoided by using the SUSPEND until RXEMPTY feature, which can be programmed into each buffer of any TG. However, using the SUSPEND until RXEMPTY feature will make the sequencer wait until the current RXRAM location is read by the VBUS master before it can start the transfer for the same buffer location again. This may affect the overall throughput of the SPI transfer. By enabling the interrupt on RXOVRN in multi-buffer mode, the user can rely on interrupts to know if a receiver overrun has occurred. The address of the overrun in RXRAM is indicated in this RXOVRN_BUF_ADDR register.

27.9.43 I/O-Loopback Test Control Register (IOLPBKTSTCR)

This register controls test mode for I/O pins. It also controls whether loop-back should be digital or analog. In addition, it contains control bits to induce error conditions into the module. These are to be used only for module testing.

All of the control/status bits in this register are valid only when the IOLPBKTSTENA field is set to Ah.

Figure 27-73. I/O-Loopback Test Control Register (IOLPBKTSTCR) [offset = 134h]

31						25		24
Reserved							SCS_FAIL_FLG	
R-0							R/W1C-0	
23		21	20	19	18	17		16
Reserved			CTRL_BITERR	CTRL_DESYNC	CTRL_PARERR	CTRL_TIMEOUT		CTRL_DLENERR
R-0			R/WP-0	R/WP-0	R/WP-0	R/WP-0		R/WP-0
15			12	11				8
Reserved				IOLPBKTSTENA				
R-0				R/WP-0				
7	6	5		3	2	1		0
Reserved		ERR_SCS_PIN			CTRL_SCS_PIN_ERR	LPBKTYPE		RXPENA
R-0		R/WP-0			R/WP-0	R/WP-0		R/WP-0

LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; WP = Write in privilege mode only; -n = value after reset

Table 27-51. I/O-Loopback Test Control Register (IOLPBKTSTCR) Field Descriptions

Bit	Field	Value	Description
31-25	Reserved	0	Reads return 0. Writes have no effect.
24	SCS FAIL FLG	0	Bit indicating a failure on <u>SPICS</u> pin compare during analog loopback. Read: No mismatches occurred on any of the eight chip select pins (vs. the internal chip select number CSNR during transfers). Write: Writing a 0 to this bit has no effect.
		1	Read: A comparison between the internal CSNR field and the analog looped-back value of one or more of the <u>SPICS</u> pins failed. A stuck-at fault is detected on one of the <u>SPICS</u> pins. Comparison is done only on the pins that are configured as functional and during transfer operation. Write: This flag bit is cleared.
23-21	Reserved	0	Reads return 0. Writes have no effect.
20	CTRL BITERR	0	Controls inducing of BITERR during I/O loopback test mode. Do not interfere with looped-back data.
		1	Induces bit errors by inverting the value of the incoming data during loopback.
19	CTRL DESYNC	0	Controls inducing of the desync error during I/O loopback test mode. Do not cause a desync error.
		1	Induce a desync error by forcing the incoming <u>SPIENA</u> pin (if functional) to remain 0 even after the transfer is complete. This forcing will be retained until the kernel reaches the idle state.
18	CTRL PARERR	0	Controls inducing of the parity errors during I/O loopback test mode. Do not cause a parity error.
		1	Induce a parity error by inverting the polarity of the parity bit.
17	CTRL TIMEOUT	0	Controls inducing of the timeout error during I/O loopback test mode. Do not cause a timeout error.
		1	Induce a timeout error by forcing the incoming <u>SPIENA</u> pin (if functional) to remain 1 when transmission is initiated. The forcing will be retained until the kernel reaches the idle state.

Table 27-51. I/O-Loopback Test Control Register (IOLPBKTSTCR) Field Descriptions (continued)

Bit	Field	Value	Description
16	CTRL DLENERR	0 1	Controls inducing of the data length error during I/O loopback test mode. Do not cause a data-length error. Induce a data-length error. <i>Master mode:</i> The $\overline{\text{SPIEN}}\text{A}$ pin (if functional) is forced to 1 when the module starts shifting data. <i>Slave mode:</i> The incoming $\overline{\text{SPICS}}$ pin (if functional) is forced to 1 when the module starts shifting data.
15-12	Reserved	0	Reads return 0. Writes have no effect.
11-8	IOLPBKSTENA	Ah All Other Values	Module I/O loopback test enable key. Enable I/O loopback test mode. Disable I/O loopback test mode.
7-6	Reserved	0	Reads return 0. Writes have no effect.
5-3	ERR SCS PIN	0 1h : 7h	Inject error on chip-select pin number x. The value in this field is decoded as the number of the chip select pin on which to inject an error. During analog loopback, if CTRL SCS PIN ERR bit is set to 1, then the chip select pin selected by this field is forced to the opposite of its value in the CSNR. Select $\overline{\text{SPICS}}[0]$ for injecting error. Select $\overline{\text{SPICS}}[1]$ for injecting error. : Select $\overline{\text{SPICS}}[7]$ for injecting error.
2	CTRL SCS PIN ERR	0 1	Enable/disable the injection of an error on the $\overline{\text{SPICS}}$ pins. The individual $\overline{\text{SPICS}}$ pins can be chosen using the ERR SCS PIN field. Disable the $\overline{\text{SPICS}}$ error-inducing logic. Enable the $\overline{\text{SPICS}}$ error-inducing logic.
1	LPBK TYPE	0 1	Module I/O loopback type (analog/digital). See Figure 27-22 for the different types of loopback modes. Enable Digital loopback when IOLPBKTSTENA = 1010. Enable Analog loopback when IOLPBKTSTENA = 1010.
0	RXPENA	0 1	Enable analog loopback through the receive pin. Note: This bit is valid only when LPBK TYPE = 1, which chooses analog loopback mode. Analog loopback is through the transmit pin. Analog loopback is through the receive pin.

27.9.44 SPI Extended Prescale Register 1 (EXTENDED_PRESCALE1 for SPIFMT0 and SPIFMT1)

This register provides an extended Prescale values for SPICLK generation to be able to interface with much slower SPI Slaves. This is an extension of SPIFMT0 and SPIFMT1 registers. For example, EPRESCALE_FMT1[7:0] of EXTENDED_PRESCALE1 and PRESCALE1 of SPIFMT1 register will always reflect the same contents. Similarly, EPRESCALE_FMT0[7:0] and PRESCALE0 of SPIFMT0 reflect the same contents.

Figure 27-74. SPI Extended Prescale Register 1 (EXTENDED_PRESCALE1 for SPIFMT0 and SPIFMT1) [offset = 138h]

31	27	26	16
Reserved		EPRESCALE_FMT1	
R-0		R/WP-0	
15	11	10	0
Reserved		EPRESCALE_FMT0	
R-0		R/WP-0	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-52. SPI Extended Prescale Register 1 (EXTENDED_PRESCALE1) Field Descriptions

Bit	Field	Value	Description
31-27	Reserved	0	Reads return 0. Writes have no effect.
26-16	EPRESCALE_FMT1	0-7FFh	EPRESCALE_FMT1. Extended Prescale value for SPIFMT1. EPRESCALE_FMT1 determines the bit transfer rate of data format 1 if the SPI/MibSPI is the network master. EPRESCALE_FMT1 is used to derive SPICLK from VCLK. If the SPI is configured as slave, EPRESCALE_FMT1 does not need to be configured. These EPRESCALE_FMT1[7:0] bits and PRESCALE1 bits of SPIFMT1 register will point to the same physically implemented register. The clock rate for data format 1 can be calculated as: $BR_{Format1} = VCLK / (EPRESCALE_FMT1 + 1)$ Write: This register field should be written if a SPICLK prescaler of more VCLK/256 is required. This field provides a prescaler of up to VCLK/2048 for SPICLK. Writing to this register field will also get reflected in the PRESCALE1 bits of SPIFMT1 register. Read: Reading this field will reflect the PRESCALE value based on the last written register field, that is, EXTENDED_PRESCALE1[26:16] or SPIFMT1[15:8] register. Note: If Extended Prescaler is required, it should be ensured that EXTENDED_PRESCALE1 register is programmed after SPIFMT1 register is programmed. This is to ensure that the final SPICLK prescale value is controlled by EXTENDED_PRESCALE1 register when a prescale of more 256 is intended on SPICLK. Writing to PRESCALE1 field of SPIFMT1 will automatically clear EPRESCALE_FMT1[10:8] bits to 000 so that the integrity of PRESCALE value is maintained.
15-11	Reserved	0	Reads return 0. Writes have no effect.

Table 27-52. SPI Extended Prescale Register 1 (EXTENDED_PRESCALE1) Field Descriptions (continued)

Bit	Field	Value	Description
10-0	EPRESCALE_FMT0	0-7FFh	EPRESCALE_FMT0. Extended Prescale value for SPIFMT0. EPRESCALE_FMT0 determines the bit transfer rate of data format 0 if the SPI/MibSPI is the network master. EPRESCALE_FMT0 is use to derive SPICLK from VCLK. If the SPI is configured as slave, EPRESCALE_FMT0 does not need to be configured. These EPRESCALE_FMT0[7:0] bits and PRESCALE0 bits of SPIFMT0 register will point to the same physically implemented register. The clock rate for data format 0 can be calculated as: $BR_{Format0} = VCLK / (EPRESCALE_FMT0 + 1)$ Write: This register field should be written if a SPICLK prescaler of more VCLK/256 is required. This field provides a prescaler of up to VCLK/2048 for SPICLK. Writing to this register field will also get reflected in the PRESCALE0 bits of SPIFMT0 register. Read: Reading this field will reflect the PRESCALE value based on the last written register field, that is, EXTENDED_PRESCALE0[10:0] or SPIFMT0[15:8] register. Note: If Extended Prescaler is required, it should be ensured that EXTENDED_PRESCALE1 register is programmed after SPIFMT0 register is programmed. This is to ensure that the final SPICLK prescale value is controlled by EXTENDED_PRESCALE1 register when a prescale of more 256 is intended on SPICLK. Writing to PRESCALE0 field of SPIFMT0 will automatically clear EPRESCALE_FMT0[10:8] bits to 000 so that the integrity of PRESCALE value is maintained.

27.9.45 SPI Extended Prescale Register 2 (EXTENDED_PRESCALE2 for SPIFMT2 and SPIFMT3)

This register provides an extended Prescale values for SPICLK generation to be able to interface with much slower SPI Slaves. This is an extension of SPIFMT2 and SPIFMT3 registers. For example, EPRESCALE_FMT3[7:0] of EXTENDED_PRESCALE2 and PRESCALE3 of SPIFMT3 register will always reflect the same contents. Similarly, EPRESCALE_FMT2[7:0] and PRESCALE2 of SPIFMT2 reflect the same contents.

Figure 27-75. SPI Extended Prescale Register 2 (EXTENDED_PRESCALE2 for SPIFMT2 and SPIFMT3) [offset = 13Ch]

31	27	26	16
Reserved		EPRESCALE_FMT3	
R-0		R/WP-0	
15	11	10	0
Reserved		EPRESCALE_FMT2	
R-0		R/WP-0	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -n = value after reset

Table 27-53. SPI Extended Prescale Register 2 (EXTENDED_PRESCALE2) Field Descriptions

Bit	Field	Value	Description
31-27	Reserved	0	Reads return 0. Writes have no effect.
26-16	EPRESCALE_FMT3	0-7FFh	EPRESCALE_FMT3. Extended Prescale value for SPIFMT3. EPRESCALE_FMT3 determines the bit transfer rate of data format 3 if the SPI/MibSPI is the network master. EPRESCALE_FMT3 is use to derive SPICLK from VCLK. If the SPI is configured as slave, EPRESCALE_FMT3 does not need to be configured. These EPRESCALE_FMT3[7:0] bits and PRESCALE3 bits of SPIFMT3 register will point to the same physically implemented register. The clock rate for data format 1 can be calculated as: $BR_{Format3} = VCLK / (EPRESCALE_FMT3 + 1)$ Write: This register field should be written if a SPICLK prescaler of more VCLK/256 is required. This field provides a prescaler of up to VCLK/2048 for SPICLK. Writing to this register field will also get reflected in the PRESCALE3 bits of SPIFMT3 register. Read: Reading this field will reflect the PRESCALE value based on the last written register field, that is, EXTENDED_PRESCALE3[26:16] or SPIFMT3[15:8] register. Note: If Extended Prescaler is required, it should be ensured that EXTENDED_PRESCALE2 register is programmed after SPIFMT3 register is programmed. This is to ensure that the final SPICLK prescale value is controlled by EXTENDED_PRESCALE2 register when a prescale of more 256 is intended on SPICLK. Writing to PRESCALE3 field of SPIFMT3 will automatically clear EPRESCALE_FMT3[10:8] bits to 000 so that the integrity of PRESCALE value is maintained.
15-11	Reserved	0	Reads return 0. Writes have no effect.

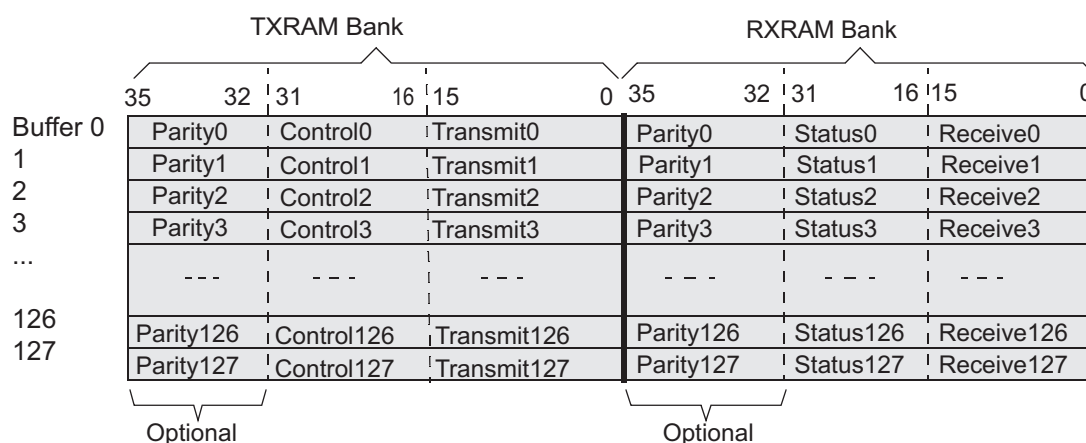
Table 27-53. SPI Extended Prescale Register 2 (EXTENDED_PRESCALE2) Field Descriptions (continued)

Bit	Field	Value	Description
10-0	EPRESCALE_FMT2	0-7FFh	EPRESCALE_FMT2. Extended Prescale value for SPIFMT2. EPRESCALE_FMT2 determines the bit transfer rate of data format 2 if the SPI/MibSPI is the network master. EPRESCALE_FMT2 is use to derive SPICLK from VCLK. If the SPI is configured as slave, EPRESCALE_FMT2 does not need to be configured. These EPRESCALE_FMT2[7:0] bits and PRESCALE2 bits of SPIFMT2 register will point to the same physically implemented register. The clock rate for data format 0 can be calculated as: $BR_{Format2} = VCLK / (EPRESCALE_FMT2 + 1)$ Write: This register field should be written if a SPICLK prescaler of more VCLK/256 is required. This field provides a prescaler of up to VCLK/2048 for SPICLK. Writing to this register field will also get reflected in the PRESCALE2 bits of SPIFMT2 register. Read: Reading this field will reflect the PRESCALE value based on the last written register field, that is, EXTENDED_PRESCALE2[10:0] or SPIFMT2[15:8] register. Note: If Extended Prescaler is required, it should be ensured that EXTENDED_PRESCALE2 register is programmed after SPIFMT2 register is programmed. This is to ensure that the final SPICLK prescale value is controlled by EXTENDED_PRESCALE2 register when a prescale of more 256 is intended on SPICLK. Writing to PRESCALE2 field of SPIFMT2 will automatically clear EPRESCALE_FMT2[10:8] bits to 000 so that the integrity of PRESCALE value is maintained.

27.10 Multi-Buffer RAM

The multi-buffer RAM is used for holding transit and received data, control and status information. The multi-buffer RAM contains two banks of up to 128 32-bit words for a maximum configuration. One bank (TXRAM) contains entries for transmit data (replicating the SPIDAT1 register). The other bank (RXRAM) contains received data (replicating the SPIBUF register). The buffers can be partitioned into multiple TGs, each containing a programmable number of buffers. Each of the buffers can be subdivided into 16-bit transmit field, 16-bit receive field, 16-bit control field, and 16-bit status field, as displayed in [Figure 27-76](#). A 4-bit parity field per word is also included in each bank of RAM.

Figure 27-76. Multi-Buffer RAM Configuration



All fields can be read and written with 8-bit, 16-bit, or 32-bit accesses.

The transmit fields can be written and read in the address range 000h to 1FFh. The transmit words contain data and control fields.

The receive RAM fields are read-only and can be accessed through the address range 200h to 3FCh. The receive words contain data and status fields.

The chip select number (CSNR) bit field of the control field for a given word is mirrored into the corresponding receive-buffer status field after transmission.

The Parity is automatically calculated and copied to Parity location

NOTE: Please refer to the specific device datasheet for the actual number of transmit and receive buffers.

Write to unimplemented buffer is overwriting the corresponding implemented buffer. In MIBSPI, if the RAM SIZE specified is 32 buffers, write to 33rd buffer overwrites 1st buffer, write to 34th buffer overwrites 2nd buffer and so on.

27.10.1 Multi-Buffer RAM Auto Initialization

When the MIBSPI is out of reset mode, auto initialization of multi-buffer RAM starts. The application code must check for BUFINITACTIVE bit to be 0 (Multi-buffer RAM initialization is complete) before configuring multi-buffer RAM.

Besides the default auto initialization after reset, the auto-initialization sequence can also be done by:

1. Enable the global hardware memory initialization key by programming a value of 1010b to the bits [3:0] of the MINITGCR register of the System module.
2. Set the control bit for the multi-buffer RAM in the MSINENA System module register. This bit is device-specific for each memory that support auto-initialization. Please refer to the device datasheet to identify the control bit for the multi-buffer RAM. This starts the initialization process. The BUFINITACTIVE bit will get set to reflect that the initialization is ongoing.
3. When the memory initialization is completed, the corresponding status bit in the MINISTAT register will be set. Also, the BUFINITACTIVE bit will get cleared.
4. Disable the global hardware memory initialization key by programming a value of 0101 to the bits [3:0] of the MINITGCR register of the System module.

Please refer to the *Architecture* chapter for more details on the memory auto-initialization process.

NOTE: During Auto Initialization process, all the Multi-buffer mode registers (except MIBSPIE) will be reset to their default values. So, it should be ensured that Auto Initialization is completed before configuring the Multi-buffer mode registers.

27.10.2 Multi-Buffer RAM Register Summary

This section describes the multi-buffer RAM control and transmit-data fields of each word of TXRAM, and the status and receive-data fields of each word of RXRAM. The base address for multi-buffer RAM is FF0E 0000h for MibSPI1 RAM, FF0C 000h for MibSPI3 RAM, and FF0A 0000h for MibSPI5 RAM.

Table 27-54. Multi-Buffer RAM Register Summary

Offset	Acronym	Register Description	Section
Base + 0h-1FFh	TXRAM	Multi-Buffer RAM Transmit Data Register	Section 27.10.3
Base + 200h-3FFh	RXRAM	Multi-Buffer RAM Receive Buffer Register	Section 27.10.4

27.10.3 Multi-Buffer RAM Transmit Data Register (TXRAM)

Each word of TXRAM is a transmit-buffer register.

NOTE: Writing to only the control fields, bits 28 through 16, does not initiate any SPI transfer in master mode. This feature can be used to set up SPICLK phase or polarity before actually starting the transfer by only updating the DFSEL bit field to select the required phase and polarity combination.

Figure 27-77. Multi-Buffer RAM Transmit Data Register (TXRAM)
[offset = RAM Base + 0h-1FFh]

31	29	28	27	26	25	24	23	16
BUFMODE		CSHOLD	LOCK	WDEL	DFSEL	CSNR		
R/W-0		R/W-0	R/W-0	R/W-0	R/W-0	R/W-0		
15								0
TXDATA								
R/W-0								

LEGEND: R/W = Read/Write; -n = value after reset

Table 27-55. Multi-Buffer RAM Transmit Data Register (TXRAM) Field Descriptions

Bit	Field	Value	Description
31-29	BUFMODE		Specify conditions that are recognized by the sequencer to initiate transfers of each buffer word. When one of the "skip" modes is selected, the sequencer checks the buffer status every time it reads from this buffer. If the current buffer status (TXFULL, RXEMPTY) does not match, the buffer is skipped without a data transfer. When one of the "suspend" modes is selected, the sequencer checks the buffer status when it reads from this buffer. If TxFULL and/or RXEMPTY do not match, the sequencer waits until a match occurs. No data transfer is initiated until the status condition of this buffer changes.
		0	disabled. The buffer is disabled.
		1h	skip single-transfer mode. Skip this buffer until the corresponding TxFULL flag is set (new transmit data is available).
		2h	skip overwrite-protect mode. Skip this buffer until the corresponding RXEMPTY flag is set (new receive data can be stored in RXDATA without data loss).
		3h	skip single-transfer overwrite-protect mode. Skip this buffer until both of the corresponding TxFULL and RXEMPTY flags are set. (new transmit data available and previous data received by the host).
		4h	continuous mode. Initiate a transfer each time the sequencer checks this buffer. Data words are retransmitted if the buffer has not been updated. Receive data is overwritten, even if it has not been read.
		5h	suspend single-transfer mode. Suspend-to-wait until the corresponding TxFULL flag is set (the sequencer stops at the current buffer until new transmit data is written in the TXDATA field).
		6h	suspend overwrite-protect mode. Suspend-to-wait until the corresponding RXEMPTY flag is set (the sequencer stops at the current buffer until the previously-received data is read by the host).
		7h	suspend single-transfer overwrite-protect mode. Suspend-to-wait until the corresponding TxFULL and RXEMPTY flags are set (the sequencer stops at the current buffer until new transmit data is written into the TXDATA field and the previously-received data is read by the host).
28	CSHOLD		Chip select hold mode. The CSHOLD bit is supported in master mode only, it is ignored in slave mode. CSHOLD defines the behavior of the chip select line at the end of a data transfer.
		0	The chip select signal is deactivated at the end of a transfer after the T2CDELAY time has passed. If two consecutive transfers are dedicated to the same chip select this chip select signal will be deactivated for at least 2VCLK cycles before it is activated again.
		1	The chip select signal is held active at the end of a transfer until a control field with new data and control information is loaded into SPIDAT1. If the new chip select number equals the previous one, the active chip select signal is extended until the end of transfer with CSHOLD cleared, or until the chip-select number changes.

Table 27-55. Multi-Buffer RAM Transmit Data Register (TXRAM) Field Descriptions (continued)

Bit	Field	Value	Description
27	LOCK	0 1	Lock two consecutive buffer words. Do not allow interruption by TGs with higher priority. Any higher-priority TG can begin at the end of the current transaction. A higher-priority TG cannot occur until after the next unlocked buffer word is transferred.
26	WDEL	0 1	Enable the delay counter at the end of the current transaction. Note: The WDEL bit is supported in master mode only. In slave mode, this bit is ignored. 0 No delay will be inserted. However, $\overline{\text{SPICS}}$ pins will still be de-activated for at least for 2VCLK cycles if CSHOLD = 0. Note: The duration for which the $\overline{\text{SPICS}}$ pin remains deactivated also depends upon the time taken to supply a new word after completing the shift operation (in compatibility mode). If TXBUF is already full, then the $\overline{\text{SPICS}}$ pin will be deasserted for at least two VCLK cycles (if WDEL = 0). 1 After a transaction, WDELAY of the corresponding data format will be loaded into the delay counter. No transaction will be performed until the WDELAY counter overflows. The $\overline{\text{SPICS}}$ pins will be de-activated for at least (WDELAY + 2) × VCLK_Period duration.
25-24	DFSEL	0 1h 2h 3h	Data word format select. Data word format 0 is selected. Data word format 1 is selected. Data word format 2 is selected. Data word format 3 is selected.
23-16	CSNR	0-FFh	Chip select (CS) number. CSNR defines the chip select pins that will be activated during the data transfer. CSNR is a bit-mask that controls all chip select pins. See Table 27-56 . Note: If your MibSPI has less than 8 chip select pins, all unused upper bits will be 0. For example, MibSPI3 has 6 chip select pins, if you write FFh to CSNR, the actual number stored in CSNR is 3Fh.
15-0	TXDATA	0-7FFFh	Transfer data. When written, these bits are copied to the shift register if it is empty. If the shift register is not empty, then they are held in TXBUF. SPIEN must be set to 1 before this register can be written to. Writing a 0 to SPIEN forces the lower 16 bits of TXDATA to 0. A write to this register (or to the TXDATA field only) drives the contents of the CSNR field on the $\overline{\text{SPICS}}$ pins, if the pins are configured as functional pins (automatic chip select, see Section 27.2). When this register is read, the contents of TXBUF, which holds the latest data written, will be returned. Note: Regardless of the character length, the transmit data should be right-justified before writing to the SPIDAT1 register.

Table 27-56. Chip Select Number Active

CSNR Value	Chip Select Active:						CSNR Value	Chip Select Active:					
	CS[5] ⁽¹⁾	CS[4] ⁽¹⁾	CS[3] ⁽¹⁾	CS[2] ⁽¹⁾	CS[1] ⁽¹⁾	CS[0]		CS[5] ⁽¹⁾	CS[4] ⁽¹⁾	CS[3] ⁽¹⁾	CS[2] ⁽¹⁾	CS[1] ⁽¹⁾	CS[0]
0h	No chip select pin is active.						20h	x					
1h						x	21h	x					x
2h					x		22h	x				x	
3h					x	x	23h	x				x	x
4h				x			24h	x			x		
5h				x		x	25h	x			x		x
6h				x	x		26h	x			x	x	
7h				x	x	x	27h	x			x	x	x
8h			x				28h	x		x			
9h			x			x	29h	x		x			x
Ah			x		x		2Ah	x		x		x	
Bh			x		x	x	2Bh	x		x		x	x
Ch			x	x			2Ch	x		x	x		
Dh			x	x		x	2Dh	x		x	x		x
Eh			x	x	x		2Eh	x		x	x	x	
Fh			x	x	x	x	2Fh	x		x	x	x	x
10h		x					30h	x	x				
11h		x				x	31h	x	x				x
12h		x			x		32h	x	x			x	
13h		x			x	x	33h	x	x			x	x
14h		x		x			34h	x	x		x		
15h		x		x		x	35h	x	x		x		x
16h		x		x	x		36h	x	x		x	x	
17h		x		x	x	x	37h	x	x		x	x	x
18h		x	x				38h	x	x	x			
19h		x	x			x	39h	x	x	x			x
1Ah		x	x		x		3Ah	x	x	x		x	
1Bh		x	x		x	x	3Bh	x	x	x		x	x
1Ch		x	x	x			3Ch	x	x	x	x		
1Dh		x	x	x		x	3Dh	x	x	x	x		x
1Eh		x	x	x	x		3Eh	x	x	x	x	x	
1Fh		x	x	x	x	x	3Fh	x	x	x	x	x	x

⁽¹⁾ If your MibSPI does not have this chip select pin, this bit is 0.

27.10.4 Multi-Buffer RAM Receive Buffer Register (RXRAM)

Each word of RXRAM is a receive-buffer register.

Figure 27-78. Multi-Buffer RAM Receive Buffer Register (RXRAM)
[offset = RAM Base + 200h-3FFh]

31	30	29	28	27	26	25	24
RXEMPTY	RXOVR	TXFULL	BITERR	DESYNC	PARITYERR	TIMEOUT	DLENERR
RS-1	RC-0	R-0	RC-0	RC-0	RC-0	RC-0	RC-0
23							16
LCSNR							
R-0							
15							0
RXDATA							
R/W-0							

LEGEND: R/W = Read/Write; R = Read only; C = Clear; S = Set; -n = value after reset

Table 27-57. Multi-Buffer Receive Buffer Register (RXRAM) Field Descriptions

Bit	Field	Value	Description
31	RXEMPTY	0 1	Receive data buffer empty. When the host reads the RXDATA field or the entire RXRAM register, it automatically sets the RXEMPTY flag. When a data transfer is completed, the received data is copied into RXDATA, and the RXEMPTY flag is cleared. New data has been received and copied into RXDATA. No data has been received since the last read of RXDATA. This flag gets set to 1 under the following conditions: - Reading the RXDATA field of the RXRAM register - Writing a 1 to clear the RXINTFLG bit in the SPI Flag Register (SPIFLG) Write-clearing the RXINTFLG bit before reading RXDATA indicates the received data is being ignored. Conversely, RXINTFLG can be cleared by reading the RXDATA field of RXRAM (or the entire register).
30	RXOVR	0 1	Receive data buffer overrun. When a data transfer is completed and the received data is copied into RXBUF while it is already full, RXOVR is set. Overruns always occur to RXBUF, not to RXRAM; the contents of RXRAM are overwritten only after it is read by the Peripheral (VBUSP) master (CPU, DMA, or other host processor). If enabled, the RXOVRN interrupt is generated when RXBUF is overwritten, and reading either SPI Flag Register (SPIFLG) or SPIVEXTx shows the RXOVRN condition. Two read operations from the RXRAM register are required to reach the overwritten buffer word (one to read RXRAM, which then transfers RXDATA into RXRAM for the second read). Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. Note: A special condition under which RXOVR flag gets set. If both RXRAM and RXBUF are already full and while another buffer receive is underway, if any errors such as TIMEOUT, BITERR, and DLEN_ERR occur, then RXOVR in RXBUF and SPI Flag Register (SPIFLG) registers will be set to indicate that the status flags are getting overwritten by the new transfer. This overrun should be treated like a normal receive overrun. No receive data overrun condition occurred since last read of the data field. A receive data overrun condition occurred since last read of the data field.
29	TXFULL	0 1	Transmit data buffer full. This flag is a read-only flag. Writing into the SPIDAT0 or SPIDAT1 field while the TX shift register is full will automatically set the TXFULL flag. Once the word is copied to the shift register, the TXFULL flag will be cleared. Writing to SPIDAT0 or SPIDAT1 when both TXBUF and the TX shift register are empty does not set the TXFULL flag. The transmit buffer is empty; SPIDAT0/SPIDAT1 is ready to accept a new data. The transmit buffer is full; SPIDAT0/SPIDAT1 is not ready to accept new data.

Table 27-57. Multi-Buffer Receive Buffer Register (RXRAM) Field Descriptions (continued)

Bit	Field	Value	Description
28	BITERR	0 1	Bit error. There was a mismatch of internal transmit data and transmitted data. Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. No bit error occurred. A bit error occurred. The SPI samples the signal of the transmit pins (master: SIMOx, slave: SOMIx) at the receive point (one-half clock cycle after the transmit point). If the sampled value differs from the transmitted value, a bit error is detected and the BITERR flag is set. Possible reasons for a bit error include noise, an excessively high bit rate, capacitive load, or another master/slave trying to transmit at the same time.
27	DESYNC	0 1	Desynchronization of slave device. This bit is valid in master mode only. The master monitors the ENA signal coming from the slave device and sets the DESYNC flag if ENA is deactivated before the last reception point or after the last bit is transmitted plus $t_{T2DELAY}$. If DESYNCENA is set, an interrupt is asserted. Desynchronization can occur if a slave device misses a clock edge coming from the master. Note: In the Compatibility Mode MibSPI, under some circumstances it is possible for a desync error detected for the previous buffer to be visible in the current buffer. This is because the receive completion flag/interrupt is generated when the buffer transfer is completed. But desynchronization is detected after the buffer transfer is completed. So, if the VBUS master reads the received data quickly when an RXINT is detected, then the status flag may not reflect the correct desync condition. In multi-buffer mode, the desync flag is always guaranteed to be for the current buffer. Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. No slave desynchronization is detected. A slave device is desynchronized.
26	PARITYERR	0 1	Parity error. The calculated parity differs from the received parity bit. If the parity generator is enabled (selected individually for each buffer) an even or odd parity bit is added at the end of a data word. During reception of the data word, the parity generator calculates the reference parity and compares it to the received parity bit. If a mismatch is detected, the PARITYERR flag is set. Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. No parity error is detected. A parity error occurred.
25	TIMEOUT	0 1	Time-out because of non-activation of $\overline{\text{SPIEN}}_A$ pin. The SPI generates a time-out when the slave does not respond in time by activating the ENA signal after the chip select signal has been activated. If a time-out condition is detected, the corresponding chip select is deactivated immediately and the TIMEOUT flag is set. In addition, the TIMEOUT flag in the status field of the corresponding buffer and in the SPI Flag Register (SPIFLG) is set. Note: This bit is valid only in master mode. Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. No $\overline{\text{SPIEN}}_A$ pin time-out occurred. An $\overline{\text{SPIEN}}_A$ signal time-out occurred.
24	DLENERR	0 1	Data length error flag. Note: This flag is cleared to 0 when the RXDATA field of the RXRAM register is read. No data-length error occurred. A data length error occurred.
23-16	LCSNR	0-FFh	Last chip select number. LCSNR in the status field is a copy of CSNR in the corresponding control field. It contains the chip select number that was activated during the last word transfer.
15-0	RXDATA	0-FFFFh	SPI receive data. This is the received word, transferred from the receive shift-register at the end of a transfer. Regardless of the programmed character length and the direction of shifting, the received data is stored right-justified in the register.

27.11 Parity Memory

The parity portion of multi-buffer RAM is not accessible by the CPU during normal operating modes. However, each read or write operation to the control/data/status portion of the multi-buffer RAM causes reads/writes to the parity portion as well.

- Each write to the multi-buffer RAM (either from the Peripheral interface or by the MibSPI itself) causes a write operation to the parity portion of RAM simultaneously to update the equivalent parity bits.
- Each read operation from the multi-buffer RAM (either from the Peripheral interface or by the MibSPI itself) causes a read operation from the parity portion of the RAM for parity comparison purpose.
- Reads/Writes to multi-buffer RAM can either be caused by any CPU/DMA accesses or by the sequencer logic of MibSPI itself.
- In case of Parity error ESM module is notified to generate MIBSPI Parity ESM interrupt. User can check the error status and address location captured in the UERRSTAT and UERRADDRx registers respectively.

For testing the parity portion of the multi-buffer RAM, which is a 4-bit field per word address (1 bit per byte), a separate parity memory test mode is available. Parity memory test mode can be enabled and disabled by the PTESTEN bit in the UERRCTRL register.

During the parity test mode, the parity locations are addressable at the address between RAM_BASE_ADDR + 0x400h and RAM_BASE_ADDR + 0x7FFh. Each location corresponds, sequentially, to each TXRAM word, then to each RXRAM word. See [Figure 27-79](#) for a diagram of the memory map of parity memory during normal operating mode and during parity test mode.

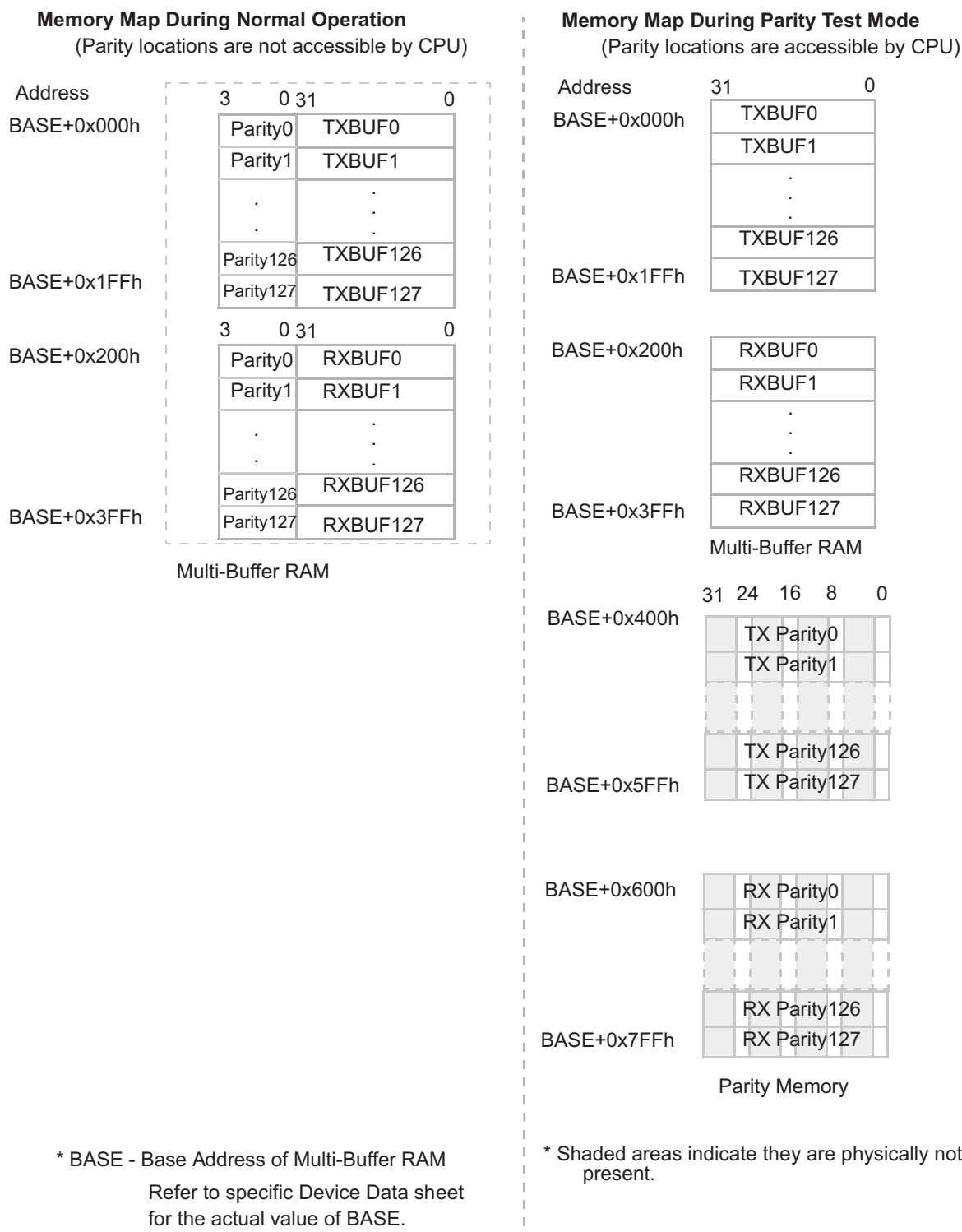
During parity test mode, after writing the data/control portion of the RAM, the parity locations can be written with incorrect parity bits to intentionally cause parity errors.

See the device-specific data sheet to get the actual base address of the multi-buffer RAM.

NOTE: The RX_RAM_ACCESS bit can also be set to 1 during the parity test mode to be enable writes to RXRAM locations. Both parity RAM testing and RXRAM testing can be done together.

There are 4 bits of parity corresponding to each of the 32-bit multi-buffer locations. Individual bits in the parity memory are byte-addressable in parity test mode. See the example in [Figure 27-80](#) for further details.

NOTE: Polarity of the parity (odd/even) varies by device. In some devices, a control register in the system module can be used to select odd or even parity.

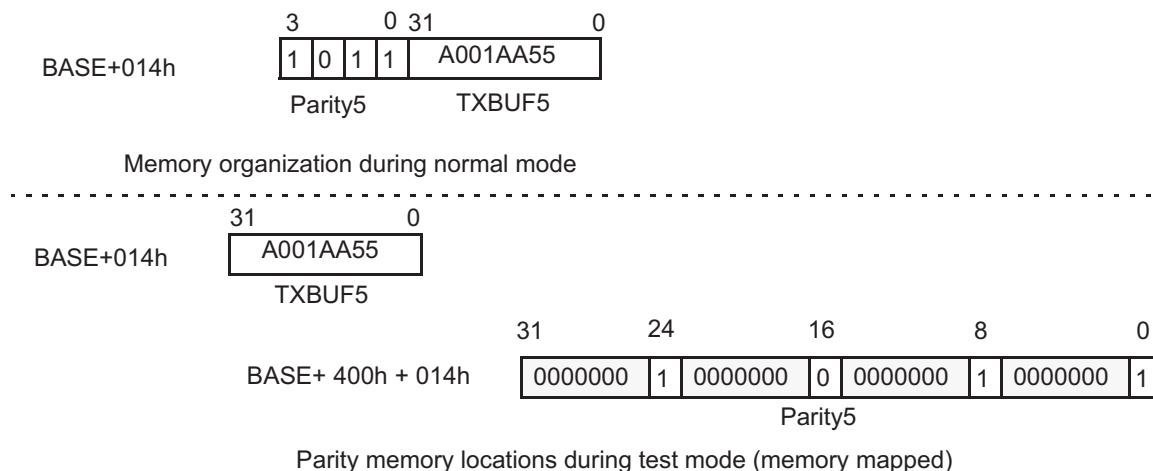
Figure 27-79. Memory Map for Parity Locations During Normal and Test Mode


27.11.1 Example of Parity Memory Organization

Suppose TXBUF5 (6th location in TXRAM) in the multi-buffer RAM is written with a value of A001_AA55. If the polarity of the parity is set to odd, the corresponding parity location parity5 will get updated with equivalent parity of 1011 in its field.

During parity-memory test mode, these bits can be individually byte addressed. The return data will be a byte adjusted with actual parity bit in the LSB of the byte. If a word is read from the word-boundary address of parity locations, then each bit of the 4-bit parity is byte-adjusted and a 32-bit word is returned. 0s will be padded into the parity bits to get each byte. See [Figure 27-80](#) for a diagram.

Figure 27-80. Example of Memory-Mapped Parity Locations During Test Mode



1 Shaded areas indicate reads return 0, writes have no effect. These registers are not physically present.

NOTE: Read Access to Parity Memory Locations

Parity memory locations can be read even without entering into parity memory test mode. Their address remains as in memory test mode. It is only to enter parity-memory test mode to enable write access to the parity memory locations.

27.12 MibSPI Pin Timing Parameters

The pin timings of SPI can be classified based on its mode of operation. In each mode, different configurations like Phase and Polarity affect the pin timings.

The pin directions are based on the mode of operation.

Master mode SPI:

- SPICLK (SPI Clock) - Output
- SPISIMO (SPI Slave In Master Out) - Output
- $\overline{\text{SPIC}}\text{S}$ (SPI Slave Chip Selects) - Output
- SPISOMI (SPI Slave Out Master In) - Input
- $\overline{\text{SPIEN}}\text{A}$ (SPI slave ready Enable) - Input

Slave mode SPI:

- SPICLK - Input
- SPISIMO - Input
- $\overline{\text{SPIC}}\text{S}$ - Input
- SPISOMI - Output
- $\overline{\text{SPIEN}}\text{A}$ - Output

NOTE: All the timing diagrams given below are with Phase = 0 and Polarity = 0. unless explicitly stated otherwise.

27.12.1 Master Mode Timings for SPI/MibSPI

Figure 27-81. SPI/MibSPI Pins During Master Mode 3-pin Configuration

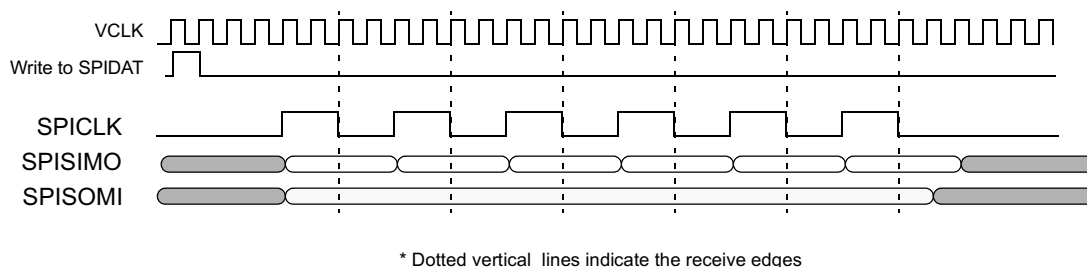


Figure 27-82. SPI/MibSPI Pins During Master Mode 4-pin with $\overline{\text{SPIC}}\text{S}$ Configuration

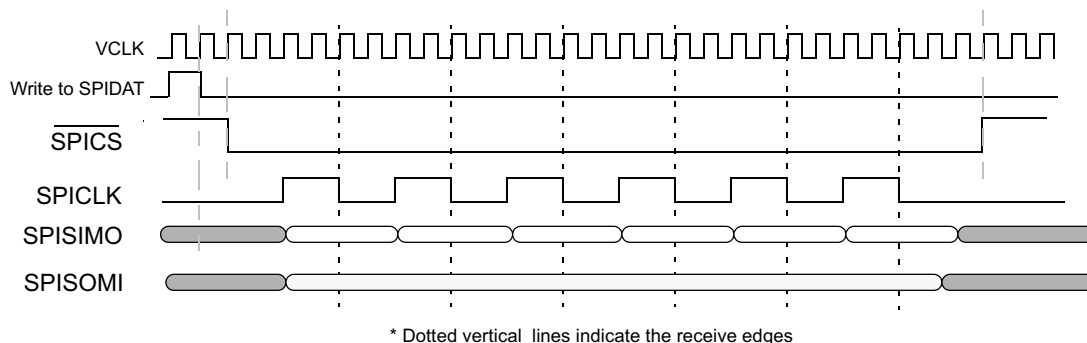


Figure 27-83. SPI/MibSPI Pins During Master Mode 4-pin with $\overline{\text{SPIEN}}$ Configuration

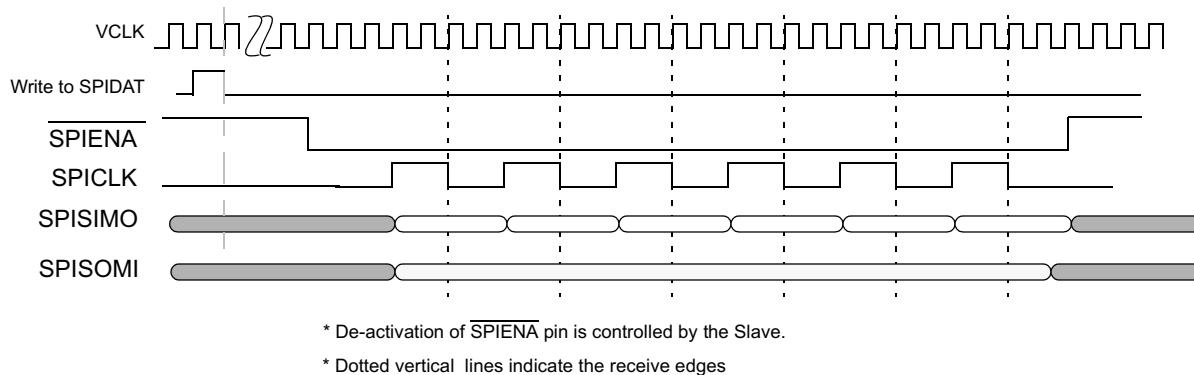
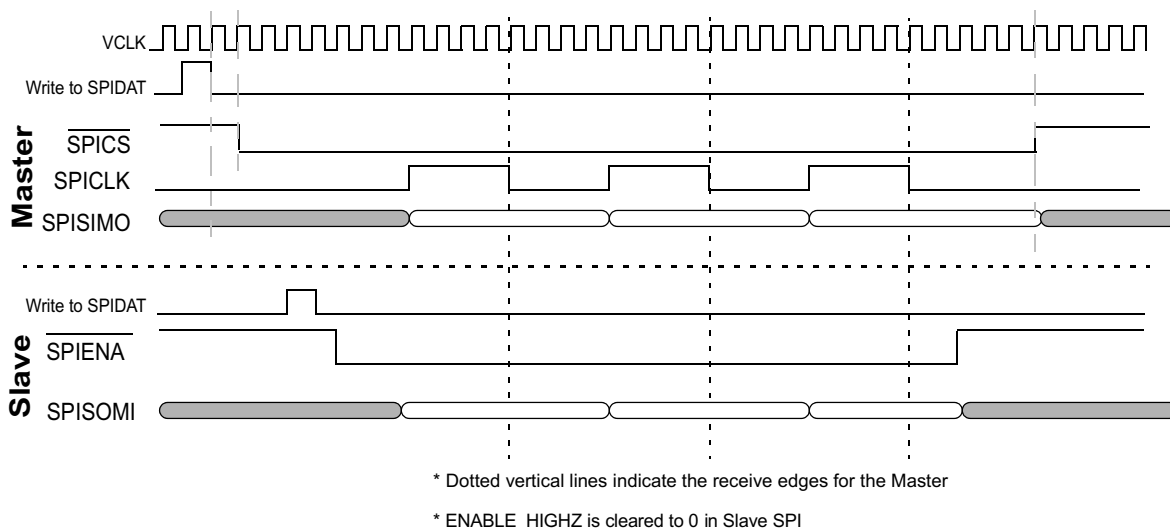


Figure 27-84. SPI/MibSPI Pins During Master/Slave Mode with 5-pin Configuration



27.12.2 Slave Mode Timings for SPI/MibSPI

Figure 27-85. SPI/MibSPI Pins During Slave Mode 3-pin Configuration

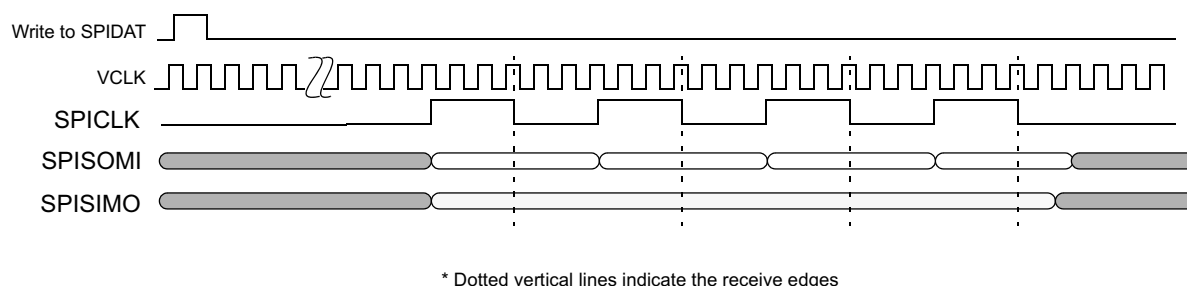


Figure 27-86. SPI/MibSPI Pins During Slave Mode 4-pin with $\overline{\text{SPIENA}}$ Configuration

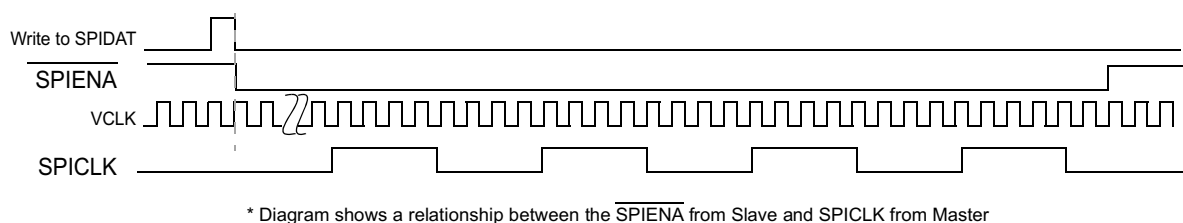


Figure 27-87. SPI/MibSPI Pins During Slave Mode in 5-pin Configuration - (Single Slave)

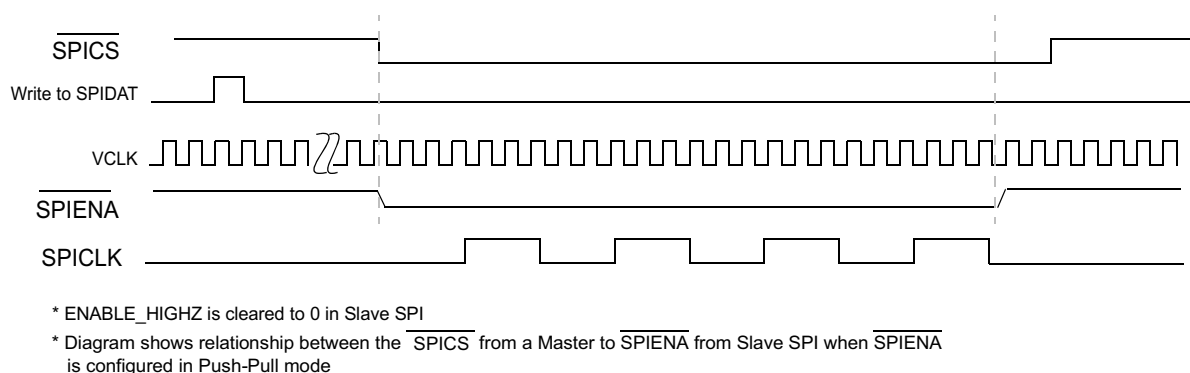
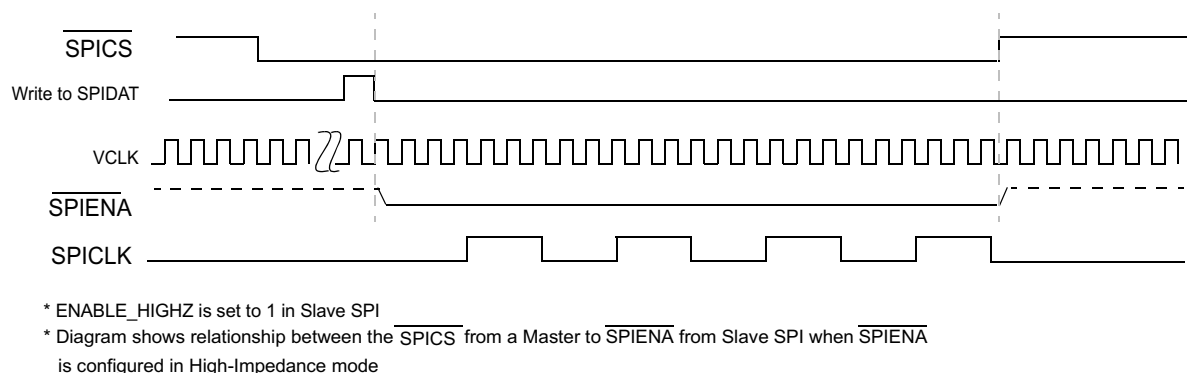


Figure 27-88. SPI/MibSPI Pins During Slave Mode in 5-pin Configuration - (Single/Multi Slave)



27.12.3 Master Mode Timing Parameter Details

In case of Master, the module drives out SPICLK. It also drives out the Transmit data on SPISIMO with respect to its internal SPICLK. In case of Master mode, the RX data on the SPISOMI pin is registered with respect to SPICLK received through the input buffer from the I/O pad.

If the chip select pin is functional, then the Master will drive out the $\overline{\text{SPICS}}$ pins before starting the SPICLK. If the $\overline{\text{SPIENA}}$ pin is functional, then the Master will wait for an active low from the Slave on the input pin to start the SPICLK.

27.12.4 Slave Mode Timing Parameter Details

In case of Slave mode, the module will drive only the SPISOMI and $\overline{\text{SPIENA}}$ pins. All other pins are inputs to it. The RX data on the SPISIMO pin will be registered with respect to the SPICLK pin. The Slave will use the $\overline{\text{SPICS}}$ pin to drive out the $\overline{\text{SPIENA}}$ pin if both are functional. If 4-pin with $\overline{\text{SPIENA}}$ is configured, then the Slave will drive out an active-low signal on the $\overline{\text{SPIENA}}$ pin when new data is written to the TX Shift Register. Irrespective of 4-pin with $\overline{\text{SPIENA}}$ or 5-pin configuration, the Slave will deassert the $\overline{\text{SPIENA}}$ pin after the last bit is received. If ENABLE_HIGHZ (SPIINT0.24) bit is 0, the de-asserted value of the $\overline{\text{SPIENA}}$ pin will be 1. Otherwise, it will depend upon the internal pull up or pull down resistor (if implemented) depending upon the Specification of the Chip.